TUGAS BESAR 1

DASAR INTELEGENSI ARTIFISIAL - IF3070



Optimalisasi *Local Search* dalam Permasalahan Pemuatan Kapal dalam Geladak Kargo

Disusun oleh *G25 – tengahtengah*:

Abel Gani	18224016
Kayta Rechia Mazaya	18224022
R. Athalla Ananda P.	18224060

INSTITUT TEKNOLOGI BANDUNG

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA - KOMPUTASI

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

DAFTAR ISI

BAB I DESKRIPSI PERSOALAN....1
1.1 Geometric Knapsack Problem....1
1.2 Local Search....2
1.3 Representasi koordinat dan ilustrasi persoalan....2
BAB II PEMBAHASAN....4
2.1 Pemilihan Inisialisasi....4
2.2 Pemilihan Objective Function (h(n))....4
2.2.1. Batasan Solusi....5
2.2.2. Komponen Objective Function....5
2.2.3. Kombinasi Skor Akhir....7
2.2.4. Implementasi Fungsi Objektif....8
2.3 Pembuatan Kelas....9
2.4 Loading Data....13
2.5 Implementasi Algoritma Local Search....14
2.5.1 Steepest Hill Climbing....14
2.5.2 Stochastic Hill Climbing....16
2.5.3 Hill-Climbing with Sideways Move....18
2.5.4 Random Restart Hill-Climbing....21
2.5.5 Simulated Annealing....23
2.5.6 Genetic Algorithm....27
2.6 Hasil Eksperimen dan Analisis....33
2.6.1. Steepest Hill Climbing....33
2.6.2. Stochastic Hill Climbing....37
2.6.3. Hill-Climbing with Sideways Move....41
2.6.4. Random Restart Hill-Climbing....45
2.6.5. Simulated Annealing....49
2.6.6. Genetic Algorithm....55
BAB III ANALISIS....75
3.1 Kedekatan dengan Global Optimum....75
3.2 Perbandingan Hasil Pencarian....77
3.3 Perbandingan Durasi Proses Pencarian....78
3.4 Konsistensi Hasil Akhir Eksperimen....79
3.5 Pengaruh Jumlah Populasi dan Banyak Generasi pada Genetic Algorithm....82
3.6 Pengaruh Temperatur Penerimaan Neighbor pada Simulated Annealing....83
BAB IV KESIMPULAN DAN SARAN....86
4.1 Kesimpulan....86
4.2 Saran....86
BAB V PEMBAGIAN TUGAS....88
REFERENSI....91
LAMPIRAN....92

DAFTAR TABEL

Tabel 2.6.1 Hasil Percobaan Steepest Hill Climbing..... 33
Tabel 2.6.2 Hasil Percobaan Stochastic Hill Climbing..... 37
Tabel 2.6.3 Hasil Percobaan Hill Climbing with Sideways Move.....41
Tabel 2.6.4 Hasil Percobaan Random Restart Hill Climbing.....45
Tabel 2.6.5 Hasil Percobaan Simulated Annealing..... 50
Tabel 2.6.6 Hasil Percobaan Genetic Algorithm.....55
Tabel 3.1. Perbandingan Algoritma dan Rata-rata Nilai Objektif Akhir..... 75
Tabel 3.3. Perbandingan Algoritma dan Rentang serta Rata-rata Durasi.....78
Tabel 3.4 Perbandingan Algoritma dan Standar Deviasi serta Koefisien Sampel..... 80

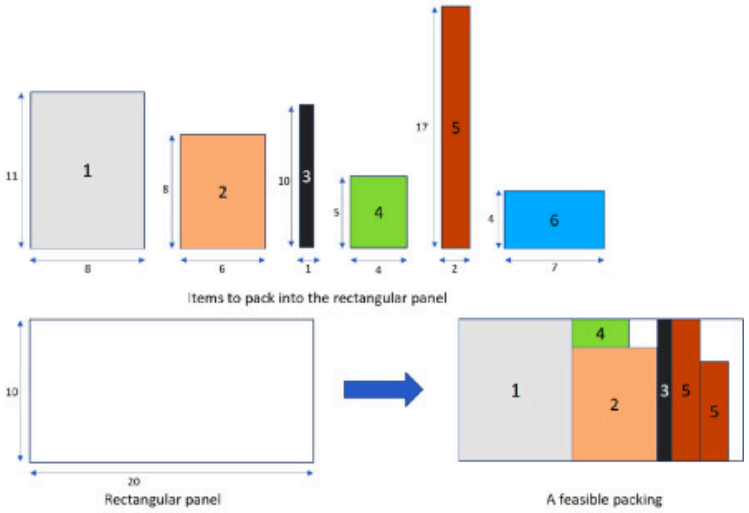
DAFTAR GAMBAR

Gambar 1.1 Ilustrasi Geometric Knapsack Problem.....1
Gambar 1.2 Representasi koordinat dan orientasi kendaraan pada geladak kapal.....3

BAB I
DESKRIPSI PERSOALAN

1.1 *Geometric Knapsack Problem*

Geometric knapsack problem merupakan permasalahan optimasi yang bertujuan memilih dan menempatkan sejumlah objek ke dalam suatu ruang berukuran tetap agar total nilai objeknya menjadi maksimum. Dalam optimasi suatu ruang berbentuk persegi panjang, setiap objek memiliki panjang, lebar, dan nilai tertentu. Penempatan setiap objek yang dipilih harus memenuhi batasan geometris, di mana semua bagian objek berada di dalam ruang dan tidak saling menumpuk. Penyelesaiannya tidak mengharuskan semua objek dimasukkan, melainkan keputusan pemilihan objek dan pengaturan posisi menjadikannya bagian dari opsi penyelesaian.



Gambar 1.1 Ilustrasi *Geometric Knapsack Problem*

Pada permasalahan yang diuraikan sesuai spesifikasi tertulis pada dokumen **Spesifikasi Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial 2026/2027** perusahaan disebut memiliki masalah pada pengangkutan kendaraan yang selama ini dilakukan secara manual. Geladak kapal direpresentasikan sebagai grid dua dimensi berbentuk persegi panjang, sementara setiap kendaraan direpresentasikan sebagai persegi dan/atau persegi panjang dengan nilai dimensi, berat, dan tarif pengiriman. Sistem diharapkan mampu menentukan kendaraan yang akan diangkut beserta koordinat dan orientasi penempatannya. Kendaraan dapat ditempatkan secara horizontal ataupun vertikal, sementara kendaraan yang tidak terpilih akan berada di luar kapal. Selain memenuhi batasan geometris tersebut, total berat kendaraan yang diangkut tidak boleh melebihi kapasitas maksimum kapal.

Tujuan penyelesaian persoalan ini adalah untuk memaksimalkan total tarif pengiriman (*ShippingFee*) dari kendaraan yang berhasil dimuat pada geladak kargo dengan batasan yang ada. Untuk mencari konfigurasi terbaik, dilakukan pencarian menggunakan algoritma *local search*.

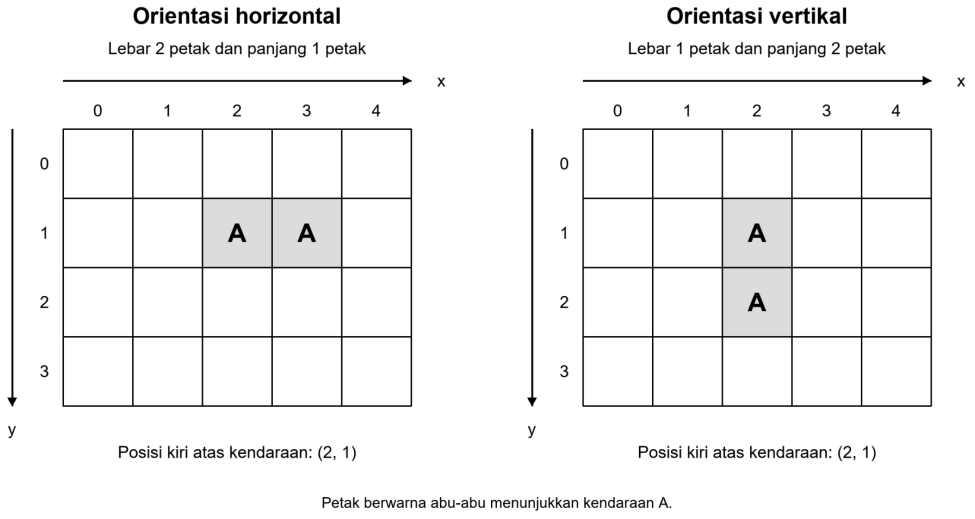
1.2 *Local Search*

Local search merupakan metode pencarian solusi yang dimulai dari suatu keadaan awal atau *initial state.* Pencarian dilakukan dengan membentuk dan mengevaluasi keadaan tetangga atau *neighbor* dari keadaan saat ini. Selanjutnya, algoritma menentukan apakah pencarian akan berpindah ke salah satu *neighbor* berdasarkan aturan yang digunakan. Pada persoalan penempatan kendaraan, satu state menggambarkan susunan kendaraan pada geladak kapal beserta kendaraan yang berada di luar kapal. *Neighbor* dibentuk dengan melakukan satu perubahan, yaitu menukar dua kendaraan, memindahkan kendaraan ke koordinat berbeda, atau mengubah orientasi kendaraan. Setiap susunan yang dihasilkan harus memenuhi batasan penempatan dan kapasitas kapal.

Algoritma *local search* memiliki cara yang berbeda dalam menentukan perpindahan ke *neighbour*. *Hill climbing* memilih perpindahan berdasarkan nilai solusi, sedangkan Simulated Annealing dapat menerima solusi yang lebih buruk dengan peluang tertentu. Proses pencarian dilakukan berulang hingga kondisi berhenti pada masing-masing algoritma terpenuhi. *Local search* tidak selalu menghasilkan solusi terbaik dari seluruh kemungkinan yang ada. Sebagai contoh, *Hill Climbing* dapat berhenti pada *local maximum,* yaitu keadaan ketika tidak ada *neighbor* dengan nilai lebih tinggi, meskipun masih terdapat solusi yang lebih baik di bagian lain ruang pencarian. Pencarian juga dapat menemui *plateau,* yaitu sekumpulan state yang memiliki nilai sama. Kondisi tersebut dapat ditangani melalui variasi pencarian, seperti mengizinkan perpindahan dengan nilai yang sama atau mengulangi pencarian dari keadaan awal yang berbeda.

1.3 Representasi koordinat dan ilustrasi persoalan

Geladak kapal dan kendaraan direpresentasikan dalam bentuk grid dua dimensi. Setiap petak menunjukkan satu satuan ukuran. Banyaknya petak yang ditempati kendaraan ditentukan oleh lebar dan panjang kendaraan tersebut. Posisi kendaraan dinyatakan dengan koordinat (x,y) yang menunjukkan petak kiri atas kendaraan. Koordinat dimulai dari (0,0) pada petak kiri atas geladak. Nilai x bertambah ke arah kanan, sedangkan nilai y bertambah ke arah bawah. Sebagai contoh, koordinat (2,1) menunjukkan petak pada kolom ketiga dan baris kedua. Kendaraan dapat ditempatkan dengan orientasi horizontal atau vertikal. Kendaraan dengan lebar dua petak dan panjang satu petak akan menempati dua petak ke arah kanan pada orientasi horizontal. Ketika orientasinya diubah menjadi vertikal, kendaraan tersebut menempati dua petak ke arah bawah. Perubahan orientasi dapat membantu menyesuaikan kendaraan dengan ruang yang tersedia pada geladak



Gambar 1.2 Representasi koordinat dan orientasi kendaraan pada geladak kapal

Sebagai ilustrasi, digunakan geladak berukuran lima kolom dan empat baris dengan kapasitas berat delapan unit. Kendaraan A memiliki berat tiga unit dan tarif pengiriman 60. Jika kendaraan B dengan berat empat unit dan tarif 40 ikut dimuat, total berat menjadi tujuh unit dan total tarif menjadi 100. Kendaraan D dengan berat satu unit dan tarif 10 masih dapat ditambahkan sehingga total berat mencapai delapan unit dan total tarif menjadi 110. Contoh ini digunakan untuk menjelaskan penempatan kendaraan dan perhitungan nilai solusi. Hasil percobaan algoritma akan dibahas setelah pengujian dilakukan.

BAB II
PEMBAHASAN

2.1 Pemilihan Inisialisasi

Inisialisasi dilakukan untuk membentuk susunan awal kendaraan sebelum proses pencarian dimulai. Sesuai spesifikasi tugas, susunan awal dibuat secara acak. Proses ini diimplementasikan melalui fungsi '`create_initial_state(ships, vehicles, seed=None)`'. Pada awal proses, seluruh geladak masih kosong dan semua kendaraan berada di luar kapal. Urutan kendaraan kemudian diacak menggunakan '`random.Random(seed)`'. Pengacakan urutan dilakukan agar kendaraan yang dicoba untuk dimuat terlebih dahulu tidak selalu sama.

Setiap kendaraan memiliki peluang sebesar 25% untuk tetap berada di luar kapal tanpa melalui proses penempatan. Peluang tersebut digunakan untuk menambah variasi susunan awal dan menyediakan kendaraan di luar kapal yang dapat dipertimbangkan selama pencarian. Angka 25% merupakan pilihan dalam implementasi. Jumlah kendaraan yang dilewati juga tidak selalu tepat 25% dari seluruh kendaraan. Untuk kendaraan yang akan dimuat, program menyusun daftar kemungkinan posisi dengan orientasi horizontal dan vertikal. Posisi yang dipertimbangkan dibatasi agar seluruh bagian kendaraan berada di dalam geladak. Daftar tersebut kemudian diacak untuk menentukan urutan percobaan penempatan.

Program mencoba menempatkan kendaraan pada salah satu posisi, lalu memeriksa susunannya menggunakan fungsi '`is_valid`'. Penempatan dipertahankan apabila kendaraan tidak bertumpuk dengan kendaraan lain dan total berat tidak melebihi kapasitas kapal. Jika penempatan tidak memenuhi batasan, kendaraan dikeluarkan dari posisi tersebut dan program mencoba pilihan berikutnya. Apabila tidak ada posisi yang sesuai, kendaraan tetap berada di luar kapal. Setelah seluruh kendaraan selesai diproses, fungsi mengembalikan state awal yang akan digunakan oleh algoritma pencarian. State tersebut dapat berisi sebagian kendaraan di dalam kapal, sementara kendaraan lainnya tetap berada di luar. Susunan awal ini belum tentu memberikan keuntungan terbesar, sehingga selanjutnya akan diperbaiki melalui proses pencarian. Parameter '`seed`' digunakan agar pengacakan dapat diulang. Penggunaan seed yang sama dengan data dan urutan masukan yang sama akan menghasilkan susunan awal yang sama. Hal ini memudahkan pengujian dan perbandingan algoritma. Pada *Random Restart,* pengacakan perlu menghasilkan variasi susunan awal pada setiap pengulangan agar pencarian tidak terus dimulai dari susunan yang sama.

2.2 Pemilihan *Objective Function* (h(n))

Objective Function digunakan untuk menilai serta mengevaluasi kualitas suatu susunan kendaraan (*state*) yang dihasilkan oleh algoritma pencarian. Solusi yang dihasilkan

diharapkan tidak hanya memberikan pendapatan pengangkutan yang tinggi, tetapi juga mempertimbangkan estimasi waktu pengiriman dan penggunaan kapasitas berat kapal. Oleh karena itu, pada tugas ini digunakan *objective function* yang menggabungkan ketiga aspek tersebut menjadi suatu skor penilaian.

Pada tugas besar ini, kelompok penulis merancang *objective function* dengan pendekatan maksimisasi, yaitu semakin besar nilai $h(n)$, semakin baik kualitas solusi yang dihasilkan. Perhitungan dilakukan berdasarkan kendaraan yang dimuat pada satu kapal. Nilai fungsi objektif diperoleh dengan membagi total tarif pengangkutan terhadap hasil perkalian faktor penalti ETA maksimum dan faktor penalti penggunaan kapasitas berat kapal. Kendaraan yang tidak dimuat tidak memberikan kontribusi terhadap skor.

2.2.1. Batasan Solusi

Sebelum nilai fungsi objektif dihitung, susunan kendaraan harus memenuhi batasan penempatan dan kapasitas kapal. Salah satu batasan utama adalah total berat seluruh kendaraan yang dimuat pada kapal tidak boleh melebihi kapasitas berat maksimum kapal. Batasan kapasitas berat dinyatakan sebagai berikut.

$$W(n) = \sum_{v \in L(n)} \text{weight}(v) \leq C$$

Dengan $L(n)$ menyatakan himpunan kendaraan yang dimuat pada kapal dalam state n, $weight(v)$ menyatakan berat kendaraan v, $W(n)$ menyatakan total berat muatan, dan C merupakan kapasitas berat maksimum kapal.

Selain kapasitas berat, solusi juga harus memenuhi kesesuaian identitas dan referensi kendaraan maupun kapal, keunikan penempatan setiap kendaraan, posisi kendaraan yang seluruhnya berada dalam batas geladak, serta larangan tumpang tindih antar kendaraan. Seluruh ketentuan tersebut diterapkan sebagai hard constraint. Solusi yang melanggar salah satu batasan dinyatakan tidak valid sehingga tidak diberikan nilai fungsi objektif.

2.2.2. Komponen *Objective Function*

Pada tugas besar ini, kelompok penulis menyusun *objective function* atas tiga komponen, yaitu pendapatan pengangkutan, faktor penalti waktu pengiriman, dan faktor penalti penggunaan kapasitas berat. Pendapatan berperan sebagai komponen yang ingin ditingkatkan, sedangkan kedua faktor penalti digunakan untuk mempertimbangkan efisiensi susunan kendaraan pada kapal. Berikut rincian penjelasan untuk masing-masing komponen tersebut.

1. **Pendapatan Pengangkutan**

Pendapatan pengangkutan dihitung berdasarkan jumlah tarif pengiriman seluruh kendaraan yang ditempatkan pada kapal. Secara matematis, komponen ini dirumuskan sebagai berikut.

$$P(n) = \sum_{v \in L(n)} \text{shipping_fee}(v)$$

Dengan $L(n)$ menyatakan himpunan kendaraan yang dimuat pada kapal dalam state n, dan $shipping_fee$menyatakan tarif pengangkutan kendaraan v.

Semakin besar jumlah tarif kendaraan yang dimuat, semakin tinggi skor yang dapat diperoleh kapal apabila komponen lainnya tetap. Nilai tarif mengikuti satuan data masukan, yaitu juta rupiah.

2. Penalti Waktu Pengiriman

Waktu pengiriman diperhitungkan menggunakan nilai *Estimated Time of Arrival* (ETA) terbesar dari seluruh kendaraan yang dimuat pada kapal. Nilai maksimum ini hanya dihitung ketika kapal memiliki muatan. Nilai tersebut digunakan sebagai pendekatan terhadap waktu penyelesaian pengiriman seluruh muatan kapal, dengan asumsi bahwa ETA kendaraan dihitung dari acuan waktu yang sama. Komponen ini dirumuskan sebagai berikut.

$$T(n) = \max_{v \in L(n)} \text{eta}(v)$$

Selanjutnya, faktor penalti waktu pengiriman ditentukan melalui persamaan berikut.

$$D_T(n) = 1 + T(n)$$

Semakin besar ETA maksimum, semakin besar faktor penalti waktu yang dikenakan. Dengan demikian, algoritma didorong untuk memilih susunan kendaraan dengan estimasi penyelesaian pengiriman yang lebih singkat apabila pendapatan dan penggunaan kapasitas beratnya setara. Penambahan konstanta 1 bertujuan menjaga penyebut tetap positif ketika ETA bernilai nol. Dalam model ini, konstanta tersebut menggunakan skala satu hari dan merupakan bagian dari keputusan pemodelan.

3. Penalti Penggunaan Kapasitas Berat

Komponen ketiga mempertimbangkan proporsi berat kendaraan terhadap kapasitas berat maksimum kapal. Total berat kendaraan yang dimuat dihitung menggunakan persamaan berikut.

$$W(n) = \sum_{v \in L(n)} \text{weight}(v)$$

Total berat tersebut kemudian dinormalisasi terhadap kapasitas maksimum kapal untuk memperoleh rasio penggunaan kapasitas berat.

$$U(n) = \begin{cases} \dfrac{W(n)}{C}, & C > 0, \\ 0, & C = 0 \text{ pada state valid.} \end{cases}$$

Dengan C menyatakan kapasitas berat maksimum kapal. Pada state valid, kapal dengan kapasitas berat nol hanya dapat memuat kendaraan dengan total berat nol. Untuk kondisi tersebut, rasio penggunaan kapasitas didefinisikan sebesar nol agar perhitungan tetap terdefinisi dan sesuai dengan implementasi. Faktor penalti penggunaan kapasitas berat selanjutnya dirumuskan sebagai berikut.

$$D_W(n) = 1 + U(n)$$

Pada state yang memenuhi batas kapasitas dan memiliki $C > 0$, rasio $U(n)$ berada pada rentang 0 hingga 1. Semakin tinggi rasio tersebut, semakin besar penalti yang diberikan. Pemilihan komponen ini tidak mencerminkan pengukuran langsung terhadap biaya operasional kapal ataupun luas geladak yang tersisa melainkan menjadi preferensi model terhadap muatan yang relatif lebih ringan ketika pendapatan dan ETA sama.

2.2.3. Kombinasi Skor Akhir

Ketiga komponen digabungkan dengan menempatkan pendapatan pengangkutan sebagai pembilang, sedangkan faktor penalti waktu dan penggunaan kapasitas berat sebagai penyebut. Untuk kapal yang memiliki muatan, fungsi objektif dirumuskan sebagai berikut.

$$h(n) = \frac{P(n)}{(1 + T(n))\,(1 + U(n))}$$

Rasio $U(n)$ mengikuti definisi pada Subbab 2.2.2 sehingga rumus ini berlaku pula pada state valid dengan kapasitas kapal nol. Jika tidak terdapat kendaraan yang

dimuat, nilai fungsi objektif didefinisikan sebagai $h(n) = 0$ dan ETA maksimum tidak dihitung. Untuk kapal dengan kapasitas positif ($C > 0$), rumus tersebut dapat dinyatakan dalam bentuk berikut.

$$h(n) = \frac{P(n)}{(1 + T(n))\left(1 + \frac{W(n)}{C}\right)}, \qquad C > 0,\ L(n) \neq \varnothing$$

Sedangkan fungsi objektif untuk kapal kosong adalah sebagai berikut.

$$h(n) = 0, \qquad L(n) = \varnothing$$

Berdasarkan rumusan tersebut, solusi dengan pendapatan lebih tinggi akan memperoleh nilai objektif lebih besar apabila kedua faktor penalti tetap. Sebaliknya, peningkatan ETA maksimum ataupun rasio penggunaan kapasitas berat dapat menurunkan skor. Dengan demikian, penambahan kendaraan tidak selalu meningkatkan nilai fungsi objektif meskipun menghasilkan tambahan pendapatan pengangkutan.

2.2.4. Implementasi Fungsi Objektif

Implementasi fungsi objektif pada berkas '`src/core/objective.py`' ditunjukkan sebagai berikut.

```
def objective_2(state: State):
    if not is_valid(state):
        raise ValueError("Nilai objective tidak bisa dihitung untuk state
tidak valid")

    value = 0.0
    for item in state.ships:
        vehicles = [state.vehicles[placement.vehicle_id] for placement in
item.placements]
        if not vehicles:
            continue
        shipping_fee = sum(vehicle.shipping_fee for vehicle in vehicles)
        weight = sum(vehicle.weight for vehicle in vehicles)
        max_eta = max(vehicle.eta for vehicle in vehicles)
        weight_utilization = weight / item.ship.max_capacity if
item.ship.max_capacity else 0.0
        value += shipping_fee / (1 + max_eta) / (1 + weight_utilization)
    return value
```

Fungsi terlebih dahulu memeriksa validitas state, kemudian menghitung total tarif pengangkutan, total berat, dan ETA maksimum kendaraan yang dimuat pada kapal. Ketiga nilai tersebut digunakan untuk menghasilkan nilai akhir $h(n)$. Pada cakupan permasalahan ini, state hanya memuat satu kapal sehingga perulangan pada implementasi memproses satu kapal tersebut. Jika kapal tidak memiliki muatan,

perhitungan ETA dilewati dan fungsi mengembalikan nilai nol. Skor ini digunakan oleh algoritma pencarian untuk membandingkan kualitas state.

2.3 Pembuatan Kelas

Representasi persoalan dalam konteks pemilihan penempatan kendaraan pada geladak kargo dibentuk menggunakan kelas `Vehicle, Ship, Placement, ShipState,` dan `State.`

```
class Vehicle:

    id: str
    width: int
    length: int
    shipping_fee: float
    weight: float
    eta: int

    def __post_init__(self):
        if not isinstance(self.id, str) or not self.id:
            raise ValueError("Identitas kendaraan harus berupa teks yang
tidak kosong")
        if type(self.width) is not int or type(self.length) is not int:
            raise ValueError("Ukuran kendaraan harus bilangan bulat")
        if self.width <= 0 or self.length <= 0:
            raise ValueError("Ukuran kendaraan harus positif")
        if any(type(value) not in (int, float) or not isfinite(value) or
value < 0
               for value in (self.weight, self.shipping_fee)):
            raise ValueError("Berat dan tarif kendaraan harus bilangan
tidak negatif yang terbatas")
        if type(self.eta) is not int or self.eta < 0:
            raise ValueError("ETA kendaraan harus bilangan bulat tidak
negatif")
```

Kelas `Vehicle` menyimpan data setiap kendaraan melalui atribut yang terdefinisi diatas. Lebar dan panjang kendaraan dinyatakan dalam satuan sel grid, menyesuaikan pendefinisian ukuran geladak pula. Ketika objek dibuat, program melakukan pemeriksaan terhadap identitas kendaraan harus berupa teks yang tidak kosong, ukuran berupa bilangan bulat positif, serta berat dan tarif pengangkutan berupa bilangan tidak negatif yang berhingga. Atribut `eta` juga diperiksa untuk memastikan nilainya tidak negatif.

```
class Ship:

    id: str
    width: int
    length: int
    max_capacity: float

    def __post_init__(self):
        if not isinstance(self.id, str) or not self.id:
            raise ValueError("Identitas kapal harus berupa teks yang tidak
kosong")
        if type(self.width) is not int or type(self.length) is not int:
            raise ValueError("Ukuran kapal harus bilangan bulat")
        if self.width <= 0 or self.length <= 0:
            raise ValueError("Ukuran kapal harus positif")
        if (type(self.max_capacity) not in (int, float)
                or not isfinite(self.max_capacity) or self.max_capacity <
0):
```

```
            raise ValueError("Kapasitas kapal harus bilangan tidak negatif
yang terbatas")
```

Kelas `Ship` menyimpan identitas kapal, lebar dan panjang geladak, serta kapasitas berat maksimum melalui atribut terdefinisi. Validasi pada kelas ini dilakukan untuk memastikan ukuran geladak berupa bilangan bulat positif, sementara kapasitas harus bernilai positif berhingga.

```
class Placement:

    vehicle_id: str
    ship_id: str
    x: int
    y: int
    orientation: str = "horizontal"

    def __post_init__(self):
        if not self.vehicle_id or not self.ship_id:
            raise ValueError("Identitas kendaraan dan kapal harus diisi")
        if type(self.x) is not int or type(self.y) is not int:
            raise ValueError("Koordinat harus bilangan bulat")
        if self.orientation not in ("horizontal", "vertical"):
            raise ValueError("Orientasi harus horizontal atau vertical")

    def occupied_size(self, vehicle_width: int, vehicle_length: int):
        if self.orientation == "vertical":
            return vehicle_length, vehicle_width
        return vehicle_width, vehicle_length

    def occupied_cells(self, vehicle: Vehicle):

        width, length = self.occupied_size(vehicle.width, vehicle.length)
        return {(x, y) for x in range(self.x, self.x + width)
                for y in range(self.y, self.y + length)}
```

Kelas `Placement` mewakili penempatan satu kendaraan pada suatu geladak kapal. Pada bagian ini didefinisikan koordinat `x`, `y`, dan `orientation`. Koordinat tersebut menjadikan sel kiri sebagai titik acuan posisi kendaraan pada grid geladak, sementara orientasi penempatan menyimpan posisi kendaraan horizontal sebagai nilai bawaan.

Pada kelas ini juga didefinisikan metode `occupied_size` untuk menentukan ukuran yang digunakan kendaraan berdasarkan orientasinya. Pada orientasi horizontal, ukuran mengikuti lebar dan panjang asli kendaraan. Berbeda halnya pada orientasi vertikal, kedua koordinat yang terdefinisi ditukar. Selanjutnya, metode `occupied_method` menghasilkan koordinat sel yang akan ditempati kendaraan. Nantinya kumpulan sel ini akan digunakan dalam pemeriksaan batasan tumpang tindih dan pembentukan grid pada geladak.

```
class ShipState:

    ship: Ship
    placements: list[Placement] = field(default_factory=list)
```

Kelas `ShipState` menjadi titik penggabungan satu objek `Ship` dan daftar `placements`. Daftar tersebut menyimpan kendaraan yang sedang dimuat pada kapal beserta posisi dan orientasi penempatannya. Karakteristik kapal akan disimpan dalam `Ship`, sementara informasi susunan muatan akan disimpan dalam `ShipState`.

```
class State:

    ships: list[ShipState] = field(default_factory=list)
    vehicles: dict[str, Vehicle] = field(default_factory=dict)

    def copy(self):
        return deepcopy(self)

    def outside_vehicle_ids(self):
        loaded = {placement.vehicle_id for ship in self.ships for placement
in ship.placements}
        return set(self.vehicles) - loaded

    def deck_grid(self, ship_id):
        ship_state = next((item for item in self.ships if item.ship.id ==
ship_id), None)
        if ship_state is None:
            raise KeyError(ship_id)
        from .constraint import is_valid

        if not is_valid(self):
            raise ValueError("Grid hanya bisa ditampilkan dari state
valid")
        grid = [[None for _ in range(ship_state.ship.width)]
                for _ in range(ship_state.ship.length)]
        for placement in ship_state.placements:
            for x, y in
placement.occupied_cells(self.vehicles[placement.vehicle_id]):
                grid[y][x] = placement.vehicle_id
        return grid

    def to_dict(self):
        from .objective import objective_1

        return {
            "vehicles": [vars(vehicle).copy() for vehicle in
self.vehicles.values()],
            "ships": [
                {
                    "ship": vars(item.ship).copy(),
                    "placements": [vars(placement).copy() for placement in
item.placements],
                    "grid": self.deck_grid(item.ship.id),
                }
                for item in self.ships
            ],
            "outside_vehicle_ids": sorted(self.outside_vehicle_ids),
            "value": objective_1(self),
        }

    def from_dict(cls, data):
        vehicles = [Vehicle(**vehicle) for vehicle in data["vehicles"]]
        ships = []
        for item in data["ships"]:
            ship_data = item["ship"] if "ship" in item else {
                key: value for key, value in item.items() if key !=
"placements"
            }
```

```
            placements = [Placement(**placement) for placement in
item.get("placements", [])]
            ships.append(ShipState(Ship(**ship_data), placements))
        if len({vehicle.id for vehicle in vehicles}) != len(vehicles):
            raise ValueError("Identitas kendaraan harus unik")
        if len({item.ship.id for item in ships}) != len(ships):
            raise ValueError("Identitas kapal harus unik")
        if not ships:
            raise ValueError("Minimal satu kapal diperlukan")
        state = cls(ships, {vehicle.id: vehicle for vehicle in vehicles})
        from .constraint import is_valid

        if not is_valid(state):
            raise ValueError("Placement awal tidak memenuhi aturan")
        return state


def load_problem(path):
    with Path(path).open(encoding="utf-8") as file:
        return State.from_dict(json.load(file))


def create_initial_state(ships, vehicles, seed=None):
    rng = random.Random(seed)
    state = State([ShipState(ship) for ship in ships], {v.id: v for v in
vehicles})
    if len(state.vehicles) != len(vehicles) or len({ship.id for ship in
ships}) != len(ships):
        raise ValueError("Identitas kendaraan dan kapal harus unik")
    if not ships:
        raise ValueError("Minimal satu kapal diperlukan")

    from .constraint import is_valid

    pending = list(vehicles)
    rng.shuffle(pending)
    for vehicle in pending:
        # Sisakan beberapa kendaraan di luar agar pencarian punya ruang
untuk memperbaiki solusi
        if rng.random() < 0.25:
            continue
        choices = []
        for ship_state in state.ships:
            for orientation in ("horizontal", "vertical"):
                width, length = Placement(vehicle.id, ship_state.ship.id,
0, 0, orientation).occupied_size(
                    vehicle.width, vehicle.length
                )
                for x in range(ship_state.ship.width - width + 1):
                    for y in range(ship_state.ship.length - length + 1):
                        choices.append((ship_state, Placement(vehicle.id,
ship_state.ship.id, x, y, orientation)))
        rng.shuffle(choices)
        for ship_state, placement in choices:
            ship_state.placements.append(placement)
            if is_valid(state):
                break
            ship_state.placements.pop()
    return state
```

Kelas `State` merepresentasikan satu bentuk solusi secara keseluruhan. Kelas ini menyimpan daftar '`ShipState`' melalui atribut `ships` dan seluruh data kendaraan berdasarkan identitas kendaraan melalui `vehicles`. Kendaraan yang tidak memiliki penempatan akan tetap tercatat dalam kamus. Daftar identitas kendaraan di luar kapal akan

diperoleh melalui `outside_vehicle_ids`, yakni selisih antara seluruh identitas kendaraan dan identitas kendaraan yang telah dimuat.

Untuk membantu proses pencarian, kelas `State` menyediakan metode `copy` yang menghasilkan salinan state sehingga memungkinkan perubahan pada kandidat solusi tidak mengubah state asal. Pada kelas ini, terdapat metode `deck_grid` yang membentuk representasi geladak berupa grid dua dimensi yang berisi identitas kendaraan pada sel yang ditempatinya, sementara sel kosong didefinisikan sebagai `None.` Untuk mengubah state menjadi struktur data, digunakan metode `to_dict` yang memuat data kendaraan, kapal, penempatan, grid, kendaraan di luar kapal, dan nilai objektif.

2.4 Loading Data

Data persoalan disimpan dalam format JSON pada file `small.json` dengan dua komponen utama, yaitu `ships` dan `vehicles` . Komponen `ships` memuat karakteristik kapal, sementara `vehicles` memuat data kendaraan yang tersedia untuk dimuat ke kapal. Data sampel dapat dilihat pada berikut:

```
{
  "ships": [
    {"id": "ship_1", "length": 6, "width": 7, "max_capacity": 15}
  ],
  "vehicles": [
    {"id": "A", "eta": 2, "length": 2, "width": 2, "weight": 3,
"shipping_fee": 60},
    {"id": "B", "eta": 1, "length": 3, "width": 1, "weight": 4,
"shipping_fee": 40},
    {"id": "C", "eta": 3, "length": 1, "width": 2, "weight": 2,
"shipping_fee": 30},
    {"id": "D", "eta": 0, "length": 1, "width": 1, "weight": 1,
"shipping_fee": 10},
    {"id": "E", "eta": 3, "length": 2, "width": 4, "weight": 2,
"shipping_fee": 70},
    {"id": "F", "eta": 4, "length": 1, "width": 3, "weight": 2,
"shipping_fee": 35},
    {"id": "G", "eta": 2, "length": 4, "width": 1, "weight": 5,
"shipping_fee": 55},
    {"id": "H", "eta": 5, "length": 2, "width": 3, "weight": 4,
"shipping_fee": 80},
    {"id": "I", "eta": 1, "length": 3, "width": 2, "weight": 3,
"shipping_fee": 65},
    {"id": "J", "eta": 0, "length": 1, "width": 2, "weight": 1,
"shipping_fee": 20},
    {"id": "K", "eta": 4, "length": 2, "width": 1, "weight": 2,
"shipping_fee": 25},
    {"id": "L", "eta": 3, "length": 3, "width": 3, "weight": 5,
"shipping_fee": 90},
    {"id": "M", "eta": 2, "length": 1, "width": 4, "weight": 3,
"shipping_fee": 45},
    {"id": "N", "eta": 5, "length": 2, "width": 2, "weight": 2,
"shipping_fee": 50},
    {"id": "O", "eta": 1, "length": 4, "width": 2, "weight": 4,
"shipping_fee": 75},
    {"id": "P", "eta": 6, "length": 6, "width": 1, "weight": 5,
"shipping_fee": 85},
    {"id": "Q", "eta": 4, "length": 1, "width": 7, "weight": 3,
"shipping_fee": 95},
```

```
    {"id": "R", "eta": 2, "length": 5, "width": 2, "weight": 6,
"shipping_fee": 110},
    {"id": "S", "eta": 0, "length": 3, "width": 4, "weight": 6,
"shipping_fee": 120},
    {"id": "T", "eta": 6, "length": 2, "width": 5, "weight": 3,
"shipping_fee": 100},
    {"id": "U", "eta": 3, "length": 7, "width": 1, "weight": 2,
"shipping_fee": 60},
    {"id": "V", "eta": 7, "length": 3, "width": 3, "weight": 2,
"shipping_fee": 105},
    {"id": "W", "eta": 7, "length": 1, "width": 1, "weight": 1,
"shipping_fee": 18},
    {"id": "X", "eta": 5, "length": 2, "width": 6, "weight": 7,
"shipping_fee": 130},
    {"id": "Y", "eta": 2, "length": 4, "width": 3, "weight": 4,
"shipping_fee": 70}
  ]
}
```

Pembacaan data dilakukan melalui fungsi `load_problem(path)`. Fungsi tersebut membuka berkas dengan encoding, membaca isinya melalui `json.load`, kemudian meneruskan data ke `State.from_dict`. Pendekatan ini akan membentuk objek `Vehcile` untuk setiap kendaraan, serta objek `Ship` dan `ShipState` untuk setiap kapal.

Pada *web app*, data sampel dibaca melalui *endpoint* **GET /api/sample**. Pengguna dapat memuat data kapal dan kendaraan dengan mengunggah berkas JSON. Isi berkas ditampilkan pada kolom dataset dan dapat langsung diubah sebelum proses optimasi berlangsung. Ketika optimasi dijalankan, isi kolom dikonversi menjadi objek JSON dan dikirim sebagai bagian dari *request* ke *endpoint* **POST /api/optimize**.

2.5 Implementasi Algoritma *Local Search*

2.5.1 Steepest Hill Climbing

a. Deskripsi Umum

Steepest Ascent Hill Climbing merupakan algoritma yang memeriksa seluruh neighbor dari state saat ini untuk memilih neighbor dengan nilai objektif tertinggi. Perpindahan hanya dilakukan apabila nilai objektif neighbor tersebut lebih tinggi daripada nilai objektif state saat ini.

Pencarian dilakukan berulang hingga tidak ditemukan neighbor yang memberikan peningkatan atau batas iterasi tercapai. Algoritma ini tidak menjamin diperolehnya global optimum karena dapat berhenti pada local maximum. Selain itu, algoritma tidak menerima perpindahan dengan nilai objektif yang sama, sehingga tidak dapat melanjutkan pencarian melalui langkah setara pada plateau.

b. Proses Pencarian dalam *Geometric Knapsack Problem*

Berikut merupakan tahapan pencarian menggunakan *Steepest Ascent Hill Climbing* :

1. State awal dibentuk secara acak dan diberikan kepada algoritma setelah memenuhi seluruh batasan.
2. Algoritma menghitung nilai objektif state awal berdasarkan total `ShippingFee` kendaraan yang diangkut.
3. Seluruh neighbor dari state saat ini diperiksa dan dihitung nilai objektifnya.
4. Algoritma memilih neighbor dengan nilai objektif tertinggi yang lebih besar daripada nilai objektif state saat ini.
5. Jika neighbor tersebut ditemukan, state saat ini diganti dengan neighbor yang dipilih.
6. Jumlah iterasi ditambahkan dan nilai objektif state saat ini dicatat.
7. Pencarian diulang hingga tidak ada neighbor yang meningkatkan nilai objektif atau batas iterasi tercapai.

c. Kelas/Metode

Algoritma diimplementasikan melalui fungsi `steepest_ascent` pada berkas `steepest.py`. Fungsi menerima `initial_state`, `objective_function`, `neighbor_generator`, dan parameters.

Inisialisasi

```
parameters = {} if parameters is None else parameters
max_iterations = validate_limit(parameters, "max_iterations", 1000)

started = perf_counter()
initial_snapshot = initial_state.copy()
current = initial_snapshot.copy()
current_value = objective_function(current)
history = [current_value]
iterations = 0
```

Bagian inisialisasi menyiapkan batas iterasi dan variabel yang digunakan selama pencarian. Parameter `max_iterations` menentukan jumlah maksimum iterasi. Nilainya diperiksa melalui `validate_limit` agar berupa bilangan bulat positif. Initial state disalin ke dalam initial_snapshot untuk menyimpan susunan awal kendaraan. Salinan berikutnya disimpan dalam current sebagai state yang digunakan selama pencarian. Nilai objektifnya dihitung melalui `objective_function` dan disimpan dalam `current_value`. Daftar history diawali dengan nilai objektif initial state, sedangkan iterations dimulai dari nol untuk menghitung jumlah iterasi. Variabel started mencatat waktu awal untuk pengukuran durasi pencarian.

Proses Pencarian

```
while iterations < max_iterations:
    best_neighbor = None
    best_neighbor_value = current_value
```

```
    for neighbor in neighbor_generator(current):
        value = objective_function(neighbor)
        if value > best_neighbor_value:
            best_neighbor = neighbor
            best_neighbor_value = value

    iterations += 1
    if best_neighbor is not None:
        current = best_neighbor
        current_value = best_neighbor_value
    history.append(current_value)
    if best_neighbor is None:
        break

return build_result(initial_snapshot, current, current_value,
history,iterations, perf_counter() - started)
```

Pada setiap iterasi, algoritma memeriksa seluruh neighbor yang dihasilkan oleh `neighbor_generator`. Variabel `best_neighbor` menyimpan neighbor yang dipilih, sedangkan `best_neighbor_value` menyimpan nilai objektifnya. Nilai pembanding awal menggunakan `current_value` sehingga hanya neighbor dengan nilai objektif lebih tinggi yang dapat dipilih. Jika ditemukan neighbor yang memberikan peningkatan, `current` dan `current_value` diperbarui. Apabila terdapat beberapa neighbor dengan nilai objektif tertinggi yang sama, algoritma mempertahankan neighbor yang ditemukan terlebih dahulu. Jumlah iterasi ditambahkan setelah pemeriksaan seluruh neighbor selesai. Nilai objektif state saat ini kemudian dicatat dalam history. Jika tidak ditemukan peningkatan, state tidak berubah dan pencarian dihentikan. Putaran pemeriksaan terakhir tetap dihitung meskipun tidak terjadi perpindahan. Setelah pencarian selesai, `build_result` menyusun hasil pencarian. Durasi dihitung dari selisih pembacaan `perf_counter` pada akhir dan awal proses, dengan satuan detik.

2.5.2 Stochastic Hill Climbing

a. Deskripsi Umum

Stochastic Hill Climbing merupakan variasi *Hill Climbing* yang mencoba satu neighbor secara acak pada setiap iterasi. Neighbor diterima apabila nilai objektifnya lebih tinggi daripada nilai objektif state saat ini. Jika neighbor memiliki nilai objektif yang sama atau lebih rendah, state saat ini dipertahankan. Algoritma kemudian mencoba kembali pada iterasi berikutnya hingga batas iterasi tercapai atau tidak terdapat neighbor valid.

Pemilihan secara acak menghasilkan urutan percobaan yang dapat berbeda pada setiap pencarian. Namun, algoritma tetap dapat berhenti tanpa mencapai global optimum karena hanya menerima peningkatan nilai objektif. Pengacakan tersebut juga tidak memungkinkan algoritma melewati plateau melalui perpindahan bernilai sama.

b. Proses Pencarian dalam *Geometric Knapsack Problem*

Berikut merupakan tahapan pencarian menggunakan *Stochastic Hill Climbing*:

1. State awal yang valid diberikan kepada algoritma.
2. Nilai objektif state awal dihitung dan dicatat.
3. Algoritma mengambil satu neighbor valid secara acak.
4. Nilai objektif neighbor dibandingkan dengan nilai objektif state saat ini.
5. Jika nilai objektif neighbor lebih tinggi, state saat ini diganti dengan neighbor tersebut.
6. Jika nilai objektif neighbor sama atau lebih rendah, state saat ini dipertahankan.
7. Jumlah iterasi ditambahkan dan nilai objektif setelah percobaan dicatat.
8. Pencarian diulang hingga batas iterasi tercapai atau tidak ada neighbor valid yang dapat dipilih.

Penolakan satu neighbor tidak langsung menghentikan pencarian karena neighbor lain masih mungkin memberikan peningkatan.

c. Kelas/Metode

Algoritma diimplementasikan melalui fungsi `stochastic_hill_climbing` pada berkas *stochastic.py*. Fungsi menerima `initial_state, objective_function, neighbor_generator`, dan parameters.

Inisialisasi

```
parameters = {} if parameters is None else parameters
max_iterations = validate_limit(parameters, "max_iterations", 1000)
rng = random.Random(parameters.get("seed"))

started = perf_counter()
initial_snapshot = initial_state.copy()
current = initial_snapshot.copy()
current_value = objective_function(current)
history = [current_value]
iterations = 0
```

Bagian inisialisasi menyiapkan batas iterasi, pengacakan, serta pencatatan proses pencarian. Parameter `max_iterations` membatasi jumlah percobaan neighbor yang dilakukan. Pengacakan menggunakan random. Random dengan *seed* yang dapat diberikan melalui parameter. *Initial state* disalin untuk menyimpan susunan awal dan membentuk state pencarian. Nilai objektif state awal disimpan dalam `current_value` dan dimasukkan sebagai elemen pertama *history*. Variabel *iterations* digunakan untuk menghitung jumlah percobaan neighbor. Variabel `started` digunakan sebagai waktu awal pengukuran durasi pencarian.

Proses Pencarian

```
while iterations < max_iterations:
    neighbor = None
    if neighbor_generator is generate_neighbors or
neighbor_generator is random_neighbor:
        try:
            neighbor = random_neighbor(current, rng)
        except NoNeighborError:
            pass
    else:
        for count, candidate in
enumerate(neighbor_generator(current), start=1):
            if rng.randrange(count) == 0:
                neighbor = candidate
    iterations += 1
    if neighbor is None:
        history.append(current_value)
        break
    neighbor_value = objective_function(neighbor)
    if neighbor_value > current_value:
        current = neighbor
        current_value = neighbor_value
    history.append(current_value)

return build_result(initial_snapshot, current, current_value,
history,iterations, perf_counter() - started)
```

Pada setiap iterasi, algoritma mencoba memperoleh satu neighbor secara acak. Ketika menggunakan pembuat neighbor dari core, pemilihan dilakukan melalui `random_neighbor`. Fungsi tersebut mengacak usulan perubahan dan mengembalikan neighbor valid pertama yang ditemukan. Dengan cara ini, algoritma tidak perlu membentuk seluruh salinan state neighbor sebelum memilih satu kandidat. Jika tidak terdapat neighbor valid, random_neighbor menghasilkan `NoNeighborError`. Setelah kesalahan tersebut ditangani, neighbor tetap bernilai `None`. Algoritma kemudian mencatat jumlah iterasi dan nilai objektif terakhir sebelum menghentikan pencarian. Bagian *else* digunakan untuk mendukung pembuat neighbor lain yang menghasilkan kumpulan kandidat. Algoritma menelusuri kumpulan tersebut dan memilih satu kandidat secara acak tanpa menyimpan seluruh kandidat dalam daftar. Setelah neighbor diperoleh, nilai objektifnya disimpan dalam `neighbor_value` dan dibandingkan dengan `current_value`. Neighbor diterima hanya jika nilai objektifnya lebih tinggi. Jika tidak, state saat ini dipertahankan dan percobaan dilanjutkan. Nilai objektif setelah setiap percobaan dicatat dalam history. Nilai objektif yang berulang dapat menunjukkan bahwa neighbor yang dicoba ditolak, meskipun pencarian masih berjalan. Hasil pencarian dikembalikan melalui `build_result`.

2.5.3 Hill-Climbing with Sideways Move

a. Deskripsi Umum

Hill Climbing with Sideways Move merupakan variasi *Hill Climbing* yang mnegizinkan perpindahan ke neighbor dengan nilai objektif yang sama. Perpindahan

tersebut dipertimbangkan ketika tidak terdapat neighbor dengan nilai objektif yang lebih tinggi. Pada persoalan penempatan kendaraan, pemindahan posisi atau perubahan orientasi dapat mengubah bentuk ruang kosong tanpa mengubah total `ShippingFee`. Susunan baru tersebut dapat memberikan kesempatan untuk memuat kendaraan lain pada iterasi berikutnya.

Jumlah *sideways move* dibatasi melalui `maximum_sideways_move`. Batas tersebut diperlukan karena perpindahan bernilai sama tidak selalu mengarah pada peningkatan dan dapat membawa pencarian kembali susunan yang pernah dikunjungi.

b. Proses Pencarian dalam Geometric Knapsack Problem

Berikut merupakan tahapan pencarian menggunakan *Hill Climbing with Sideways Move* :

1. State awal yang valid diberin kepada algoritma dan nilai objektifnya dihitung.
2. Seluruh neighbor dari state saat ini diperiksa.
3. Neighbor dikelompokkan menjadi neighbor dengan nilai objektif lebih tinggi dan neighbor dengan nilai objektif sama.
4. Jika terdapat peningkatan, algoritma memilih neighbor dengan nilai objektif lebih tinngi dan neighbor dengan nilai objektif sama.
5. Jika terdapat peningkatan, algoritma dapat memilih satu neighbor bernilai sama secara acak selama batas sideways belum tercapai.
6. Jumlah iterasi dan nilai objektif setelah setiap putaran dicatat.
7. Pencarian dihentikan ketika batas sideways tercapai, batas iterasi tercapai, atau tidak terdapat neighbour yang dapat diterima.

c. Kelas/Metode

Algoritma diimplementasikan melalui fungsi `sideways_move` pada berkas *sideways.py.* Fungsi menerima `initial_state`, `objective_function`, `neighbor_generator`, dan `parameters.`

Inisialisasi

```
parameters = {} if parameters is None else parameters
max_iterations = validate_limit(parameters, "max_iterations", 1000)
maximum_sideways_move = validate_limit(
    parameters, "maximum_sideways_move", 100, minimum=0
)
rng = random.Random(parameters.get("seed"))

started = perf_counter()
initial_snapshot = initial_state.copy()
current = initial_snapshot.copy()
current_value = objective_function(current)
history = [current_value]
iterations = 0
sideways_moves = 0
```

Bagian inisialisasi menyiapkan dua batas pencarian. Parameter `max_iterations` membatasi jumlah iterasi, sedangkan `maximum_sideways_move` membatasi jumlah perpindahkan dengan nilai objektif yang sama. Batas *sideways* diperbolehkan bernilai nol untuk menonaktifkan perpindahan tersebut. Variabel `sideways_move` dimulai dari nol untuk menghitung jumlah *sideways move* yang diterima. Pengacakan melalui `rng` digunakan untuk memilih salah satu neighbor ketika terdapat beberapa kandidat dengan nilai objektif yang sama. *Initial state* disalin `initial_snapshot` dan `current`. Nilai objektif state awal disimpan dala `current_value` dan dicatat pada `history.` Variabel `iterations` dan `started` digunakan untuk mencatat jumlah iterasi serta durasi pencarian.

Proses Pencarian

```
while iterations < max_iterations:
    improving = []
    equal = []
    for neighbor in neighbor_generator(current):
        value = objective_function(neighbor)
        if value > current_value:
            improving.append((value, neighbor))
        elif value == current_value:
            equal.append(neighbor)

    iterations += 1
    if improving:
        current_value, current = max(improving, key=lambda item:
item[0])
    elif equal and sideways_moves < maximum_sideways_move:
        current = rng.choice(equal)
        sideways_moves += 1
    else:
        history.append(current_value)
        break

    history.append(current_value)
    if sideways_moves == maximum_sideways_move and
maximum_sideways_move > 0:
        break

return build_result(
    initial_snapshot, current, current_value, history, iterations,
    perf_counter() - started, sideways_moves=sideways_moves,
    maximum_sideways_move=maximum_sideways_move,
)
```

Pada setiap iterasi, algoritma memeriksa seluruh neighbor dan menyimpan kandidat dalam dua daftar. Daftar `improving` berisi pasangan nilai objektif dan neighbor yang memberikan peningkatan. Daftar `equal` berisi neighbor dengan nilai objektif yang sama. Neighbor dengan nilai objektif lebih rendah tidak dipilih. Jika `improving` berisi kandidat, algoritma memilih neighbor dengan nilai objektif tertinggi. Jika tidak terdapat peningkatan, algoritma memilih satu neighbor dari `equal` secara

acak selama batas sideways belum tercapai. Perpindahan tersebut mengubah `current`, tetapi tidak mengubah `current_value.`

Pada implementasi ini, `sideways_moves` menghitung total perpindahan bernilai sama selama satu pencarian. Penghitung tersebut tidak dikembalikan ke nol setelah terjadi peningkatan. Ketika jumlah sideways move mencapai `maximum_sideways_move,` state hasil perpindahan tetap dicatat dan pencarian langsung dihentikan sesuai batas yang digunakan pada spesifikasi tugas. Jumlah iterasi dan nilai objektif dicatat pada setiap putaran, termasuk putaran ketika tidak ada neighbor yang dapat diterima. Hasil pencarian disusun melalui `build_result` dengan tambahan informasi jumlah *sideways move* yang dilakukan dan batas *sideways* yang digunakan.

2.5.4 Random Restart Hill-Climbing

a. Deskripsi Umum

Random Restart Hill Climbing menjalankan beberapa pencarian *Hill Climbing* dari state awal yang dibentuk ulang secara acak. Pada implementasi ini, setiap pencarian menggunakan *Steepest Ascent*. Pengulangan memberikan kesempatan untuk melakukan pencarian dari susunan kendaraan yang berbeda. Solusi dengan nilai objektif tertinggi dari seluruh pencarian disimpan sebagai hasil akhir. Jumlah restart dibatasi melalui parameter `maximum_restart.`

Random Restart tetap tidak menjamin ditemukannya global optimum. Hasil pencarian dipengaruhi oleh state awal yang terbentuk dan jumlah restart yang dilakukan.

b. Proses Pencarian dalam *Geometric Knapsack Problem*

Berikut merupakan tahapan pencarian menggunakan Random *Restart Hill Climbing* :

1. State awal dibentuk melalui fungsi pembuat initial state.
2. Steepest Ascent dijalankan dari state tersebut hingga kondisi berhentinya terpenuhi.
3. Hasil pencarian dibandingkan dengan solusi terbaik yang telah ditemukan.
4. Jika nilai objektif hasil pencarian lebih tinggi, solusi terbaik diperbarui.
5. Jumlah iterasi dan riwayat nilai objektif pencarian tersebut disimpan.
6. Jika batas restart belum tercapai, algoritma memanggil kembali pembuat initial state dan menjalankan Steepest Ascent.
7. Proses diulang hingga jumlah restart mencapai batas.
8. Solusi dengan nilai objektif tertinggi dari seluruh pencarian dikembalikan sebagai hasil akhir.

c. Kelas/Metode

Algoritma diimplementasikan melalui fungsi `random_restart` pada berkas *random_restart.py*. Fungsi menerima `initial_state_fn`, `objective_function`, `neighbor_generator`, dan `parameters`.

Argumen `initial_state_fn` berupa fungsi tanpa argumen yang mengembalikan state awal. Pembuatan state tersebutperlu melakukan pengacakan baru pada setiap pemanggilan agar restart dapat mencoba susunana awal yang berbeda.

Inisialisasi

```
parameters = {} if parameters is None else parameters
maximum_restart = validate_limit(parameters, "maximum_restart", 10,
minimum=0)
max_iterations = validate_limit(parameters, "max_iterations", 1000)
if not callable(initial_state_fn):
    raise ValueError("initial_state_fn harus berupa fungsi")

started = perf_counter()
first_state = initial_state_fn()
initial_snapshot = first_state.copy()
best_state = None
best_value = None
history = []
history_per_restart = []
iterations_per_restart = []
total_iterations = 0
```

Bagian inisialisasi menyiapkan batas jumlah restart dan batas iterasi untuk setiap pencarian *Steepest Ascent*. Parameter `maximum_restart` menghitung pengulangan setelah pencarian pertama. Jika bernilai dua, algoritma melakukan satu pencarian awal dan dua restart. Jika bernilai nol, pencarian awal tetap dilakukan. Fungsi `initial_state_fn` diperiksa menggunakan `callable` agar dapat dipanggil untuk menghasilkan state awal. State pertama disimpan dalam `first_state`, kemudian disalin ke dalam `initial_snapshot` untuk pencatatan hasil.

Variabel `best_state` menyimpan solusi terbaik, sedangkan `best_value` menyimpan nilai objektifnya. Daftar `iterations_per_restart` dan `history_per_restart` digunakan untuk menyimpan jumlah iterasi serta riwayat nilai objektif setiap pencarian. Keduanya mencakup pencarian awal pada indeks nol. Variabel `total_iterations` menghitung jumlah iterasi seluruh pencarian. Pengukuran durasi dimulai sebelum pembentukan state pertama sehingga mencakup pembentukan ulang state dan proses pencariannya.

Proses Pencarian

```
for run_index in range(maximum_restart + 1):
    start_state = first_state if run_index == 0 else
initial_state_fn()
    result = steepest_ascent(
        start_state, objective_function, neighbor_generator,
        {"max_iterations": max_iterations},
    )
    iterations_per_restart.append(result["iterations"])
```

```
    history_per_restart.append(result["history"])
    total_iterations += result["iterations"]

    if best_value is None or result["best_value"] > best_value:
        best_state = result["best_state"]
        best_value = result["best_value"]

    if not history:
        history.append(objective_function(initial_snapshot))
    history.extend(max(history[-1], value) for value in
result["history"][1:])

return build_result(
    initial_snapshot, best_state, best_value, history,
total_iterations,
    perf_counter() - started, restarts=len(iterations_per_restart) -
1,
    iterations_per_restart=iterations_per_restart,
    history_per_restart=history_per_restart,
)
```

Perulangan dilakukan sebanyak satu pencarian awal ditambah jumlah restart. Pencarian pertama menggunakan `first_state`, sedangkan pencarian berikutnya memanggil kembali `initial_state_fn`. Setiap state awal kemudian diberikan kepada `steepest_ascent`. Batas `max_iterations` berlaku untuk setiap pencarian tersebut. Jumlah iterasi hasil pencarian ditambahkan ke `iterations_per_restart` dan dijumlahkan dalam `total_iterations`. Riwayat nilai objektifnya disimpan dalam `history_per_restart`. Setelah satu pencarian selesai, nilai objektif hasilnya dibandingkan dengan `best_value.` Jika lebih tinggi, `best_state` dan `best_value` diperbarui. Hasil pencarian yang lebih rendah tidak menggantikan solusi terbaik dan tidak menghentikan *restart* berikutnya.

Daftar *history* mencatat perkembangan nilai objektif terbaik yang ditemukan sepanjang proses. Sementara itu, `history_per_restart` menyimpan riwayat asli setiap pencarian, termasuk nilai objektif awalnya. Riwayat per pencarian digunakan untuk menampilkan perjalanan nilai objektif ketika restart dimulai dari nilai yang lebih rendah. Jumlah restart dihitung dari banyaknya pencarian dikurangi satu karena pencarian pertama bukan restart. Setelah seluruh pencarian selesai, `build_result` mengembalikan solusi terbaik. Dengan demikian, state akhir yang dikembalikan dapat berasal dari pencarian awal maupun salah satu restart dan tidak harus sama dengan hasil pencarian terakhir.

2.5.5 Simulated Annealing

a. Deskripsi Umum

Simulated Annealing merupakan algoritma pencarian yang menggunakan temperatur pada setiap iterasinya untuk mengatur perpindahan ke suatu *neighbor*. Pada setiap iterasi, algoritma memilih neighbor secara acak dan membandingkan nilai

objektifnya dengan state saat ini. *Neighbor* dengan nilai objektif sama atau lebih tinggi selalu diterima, sementara *neighbor* dengan nilai lebih rendah dapat diterima berdasarkan probabilitas yang didefinisikan sebagai nilai $P = e^{\frac{delta\ E}{T}}$.

Probabilitas ini akan melihat selisih nilai objektif (delta *E*) dan temperatur saat ini (T). Pada setiap iterasi, temperatur akan mengalami penurunan nilai dengan atribut *cooling rate*. Pendekatan ini membuat peluang menerima perpindahan yang menurunkan nilai objektif semakin kecil untuk selisih nilai yang sama. Selama pencarian berlangsung, algoritma menyimpan state terbaik secara terpisah dari state saat ini. Pencarian berakhir ketika temperatur mencapai batas minimum, batas iterasi, atau tidak lagi tersedia *neighbor*.

b. Proses Pencarian dalam *Geometric Knapsack Problem*

Pencarian permasalahan ini dengan *Simulated Annealing* dilakukan sebagai berikut:

1. Susunan kendaraan yang memenuhi batasan dan kapasitas kapal dibentuk sebagai suatu susunan.
2. Hitung nilai objektif susunan awal berdasarkan total *ShippingFee* kendaraan yang diangkut.
3. Tentukan temperatur awal dan aturan penurunan temperatur (*cooling rate*).
4. Pilih satu susunan tetangga secara acak lalu hitung nilai objektifnya.
5. Jika nilai objektif tetangga sama atau lebih tinggi, susunan diterima sebagai solusi saat ini.
6. Jika nilai objektif tetangga lebih rendah, dilakukan pemeriksaan berdasarkan probabilitas $P = e^{\frac{delta\ E}{T}}$ yang didefinisikan sebagai selisih nilai objektif dan temperatur.
7. Perbarui solusi terbaik apabila ditemukan susunan dengan nilai objektif lebih tinggi.
8. Turunkan temperatur setiap iterasi berlangsung.
9. Lakukan iterasi pencarian hingga tercapai kondisi penghentian, seperti iterasi maksimum ataupun nilai temperatur mencapai batas minimum.
10. Kembalikan susunan terbaik yang ditemukan selama proses pencarian berlangsung.

c. Kelas/Metode

Pada tahap inisialisasi, fungsi menerima state awal, *objective function*, pembentuk *neighbor*, dan parameter pencarian. Parameter yang digunakan mencakup temperatur awal, *cooling rate*, temperatur minimum, batas iterasi, dan *seed* pengacakan. Program memeriksa kesesuaian parameter serta validasi state awal sebelum melakukan pencarian. State awal ini disalin sebagai state saat ini yang

sekaligus mendefinisikan state terbaik. Implementasi dari kode juga menyiapkan atribut yang mendefinisikan riwayat *objective*, nilai terbaik, riwayat temperatur, dan faktor eksponensial. Sementara untuk atribut iterasi, perhitungan ketika pencarian *stuck*, dan penerimaan nilai lebih buruk dari state sebelumnya di-*set* ke nilai 0.

Inisialisasi

```
def simulated_annealing(initial_state, objective_function,
neighbor_generator, parameters = None):
    started = time.perf_counter() # function to measure the length
of time taken
    p = parameters or {} # filled with default values if parameters
is None

    temperature = float(p.get("initial_temperature", 1000.0)) #
start at T = 1000
    cooling_rate = float(p.get("cooling_rate", 0.95)) # T = T * 0.95
each iteration
    min_temperature = float(p.get("min_temperature", 0.001)) # stop
if T < 0.001
    max_iterations = p.get("max_iterations", 1000) # maximum of 1000
iterations
    rng = random.Random(p.get("seed")) # random number generator for
reproducability

    if not math.isfinite(temperature) or temperature <= 0 or
min_temperature <= 0:
        raise ValueError("Temperatur harus lebih besar dari 0")
    if not math.isfinite(cooling_rate) or cooling_rate <= 0 or
cooling_rate >= 1:
        raise ValueError("cooling_rate harus berada di antara 0 dan
1")
    if not math.isfinite(min_temperature) or min_temperature <= 0:
        raise ValueError("min_temperature harus lebih besar dari 0")
    if type(max_iterations) is not int or max_iterations < 0:
        raise ValueError("max_iterations harus integer non-negatif")
    if not is_valid(initial_state):
        raise ValueError("initial_state tidak valid pada state ini")

    current = initial_state.copy()
    current_value = objective_function(current)
    best = current.copy()
    best_value = current_value

    history = [current_value]
    best_history = [best_value]
    temperature_history = [temperature]
    e_deltaet_history = []
    log_e_deltaet_history = []

    iterations = 0
    stuck_count = 0
    accept_worse = 0
```

Fungsi `sample_neighbor` memilih satu kandidat secara acak dari *neighbor* yang dihasilkan. Pemilihan dilakukan menggunakan sampling dengan menelusuri kandidat satu per satu. Setiap kandidat dihitung nilai objektifnya, kemudian dapat menggantikan kandidat terpilih berdasarkan bilangan acak. Dengan pendekatan tersebut, setiap *neighbor* dapat memiliki peluang pemilihan yang sama tanpa perlu

menyimpan seluruh nilai dalam sebuah *list*. Fungsi juga mencatat jika terdapat *neighbor* dengan nilai *objective* lebih tinggi daripada state saat ini melalui `has_improvement`. Hasil yang dikembalikan nantinya meliputi kandidat terpilih, nilai objektifnya, dan penanda perbaikan.

sample_neighbor

```
def sample_neighbor(state, current_value, objective_function,
neighbor_generator, rng):
    selected = None # state from neighbor_generator
    selected_value = None # objective value of selected
    count = 0
    has_improvement = False

    for candidate in neighbor_generator(state):
        value = objective_function(candidate)
        has_improvement = has_improvement or value > current_value #
if any neighbor is better than current
        count += 1

        if rng.randrange(count) == 0:
            selected = candidate.copy() # value is copied to avoid
mutation of the original candidate
            selected_value = value

    return selected, selected_value, has_improvement
```

Proses pencarian dijalankan selama batas iterasi belum tercapai dan temperatur masih lebih besar daripada batas minimum yang telah didefinisikan sebelumnya. Pada setiap iterasi, program memanggil `sample_neighbor`. Apabila tidak terdapat kandidat, pencarian akan berhenti. Selisih nilai objektif dihitung dengan mengurangkan nilai state saat ini dari nilai kandidat. Kandidat dengan nilai sama dan/atau lebih tinggi selalu diterima, sedangkan kandidat yang lebih rendah akan tetap diterima bila pengecekan terhadap nilai $P = e^{\frac{delta\ E}{T}}$ diterima. Pengecekan ini dilakukan dengan membandingkan kandidat dengan bilangan acak antara 0 dan 1. Kandidat diterima bila bilangan acak lebih kecil dibanding nilai P.

Ketika kandidat diterima, state saat ini akan diperbarui. Jika nilainya melampaui nilai objektif terbaik, maka program akan memperbarui `best`. Setelahnya, temperatur akan dikalikan dengan *cooling rate* setiap iterasi selesai dilakukan. Pada akhir proses, fungsi mengembalikan solusi terbaik beserta riwayat dan statistik pencarian.

Proses Pencarian

```
    for iteration in range(max_iterations):
        if temperature <= min_temperature:
            break # stop if temperature is under threshold

        candidate, candidate_value, has_improvement =
```

```
sample_neighbor(current, current_value, objective_function,
neighbor_generator, rng)

        if not has_improvement:
            stuck_count += 1

        if candidate is None:
            break

        delta = candidate_value - current_value # delta E
        exponent = delta / temperature # delta E / T

        try:
            e_deltaet = math.exp(exponent) # e^(delta E / T)
        except OverflowError:
            e_deltaet = float("inf") if exponent > 0 else 0.0

        e_deltaet_history.append(e_deltaet)
        log_e_deltaet_history.append(exponent)

        accepted = (delta >= 0 or rng.random() < e_deltaet)

        if accepted: # fulfilling rules either better value or worse
value with probability
            if delta < 0:
                accept_worse += 1
            current = candidate
            current_value = candidate_value

            if current_value > best_value:
                best = current.copy()
                best_value = current_value

        iterations += 1
        temperature *= cooling_rate
        history.append(current_value)
        best_history.append(best_value)
        temperature_history.append(temperature)

    return {
        "initial_state": initial_state.copy(),
        "best_state": best.copy(),
        "best_value": best_value,
        "final_state": best.copy(),
        "final_value": best_value,
        "terminal_state": current.copy(),
        "terminal_value": current_value,
        "history": history,
        "best_history": best_history,
        "temperature_history": temperature_history,
        "e_deltaet_history": e_deltaet_history,
        "log_e_deltaet_history": log_e_deltaet_history,
        "iterations": iterations,
        "duration": time.perf_counter() - started,
        "stuck_count": stuck_count,
        "accept_worse": accept_worse,
    }
```

2.5.6 Genetic Algorithm

a. Deskripsi Umum

Genetic Algorithm adalah algoritma pencarian yang melakukan proyeksi sekumpulan solusi dengan proses seleksi, *crossover*, dan mutasi. Setiap solusi disebut individu, sementara kumpulannya disebut populasi. Kualitas setiap individu dinilai

menggunakan nilai *fitness* yang menjadi dasar pemilihan *parent* untuk menghasilkan individu berikutnya.

Pada implementasi yang diaplikasikan saat ini, seleksi *parent* menggunakan *roulette wheel selection*. Individu yang terpilih dengan *fitness* lebih tinggi akan memiliki peluang pemilihan lebih besar. *Crossover* akan membentuk individu baru dengan menggabungkan informasi penempatan kendaraan dari dua *parent*. Sementara untuk mutasi sendiri prosesnya adalah pengubahan individu melalui pemilihan *neighbor* secara acak. Proses pembentukan populasi dilakukan berulang hingga batas generasi tercapai, dengan tetap menyimpan individu terbaik yang ditemukan sepanjang proses pencarian.

b. Proses Pencarian dalam *Geometric Knapsack Problem*

Pencarian permasalahan ini dengan *Genetic Algorithm* dilakukan sebagai berikut:

1. Bentuk populasi awal yang terdiri atas susunan beberapa kendaraan yang memenuhi batasan.
2. Hitung *fitness score* setiap individu berdasarkan total *ShippingFee* kendaraan yang diangkut.
3. Pilih individu yang akan menjadi *parent* berdasarkan *fitness score.*
4. Lakukan *crossover* dengan menggabungkan informasi dari dua *parent* untuk menghasilkan individu baru.
5. Lakukan mutasi untuk menghasilkan variasi susunan kendaraan.
6. Pastikan individu yang dihasilkan memenuhi batasan penempatan dan tidak melebihi batas maksimum kapasitas kapal.
7. Bentuk populasi generasi berikutnya dari individu baru.
8. Evaluasi populasi baru dan perbarui solusi terbaik.
9. Ulangi proses evolusi hingga situasi penghentian yang didefinisi tercapai.
10. Kembalikan susunan terbaik yang ditemukan selama proses pencarian.

c. Kelas/Metode

Fungsi `genetic_algorithm` menerima fungsi penyedia populasi awal, *objective fuction*, dan parameter pencarian. Parameter yang digunakan meliputi jumlah individu, batas iterasi, *random seed*, dan persentase *crossover* dan *mutation*. Sebelum melakukan pemrosesan terhadap populasi, program memeriksa bahwa jumlah individu setidaknya dua, batas iterasi tidak negatif, jumlah keturunan lebih kecil daripada jumlah individu, dan peluang berada pada rentang nol sampai satu.

Pada bagian ini, populasi awal terdiri atas sejumlah susunan kendaraan sesuai parameter `population_size`. Setiap susunan kemudian dibentuk melalui fungsi `create_initial_state` menggunakan data kapal dan kendaraan yang sama. *Seed*

untuk pembentukan setiap individu dihasilkan dari *seed* inisialisasi agar populasi awal dapat direproduksi. Seluruh state kemudian dikumpulkan dalam satu *list*.

Inisialisasi

```
def genetic_algorithm(initial_population, objective_function,
parameters = None):
    started = time.perf_counter() # function to measure the length
of time taken
    p = parameters or {}

    population_size = p.get("population_size", 50)
    max_iterations = p.get("max_iterations", 100)
    crossover_rate = float(p.get("crossover_rate", 0.8))
    mutation_rate = float(p.get("mutation_rate", 0.2))
    rng = random.Random(p.get("seed"))

    if type(population_size) is not int or population_size < 2:
        raise ValueError("population_size minimal bernilai 2") #
need minimum of 2 population to select parents
    if type(max_iterations) is not int or max_iterations < 0:
        raise ValueError("max_iterations harus integer non-negatif")
    if any(not math.isfinite(rate) or not 0 <= rate <= 1
           for rate in (crossover_rate, mutation_rate)):
        raise ValueError("Probabilitas harus berhingga dan bernilai
antara 0 dan 1")

    population = [state.copy() for state in initial_population()] #
copy the initial population to avoid mutation of the original states
    if len(population) != population_size:
        raise ValueError("Jumlah state awal harus sama dengan
population_size")
    if not all(is_valid(state) for state in population):
        raise ValueError("Semua state awal harus valid")

    reference = population[0] # first state as reference
    reference_ships = {}
    for item in reference.ships:
        reference_ships[item.ship.id] = item.ship # define a
dictionary of ship id of first state

    for state in population:
        ships = {}
        for item in state.ships:
            ships[item.ship.id] = item.ship # define a dictionary of
ship id for each state
        if state.vehicles != reference.vehicles or ships !=
reference_ships:
            raise ValueError("Semua individu harus memakai acuan
yang sama")
```

Dengan pendekatan pencarian *roulette*, nilai *fitness* setiap individu diperoleh langsung dari *objective function* dan disimpan dalam `scores`. Individu dengan nilai tertinggi akan dipilih sebagai solusi terbaik populasi awal. Program akan menyalin nilai melalui `initial_best`. Atribut ini digunakan sebagai acuan solusi terbaik sepanjang pencarian.

Roulette dan Fitness Score

```
    scores = [objective_function(state) for state in population] #
calculate the objective function for each state
    best_index = max(range(population_size), key = lambda i:
scores[i])
    best = population[best_index].copy()
    best_value = scores[best_index]
    initial_best = best.copy()

    history = [max(scores)]
    mean_history = [sum(scores) / population_size]
    best_history = [best_value]
```

Fungsi `genes` membentuk representasi penempatan kendaraan. Identitas kendaraan digunakan sebagai *key*, sementara objek `Placement` menjadi *value*. Pemetaan tersebut diperoleh dengan menelusuri seluruh penempatan pada setiap kapal. Kendaraan di luar kapal tidak memiliki entri mengingat tidak memiliki penempatan. Representasi ini digunakan dalam *crossover* untuk mengambil informasi penempatan kendaraan dari masing-masing *parent*.

genes

```
def genes(state):
    result = {}

    for item in state.ships: # consists of ships, each ship has
placements
        for placement in item.placements: # each placement is a gene
            result[placement.vehicle_id] = placement
```

Fungsi `select_parent` memilih satu individu berdasarkan nilai *fitness* untuk dijadikan *parent*. Pemilihan akan dilakukan secara acak dengan melihat nilai *fitness*, mengutamakan individu dengan *fitness* lebih tinggi memiliki peluang lebih besar untuk dipilih. Program menjumlahkan *fitness* seluruh individu, kemudian menghasilkan bilangan acak dari nol hingga kurang dari jumlah tersebut. Nilai *fitness* kembali dijumlahkan secara bertahap sesuai urutan populasi. Individu akan dipilih ketika penjumlahan pertama kali mencapai/melebihi nilai bilangan acak yang dihasilkan.

select_parent

```
def select_parent(population, fitness_scores, rng):
    if not population or len(population) != len(fitness_scores):
        raise ValueError("population and fitness_scores harus
memiliki panjang yang sama dan tidak kosong")

    for fitness in fitness_scores:
        if not math.isfinite(fitness) or fitness < 0:
            raise ValueError("fitness_scores harus berhingga dan
non-negatif")

    total_fitness = sum(fitness_scores)
```

```
    if not math.isfinite(total_fitness):
        raise ValueError("total_fitness harus berhingga")

    if total_fitness == 0:
        return rng.choice(population) # if all fitness scores are
zero, select randomly

    spin = rng.random() * total_fitness
    cumulative = 0.0
    last_positive = None

    for index, fitness in enumerate(fitness_scores):
        if fitness > 0:
            last_positive = index
            cumulative += fitness

            if cumulative >= spin:
                return population[index]

    return population[last_positive]
```

Fungsi `crossover` membentuk *child* dari salinan *parent* pertama, kemudian mengosongkan seluruh penempatannya. Informasi gen kedua *parent* diperoleh melalui fungsi *genes* yang sebelumnya telah terdefinisi. Setelah urutan kendaraan diacak, program memilih sumber gen setiap kendaraan dari salah satu *parent* dengan peluang 50% untuk masing-masing.

Apabila kendaraan memiliki penempatan pada *parent* terpilih, penempatan tersebut akan dicoba pada *child*. Program kemudian memeriksa validitas *child*, jika terdapat pelanggaran batasan dari yang telah terdefinisi maka akan membatalkan penempatan tersebut.

crossover

```
def crossover(parent_a, parent_b, rng):
    child = parent_a.copy() # copy of state from parent a
    for item in child.ships:
        item.placements.clear()

    genes_a = genes(parent_a) # dictionary of vehicle_id to
placement for parent a
    genes_b = genes(parent_b)

    ships = {}
    for item in child.ships:
        ships[item.ship.id] = item

    vehicle_ids = sorted(child.vehicles) # list of vehicle ids in
child state
    rng.shuffle(vehicle_ids)

    for vehicle_id in vehicle_ids:
        if rng.random() < 0.5:
            source = genes_a # choose parent's gene with 50%
probability
        else:
            source = genes_b

        placement = source.get(vehicle_id)
```

```
        if placement is None:
            continue # condition when the vehicle_id is outside the
ship

        target = ships[placement.ship_id]
        target.placements.append(placement)

        if not is_valid(child):
            target.placements.pop() # erase the placement if it
makes the child invalid

    return child
```

Fungsi `mutate` menghasilkan perubahan individu dengan memilih satu *neighbor* valid secara acak. Kandidat diperoleh melalui `generate_neighbors`, sehingga perubahan mengikuti operasi penempatan yang tersedia pada pembentuk *neighbor*. Apabila kandidat terpilih tersedia, fungsi akan menyalin dan mengembalikannya sebagai hasil mutasi. Keputusan menjalankan mutasi akan bergantung pada `mutation_rate` yang telah didefinisikan pada perulangan utama algoritma.

mutate

```
def mutate(state, rng):
    selected = None
    for count, candidate in enumerate(generate_neighbors(state),
start = 1): # iterate count while generating neighbors
        if rng.randrange(count) == 0: # randomly select a neighbor
with uniform probability
            selected = candidate.copy()

    if selected:
        return selected
    else:
        return state.copy()
```

Program akan mengurutkan individu berdasarkan *fitness score*. Sisa populasi kemudian dibentuk dengan memilih dua *parent* melalui `select_parent`. *Crossover* akan dijalankan berdasarkan `crossover_rate`, begitu juga dengan *child* yang akan mengalami mutasi berdasarkan `mutation_rate` sebelum dimasukkan ke populasi berikutnya.

Setelah jumlah individu terpenuhi, populasi lama diganti dengan populasi baru dan seluruh nilai *fitness* dihitung kembali. Program memperbarui solusi terbaik apabila ditemukan individu dengan nilai lebih tinggi, kemudian mencatat nilai maksimum populasi, rata-rata *fitness*, dan nilai terbaik sepanjang pencarian. Proses ini berlangsung hingga batas penghasilan generasi tercapai.

Proses Pencarian

```
    for iteration in range(max_iterations):
        next_population = []

        while len(next_population) < population_size:
            parent_a = select_parent(population, scores, rng) # pick
from the population based on tournament selection
            parent_b = select_parent(population, scores, rng)

            if rng.random() < crossover_rate: # crossover with
probability crossover_rate
                child = crossover(parent_a, parent_b, rng)
            else:
                child = parent_a.copy()

            if rng.random() < mutation_rate: # mutate with
probability mutation_rate
                child = mutate(child, rng)

            next_population.append(child) # add the child to the
next population

        population = next_population
        scores = [objective_function(state) for state in population]
        best_index = max(range(population_size), key = lambda i:
scores[i]) # find the index of the best individual

        if scores[best_index] > best_value:
            best = population[best_index].copy()
            best_value = scores[best_index]

        history.append(max(scores))
        mean_history.append(sum(scores) / population_size)
        best_history.append(best_value)

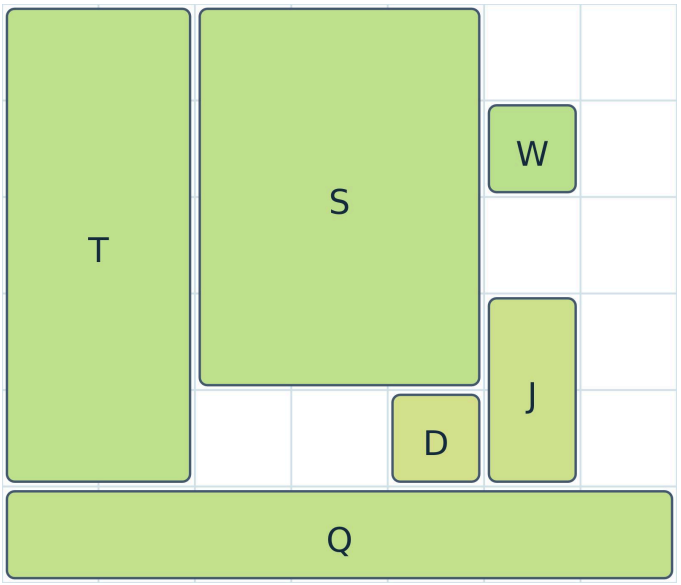
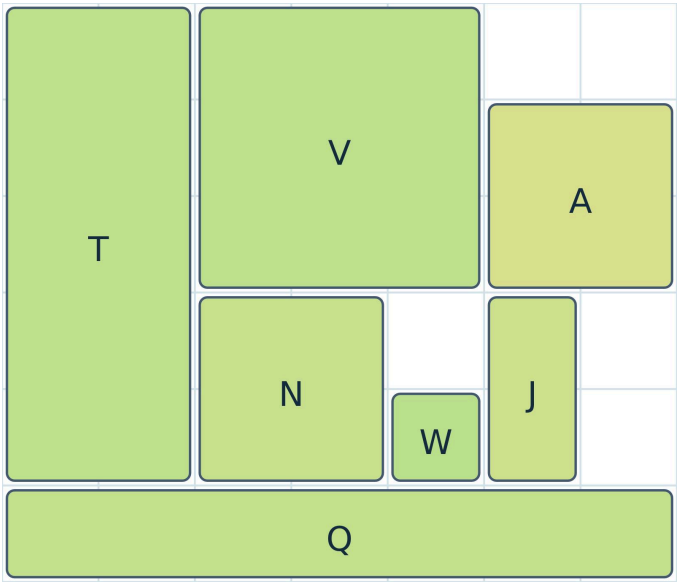
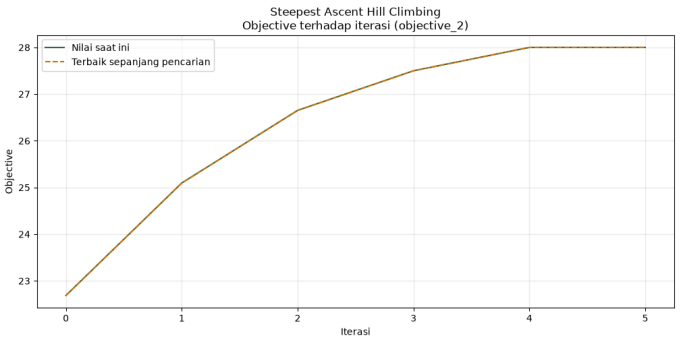
    return {
        "initial_state": initial_best,
        "best_state": best.copy(),
        "best_value": best_value,
        "final_state": best.copy(),
        "final_value": best_value,
        "history": history,
        "mean_history": mean_history,
        "best_history": best_history,
        "population_size": population_size,
        "iterations": max_iterations,
        "duration": time.perf_counter() - started,
    }
```

2.6 Hasil Eksperimen dan Analisis

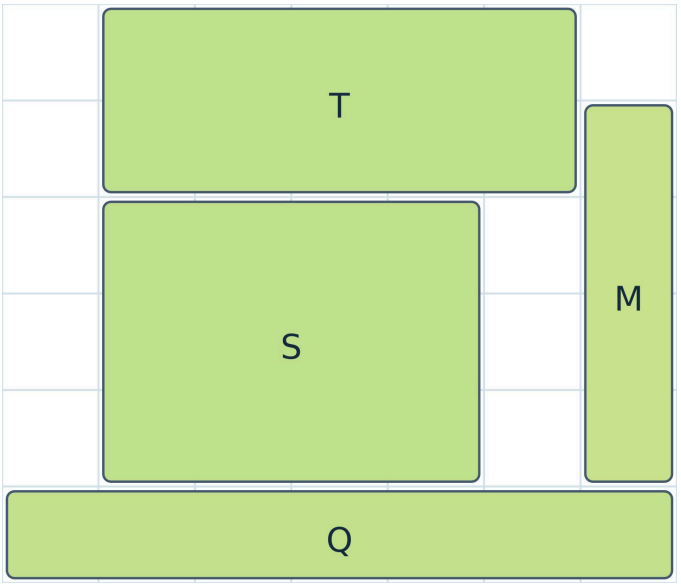
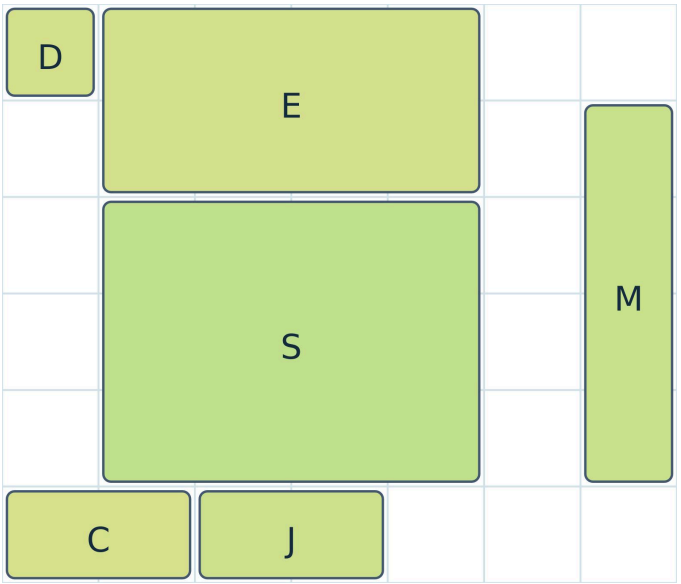
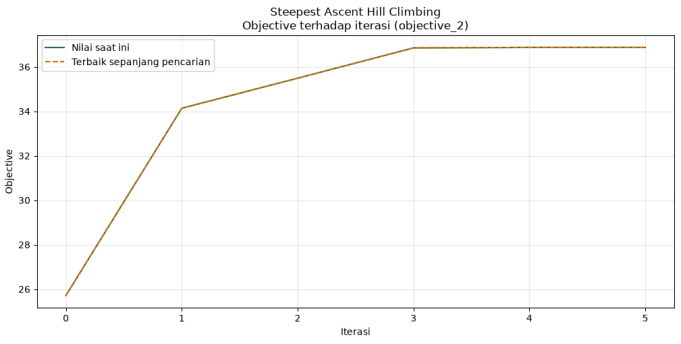
2.6.1. Steepest Hill Climbing

Tabel 2.6.1 Hasil Percobaan *Steepest Hill Climbing*

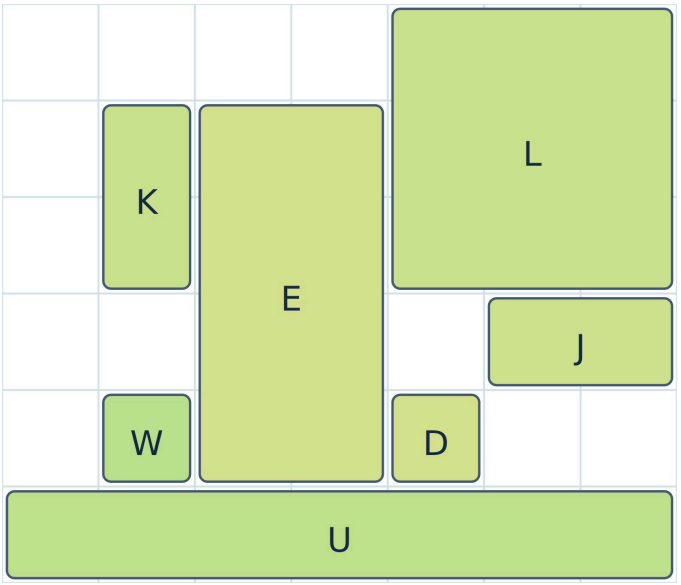
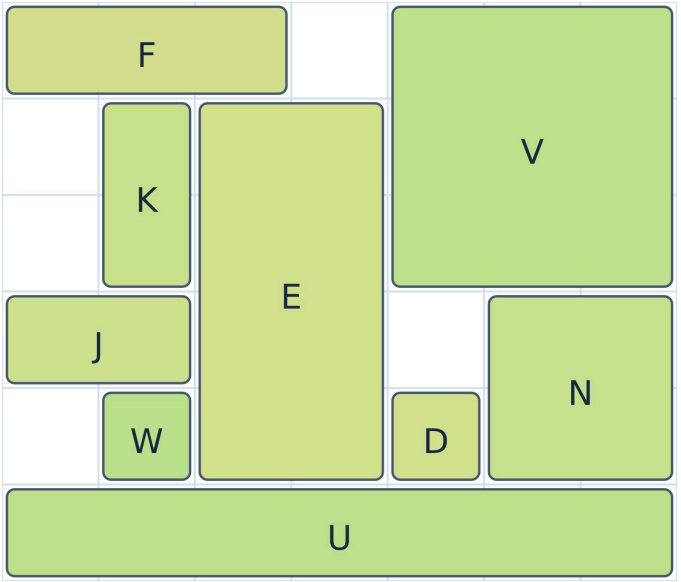
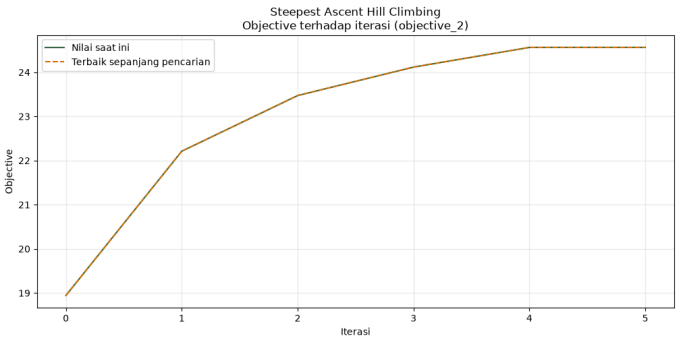
Percobaan ke-	Nilai *initial objective function*	Nilai *final objective function*	Durasi (*s*)	Jumlah iterasi
1	22.69	28.00	0.7268	5
2	25.71	36.88	0.6798	5
3	18.94	24,56	0.6847	5

Percobaan 1	
Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 2

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 3

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Berdasarkan tiga percobaan yang dilakukan, *Steepest Ascent* menghasilkan nilai objektif akhir sebesar 28.00, 36.88, dan 24.56. Dibandingkan dengan nilai objektif awal, peningkatan pada masing-masing percobaan sebesar 5.31, 11.17, dan 6.62. Rata-rata nilai objektifnya sebesar 29.81. Grafik menunjukkan bahwa nilai

objektif meningkat pada bagian awal pencarian, kemudian menjadi tetap pada bagian akhir. Pola tersebut sesuai dengan aturan *Steepest Ascent* yang hanya menerima neighbor dengan nilai objektif lebih tinggi. Garis nilai objektif saat ini dan nilai terbaik sepanjang pencarian juga berimpit karena algoritma tidak menerima perpindahan yang menurunkan nilai objektif.

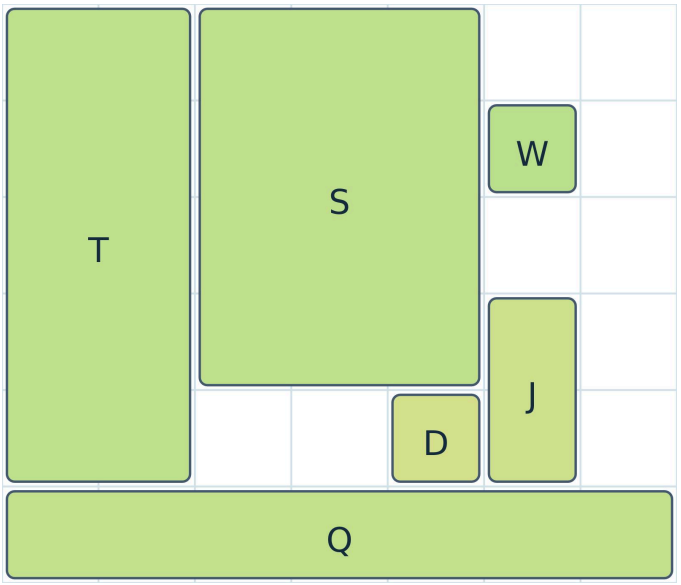
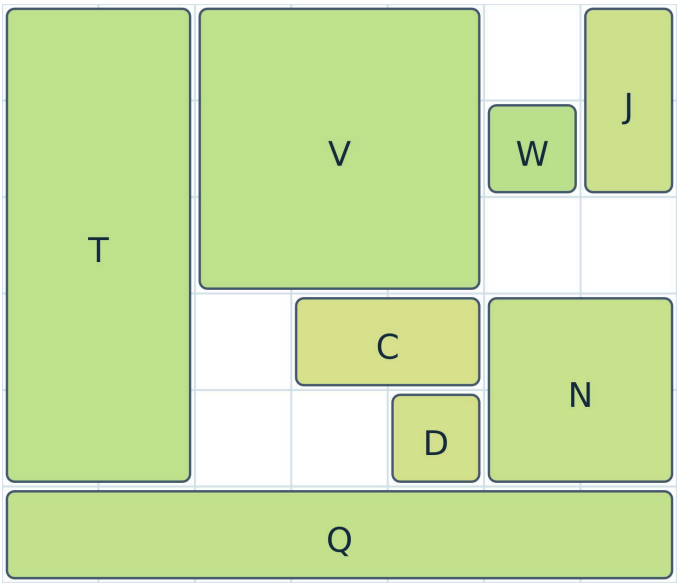
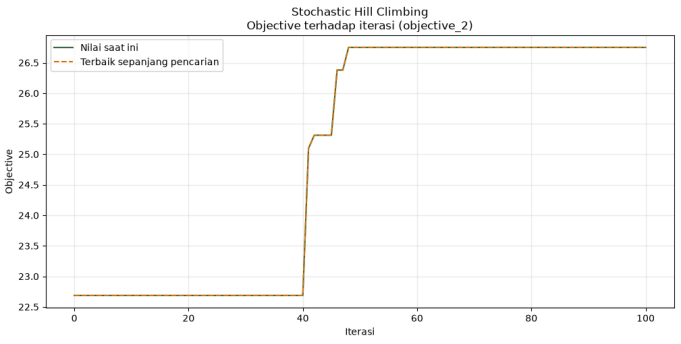
Seluruh percobaan tercatat berlangsung selama lima iterasi. Dalam pencatatan implementasi, jumlah tersebut mencakup putaran pemeriksaan yang tidak menghasilkan perpindahan. Dengan demikian, jumlah iterasi tidak selalu sama dengan jumlah perubahan state yang diterima. Perbedaan hasil akhir menunjukkan bahwa state awal mempengaruhi jalur pencarian. Percobaan kedua memperoleh nilai objektif akhir tertinggi, sedangkan percobaan ketiga memperoleh nilai terendah. Walaupun algoritma selalu memilih neighbor terbaik pada setiap iterasi, pilihan tersebut hanya didasarkan pada perubahan yang dapat dilakukan langsung dari state saat itu. Algoritma belum tentu dapat mencapai susunan yang lebih baik apabila diperlukan beberapa perpindahan dengan nilai objektif sama atau lebih rendah. Durasi ketiga percobaan adakah 0.7268 detik, 0.6798 detik, dan 0.6847 detik, dengan rata-rata 0.6971 detik. Durasi yang relatif singkat berkaitan dengan sedikitnya iterasi yang dijalankan. Namun, berhentinya pencarian dalam waktu singkat tidak membuktikan bahwa solusi yang diperoleh merupakan global optimum.

2.6.2. Stochastic Hill Climbing

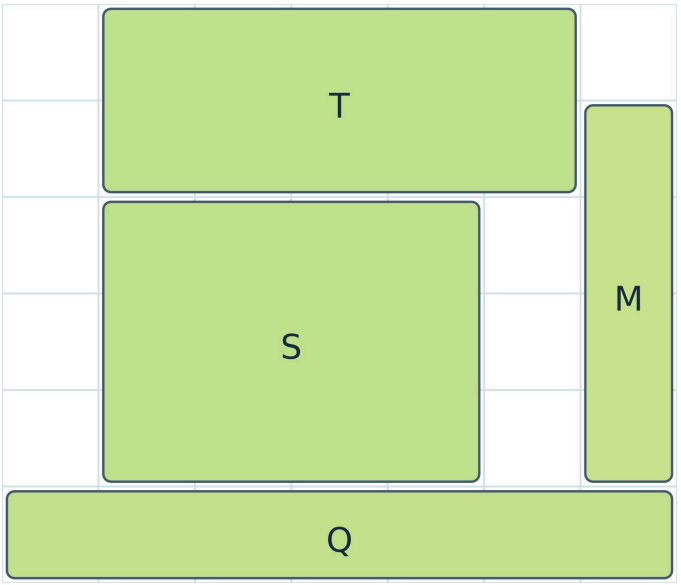
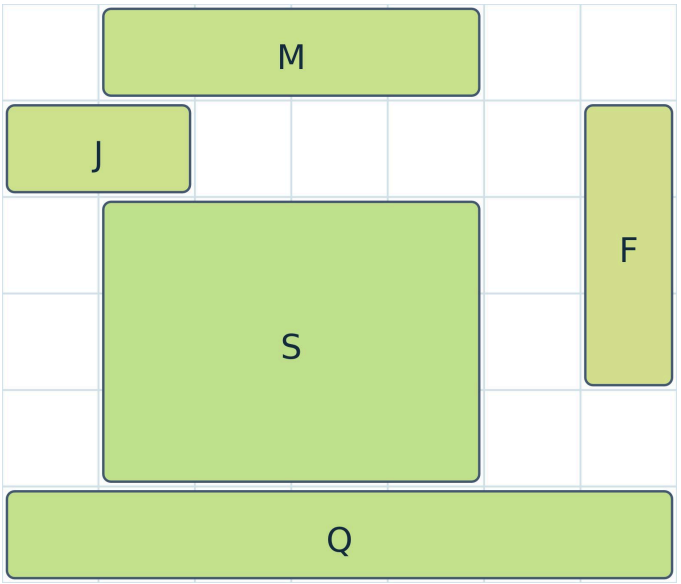
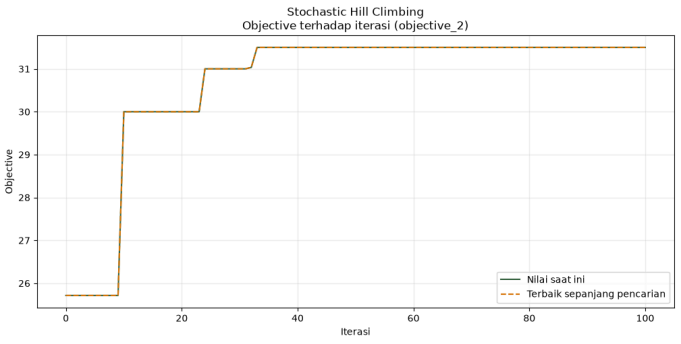
Tabel 2.6.2 Hasil Percobaan *Stochastic Hill Climbing*

Percobaan ke-	Nilai *initial objective function*	Nilai *final objective function*	Durasi (*s*)	Jumlah iterasi
1	22.69	26.75	0.5358	100
2	25.71	31.50	0.5351	100
3	18.94	22,82	0.5391	100

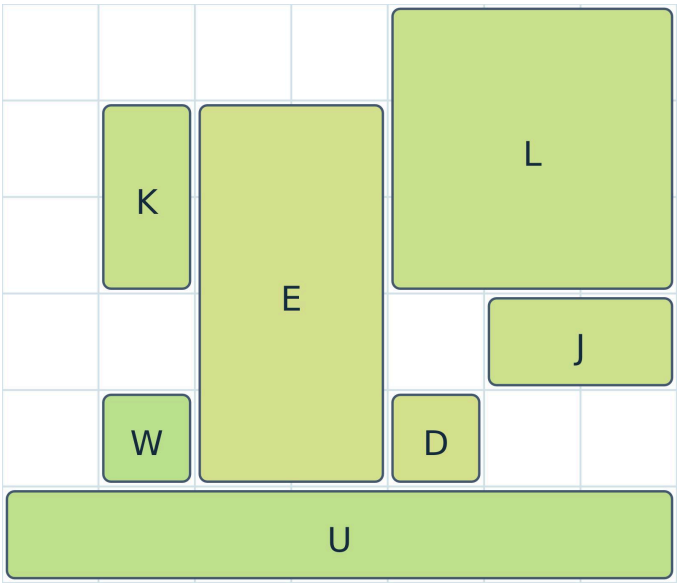
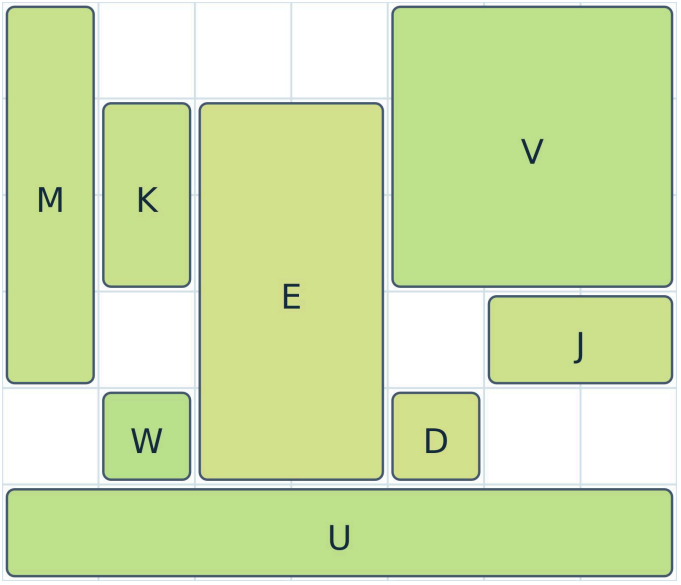
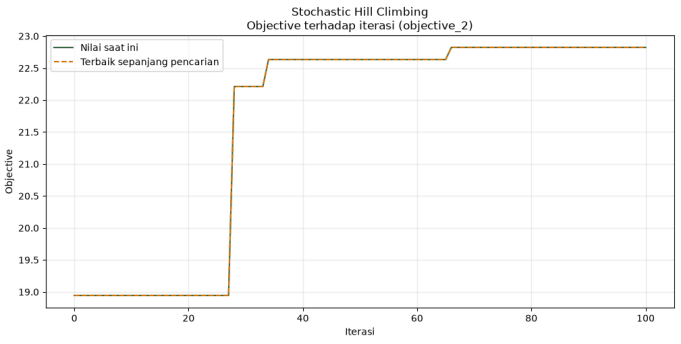
Percobaan 1

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 2

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 3

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Stochastic Hill Climbing menghasilkan nilai objektif akhir sebesar 26.75, 31.50, dan 22.82. Peningkatan terhadap nilai objektif awal masing-masing sebesar 4.06, 5.79, dan 3.88. Rata-rata nilai objektif akhir yang diperoleh adalah 27.02. Grafik memperlihatkan pola kenaikan bertahap yang diselingi rentang nilai objektif tetap.

Pada percobaan pertama, nilai objektif bertahan di sekitar nilai awal hingga mendekati iterasi ke-40. Setelah itu, terjadi beberapa peningkatan sebelum nilai objektif kembali tetap hingga pencarian selesai. Pada percobaan kedua dan ketiga, peningkatan juga terjadi pada beberapa bagian pencarian, tetapi waktunya berbeda. Pola tersebut dipengaruhi oleh pemilihan neighbor secara acak. Pada suatu iterasi, neighbor yang dipilih belum tentu memberikan peningkatan. Jika nilai objektifnya sama atau lebih rendah, neighbor ditolak dan state saat ini dipertahankan. Karena itu, bagian grafik yang mendatar tidak langsung menunjukkan bahwa algoritma telah berhenti atau seluruh neighbor tidak dapat memberikan peningkatan.

Ketiga percobaan berlangsung hingga 100 iterasi. Setelah tidak memperoleh peningkatan selama beberapa percobaan, algoritma tetap mencoba neighbor berikutnya sampai batas iterasi tercapai. Perilaku ini berbeda dari *Steepest Ascent* yang dapat menghentikan pencarian setelah pemeriksaan seluruh neighbor tidak menemukan peningkatan.

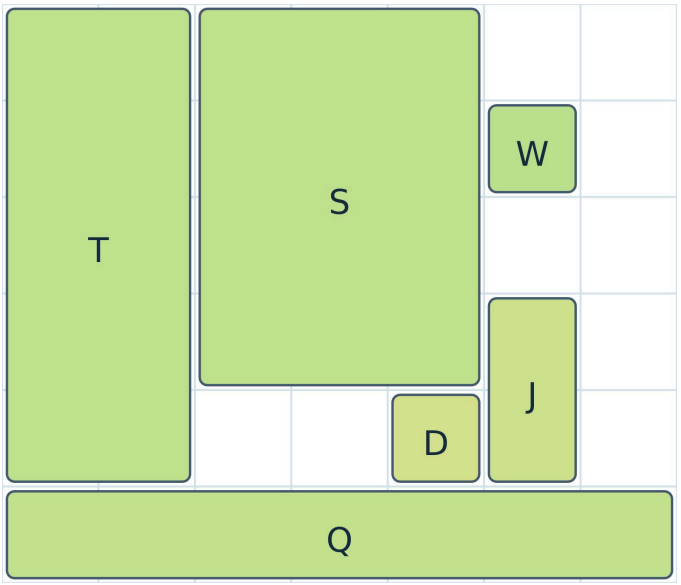
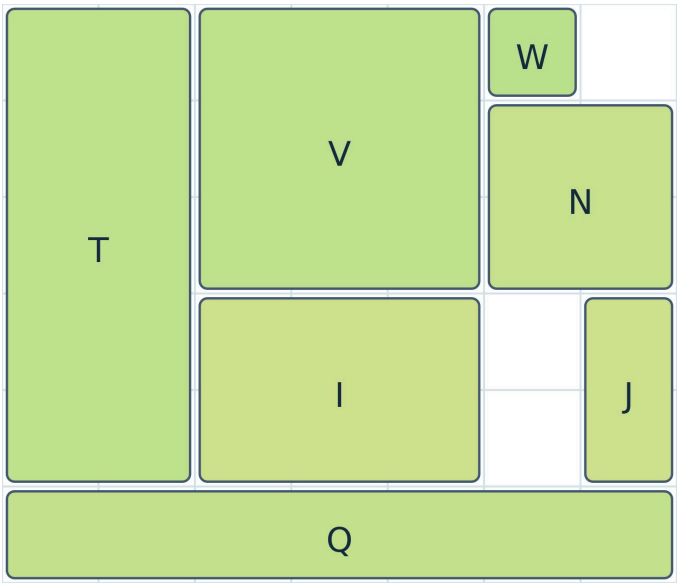
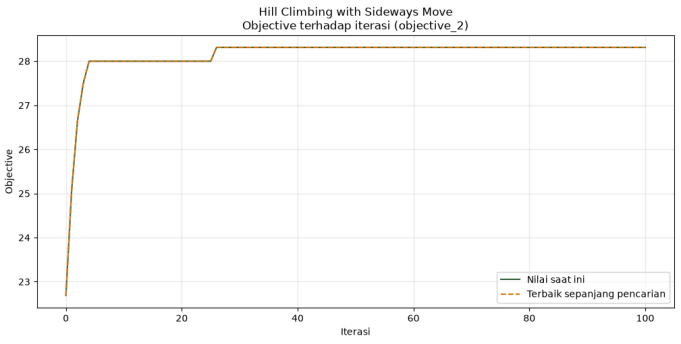
Nilai objektif akhir *Stochastic* lebih rendah daripada *Steepest Ascent* pada ketiga percobaan. Pada konfigurasi ini, pemilihan satu neighbor secara acak belum menghasilkan solusi sebaik pemeriksaan seluruh neighbor. Namun, hasil tersebut tidak berarti *Stochastic* selalu menghasilkan solusi yang lebih rendah pada semua data dan pengaturan pencarian. Durasi ketiga percobaan adalah 0.5358 detik, 0.5351 detik, dan 0.5391 detik. Rata-rata durasinya sebesar 0.5367 detik, lebih singkat daripada *Steepest Ascent* meskipun jumlah iterasinya lebih banyak. Hal ini dapat dijelaskan melalui perbedaan pekerjaan setiap iterasi. *Stochastic* menghitung nilai objektif satu neighbor yang dipilih, sedangkan *Steepest Ascent* menghitung nilai objektif seluruh neighbor.

2.6.3. Hill-Climbing with Sideways Move

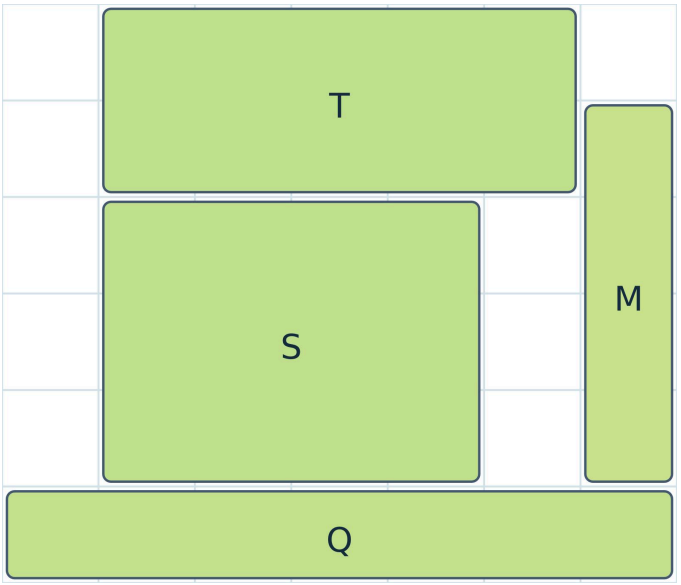
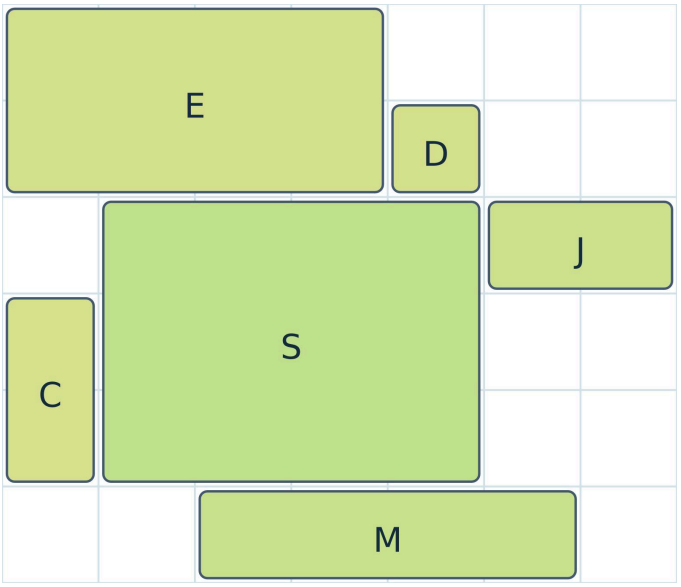
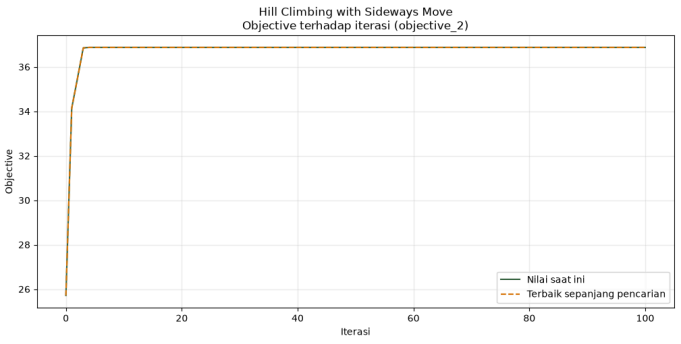
Tabel 2.6.3 Hasil Percobaan *Hill Climbing with Sideways Move*

Percobaan ke-	Nilai *initial objective function*	Nilai *final objective function*	Durasi (s)	Jumlah iterasi	Jumlah maksimum *sideways move*
1	22.69	28.31	10.8456	100	95/100
2	25.71	36.88	10.4481	100	96/100
3	18.94	24.88	11.5085	100	95/100

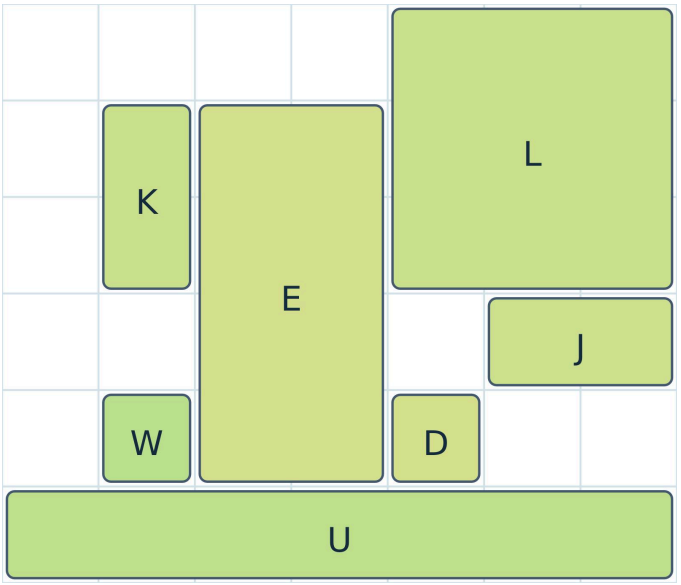
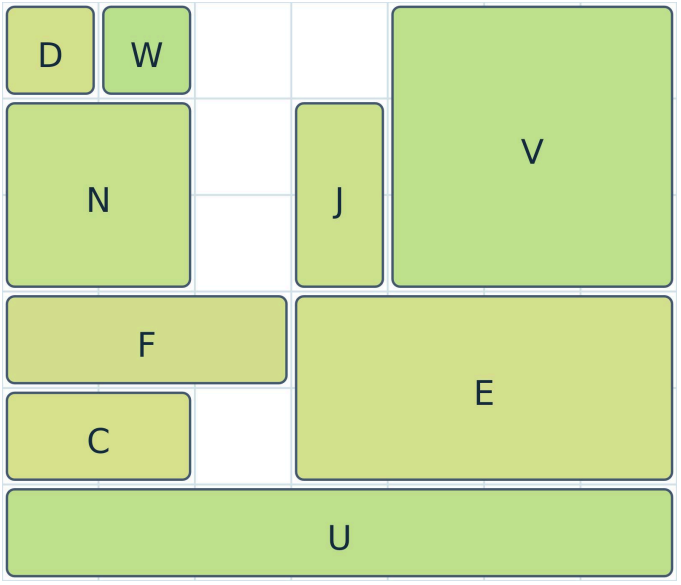
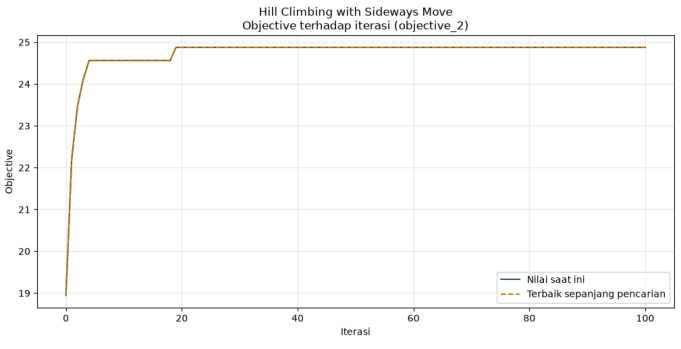
Percobaan 1

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 2

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 3

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Hill Climbing with Sideways Move menghasilkan nilai objektif akhir sebesar 28.31, 36.88, dan 24.88. Peningkatan terhadap nilai objektif awal masing-masing sebesar 5.62, 11.17, dan 5.94. Rata-rata nilai objektif akhirnya sebesar 30.02. Dibandingkan dengan *Steepest Ascent, Sideways Move* menghasilkan tambahan nilai

objektif sebesar 0.31 pada percobaan pertama dan 0.32 pada percobaan ketiga. Pada percobaan kedua, kedua algoritma memperoleh nilai objektif akhir yang sama, yaitu 36.88. Grafik percobaan pertama menunjukkan bahwa nilai objektif sempat bertahan di sekitar 28.00, kemudian meningkat menjadi 28.31. Pada percobaan ketiga, nilai objektif juga sempat bertahan di sekitar 24.56 sebelum meningkat menjadi 24.88. Kenaikan setelah rentang nilai objektif tetap tersebut menunjukkan manfaat melanjutkan pencarian melalui perubahan susunan yang bernilai sama. Bagian grafik yang mendatar tidak berarti kendaraan tidak mengalami perubahan posisi. *Sideways Move* dapat menerima perpindahan yang mengubah susunan kendaraan tanpa mengubah nilai objektifnya. Pada ketiga percobaan, jumlah *sideways move* yang dilakukan masing-masing sebanyak 95, 96, dan 95, dengan batas maksimum 100.

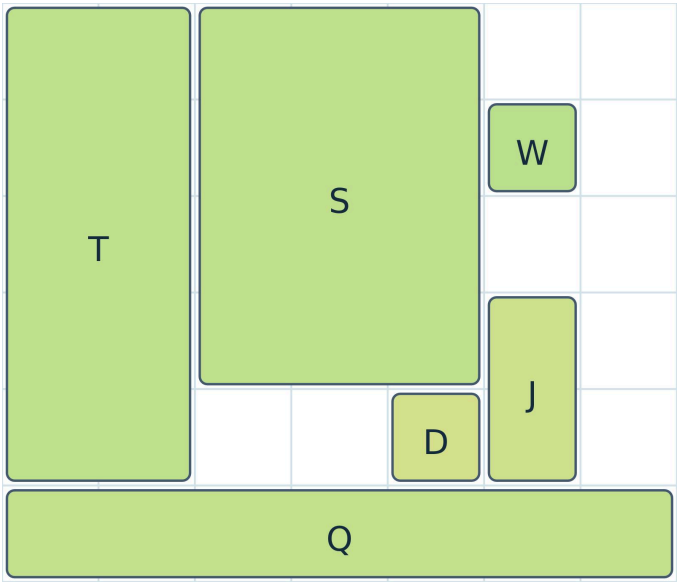
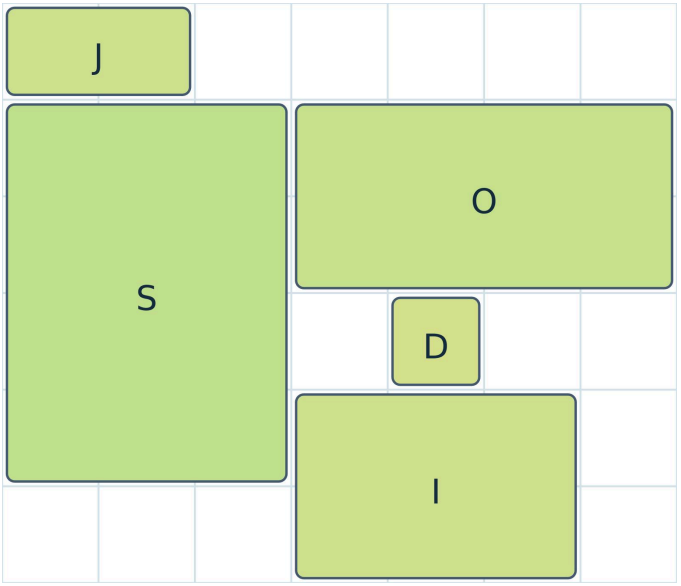
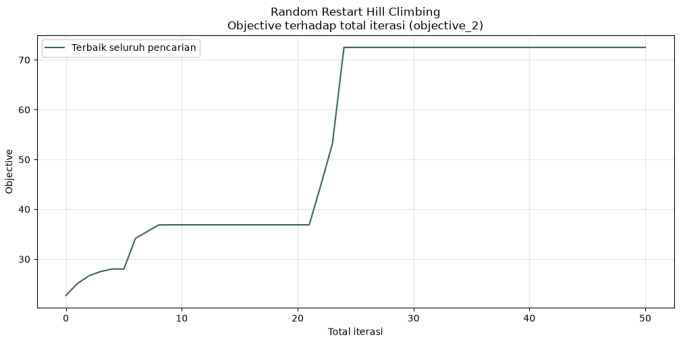
Seluruh percobaan berlangsung selama 100 iterasi. Jumlah *sideways move* yang tercatat belum mencapai batas maksimum, sehingga penghentian tidak dapat dinyatakan terjadi karena batas *sideways* telah tercapai. Informasi ini juga menunjukkan bahwa jumlah iterasi dan jumlah sideways move merupakan dua metrik yang berbeda.

Durasi percobaan pertama, kedua, dan ketiga masing-masing sebesar 10.8456 detik, 10.4481 detik, dan 11.5085 detik. Rata-rata durasinya sebesar 10.9341 detik. Durasi tersebut jauh lebih lama daripada *Steepest Ascent* karena pencarian terus memeriksa seluruh neighbor selama banyak iterasi, termasuk ketika hanya terjadi perpindahan dengan nilai objektif yang sama. Berdasarkan hasil tersebut, *Sideways Move* memberikan perbaikan pada dua dari tiga percobaan. Namun, tambahan nilai objektif yang diperoleh relatif kecil dibandingkan dengan penambahan durasi. Hal ini menunjukkan bahwa perpindahan bernilai sama dapat membantu pencarian, tetapi tidak selalu menghasilkan peningkatan yang besar.

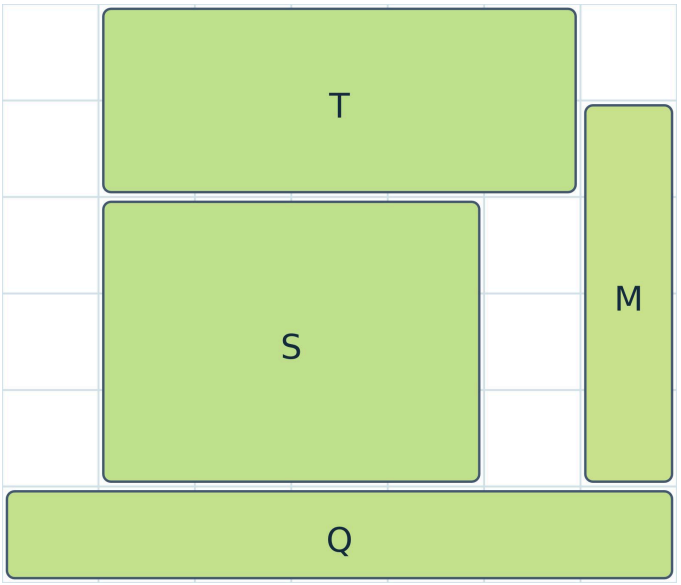
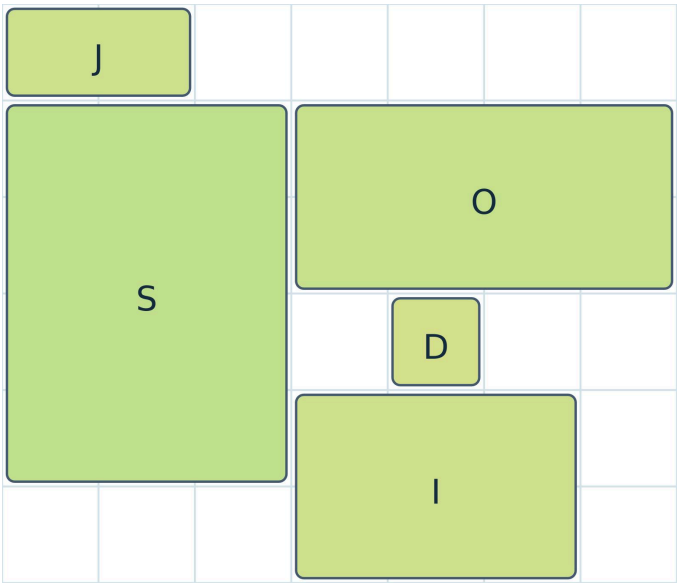
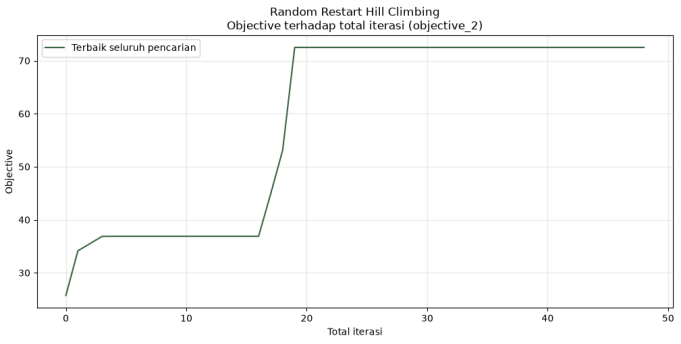
2.6.4. Random Restart Hill-Climbing

Tabel 2.6.4 Hasil Percobaan *Random Restart Hill Climbing*

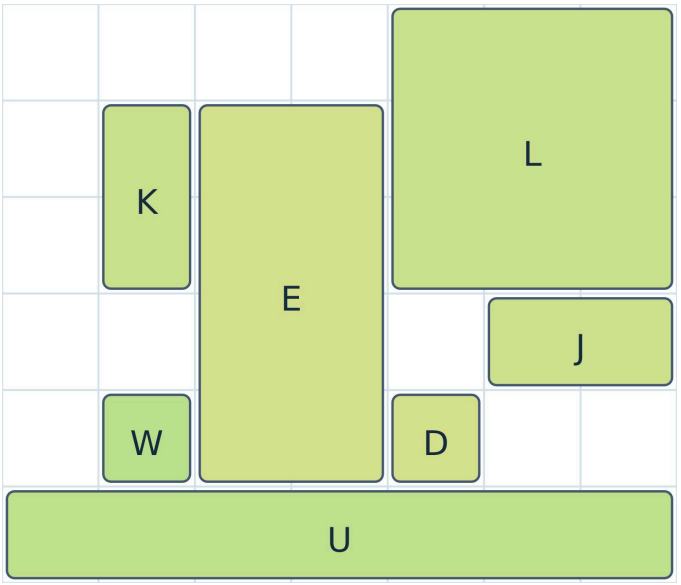
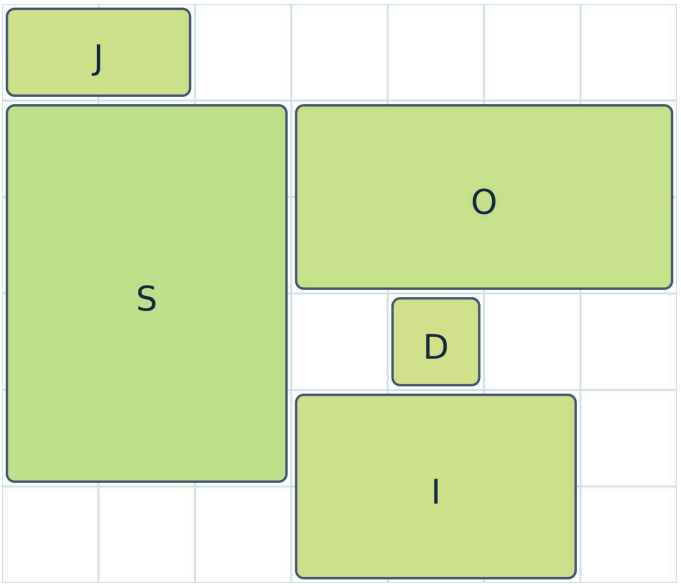
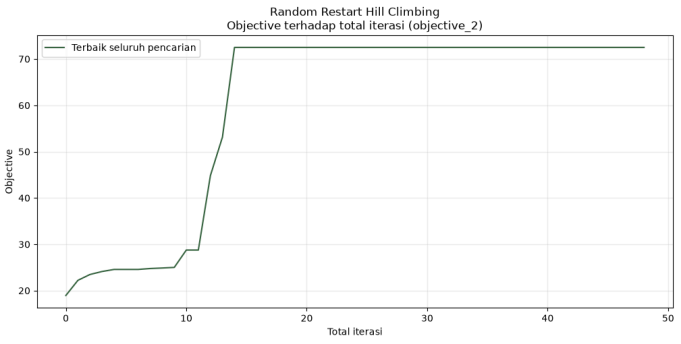
Percobaan ke-	Nilai *initial objective function*	Nilai *final objective function*	Durasi (s)	Jumlah iterasi	Max *restart*	Jumlah *restart*	Jumlah iterasi per *restart* (avg)
1	22.69	72.50	12.1217	50	10	10	4.55
2	25.71	72.50	11.3232	48	10	10	4.36
3	18.94	72.50	11.6445	48	10	10	4.36

Percobaan 1	
Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 2

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 3

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Random Restart menghasilkan nilai objektif akhir sebesar 72.50 pada ketiga percobaan. Peningkatan terhadap nilai objektif awal masing-masing sebesar 49.81, 46.79, dan 53.56. Rata-rata nilai objektif akhirnya juga sebesar 72.50, sehingga menjadi hasil tertinggi di antara keempat varian *Hill Climbing* pada percobaan ini.

Setiap percobaan menggunakan batas sepuluh *restart.* Sesuai implementasi, jumlah tersebut berarti satu pencarian awal dan sepuluh pencarian ulang, sehingga terdapat sebelas pencarian *Steepest Ascent*. Total iterasi yang tercatat adalah 50, 48, dan 48. Jika dibagi dengan sebelas pencarian, rata-rata iterasi setiap pencarian masing-masing sekitar 4,55, 4,36, dan 4,36. Grafik menunjukkan beberapa peningkatan nilai objektif terbaik, termasuk kenaikan yang lebih besar pada bagian tengah pencarian. Pola ini sesuai dengan penggunaan beberapa state awal yang memungkinkan algoritma menemukan solusi di bagian ruang pencarian yang berbeda. Setelah nilai objektif 72.50 ditemukan, grafik tetap mendatar hingga seluruh pencarian selesai. Grafik *Random Restart* menampilkan nilai objektif terbaik sepanjang proses. Oleh karena itu, garis yang mendatar tidak berarti setiap pencarian berikutnya menghasilkan state dengan nilai objektif 72.50. Pencarian berikutnya dapat menghasilkan nilai yang lebih rendah, tetapi hasil tersebut tidak menggantikan solusi terbaik. Perjalanan nilai objektif setiap pencarian perlu dilihat melalui `history_per_restart.` Visualisasi hasil akhir ketiga percobaan menunjukkan susunan kendaraan yang sama, yaitu kendaraan J, S, O, D, dan I. Berdasarkan data kendaraan, susunan tersebut memiliki total tarif pengiriman sebesar 290, total berat 15, dan ETA maksimum satu hari. Dengan kapasitas kapal sebesar 15, perhitungan `objective_2` menghasilkan skor 72.50. Hasil tersebut memperlihatkan pengaruh ETA terhadap pemilihan kendaraan. Susunan dengan skor lebih tinggi tidak harus mengangkut kendaraan lebih banyak atau memiliki total tarif lebih besar. Pada objective tambahan ini, ETA maksimum yang lebih rendah dapat meningkatkan skor meskipun total tarifnya lebih kecil daripada susunan lain.

Durasi ketiga percobaan adalah 12.1217 detik, 11.3232 detik, dan 11.6445 detik, dengan rata-rata 11.6965 detik. Durasi yang lebih panjang berkaitan dengan pengulangan pembentukan state awal dan pelaksanaan *Steepest Ascent.* Kesamaan hasil akhir menunjukkan konsistensi pada tiga percobaan yang dicatat. Namun, nilai 72,50 belum dapat dinyatakan sebagai global optimum hanya berdasarkan hasil tersebut. Kesimpulan yang dapat diberikan adalah *Random Restart* memperoleh nilai objektif akhir tertinggi di antara empat varian *Hill Climbing* pada data dan pengaturan percobaan ini.

2.6.5. Simulated Annealing

Temperatur awal: 1000

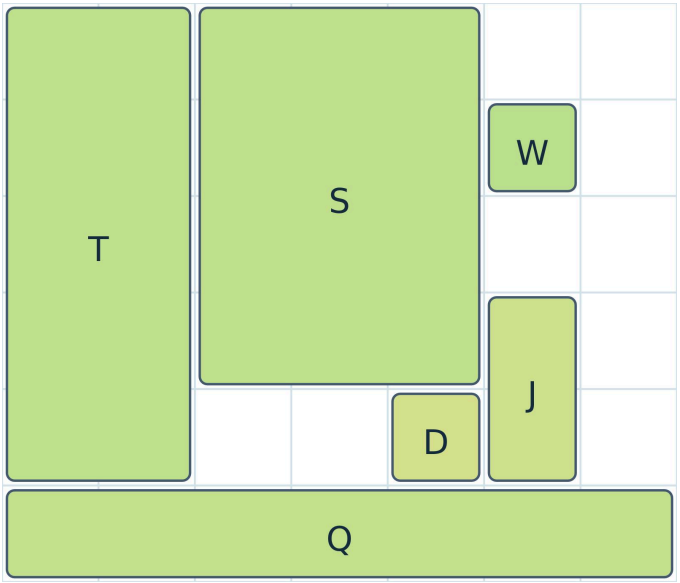
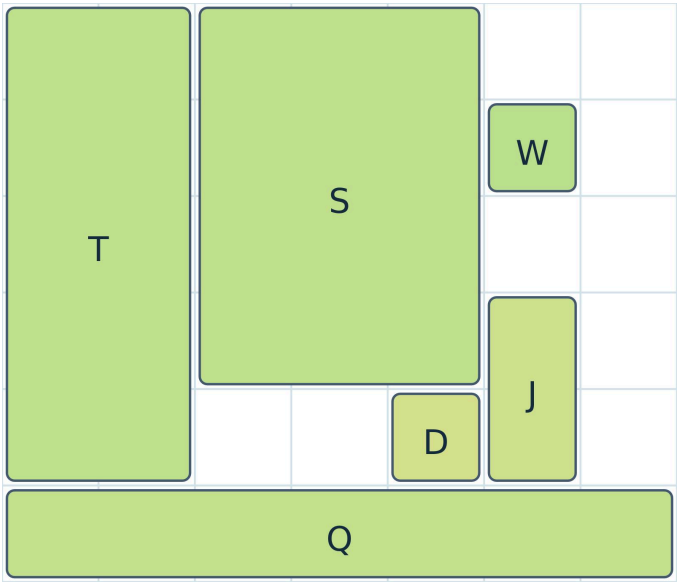
Temperatur minimum: 0.001

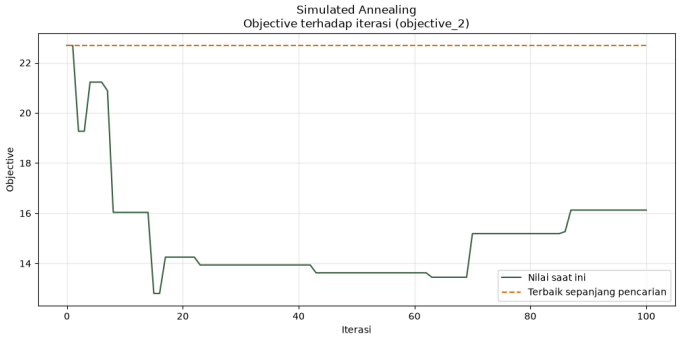
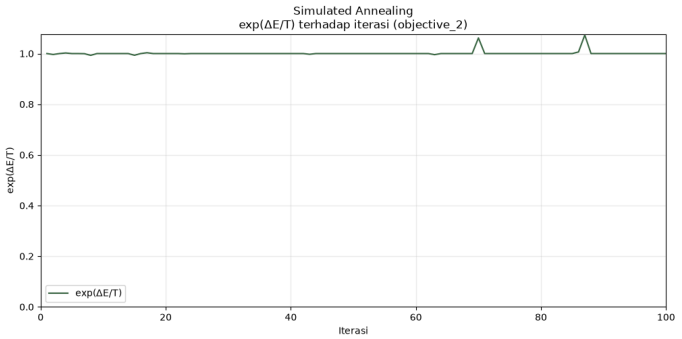
Cooling rate: 0.95

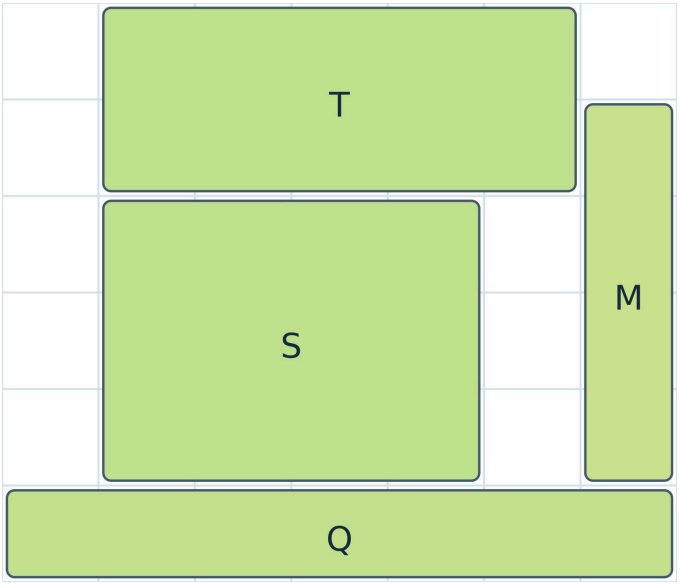
Jumlah iterasi maksimum: 100

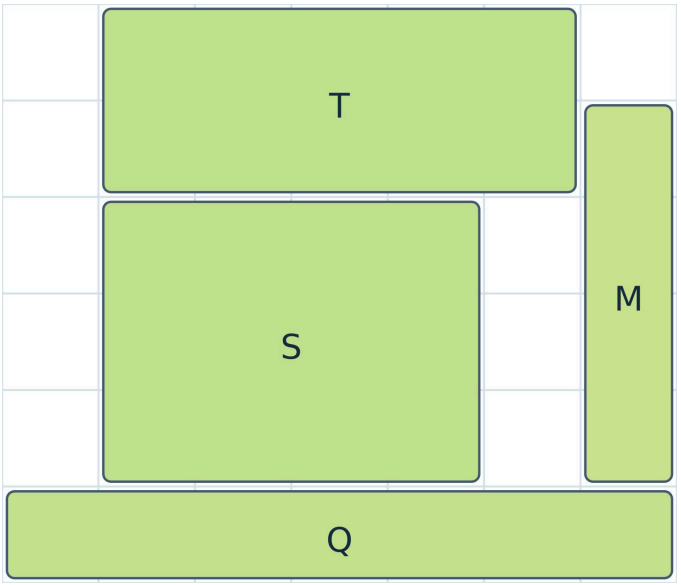
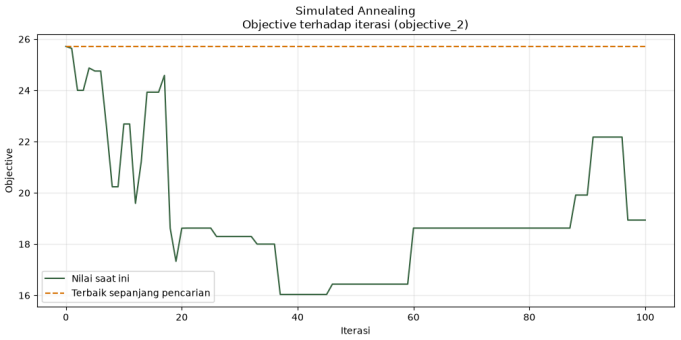
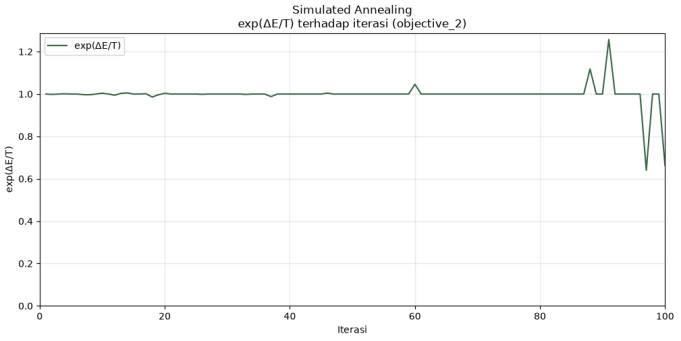
Tabel 2.6.5 Hasil Percobaan *Simulated Annealing*

Percobaan ke-	Jumlah Iterasi	Nilai *initial objective function*	Nilai *final objective function*	Temperatur Akhir	Durasi (*s*)	Frekuensi *stuck* di *local optima*
1	100	22.69	22.69	5.920529	22.82839	4
2	100	25.71	25.71	5,920529	24,44500	6
3	100	18.94	19.59	5,920529	24.94490	2

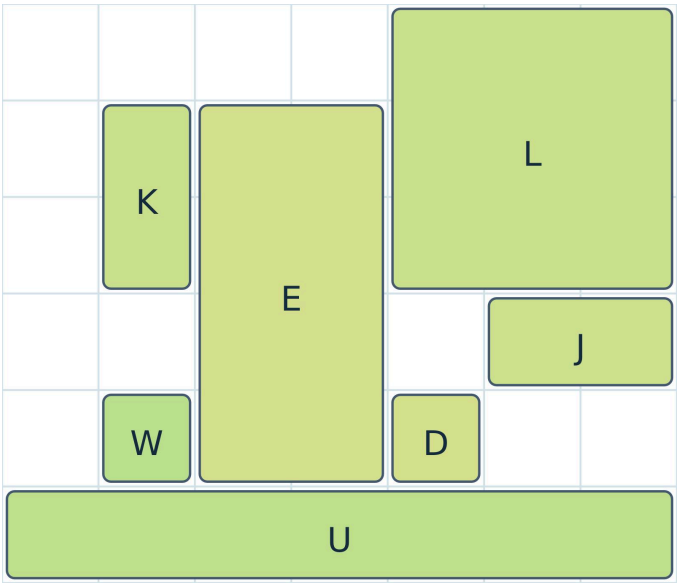
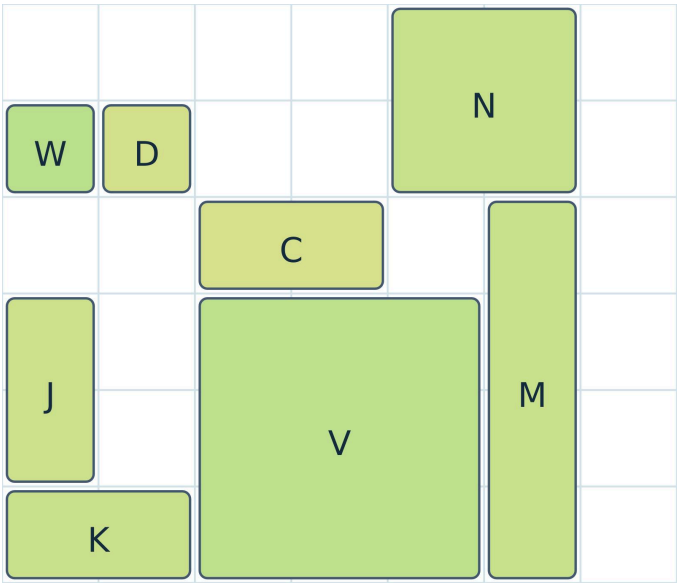
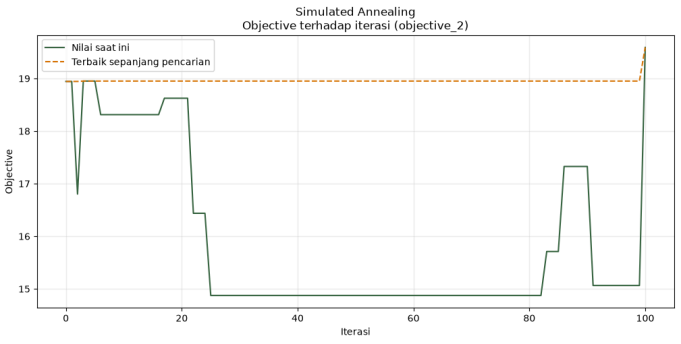
Percobaan 1	
Initial state	
Best state	

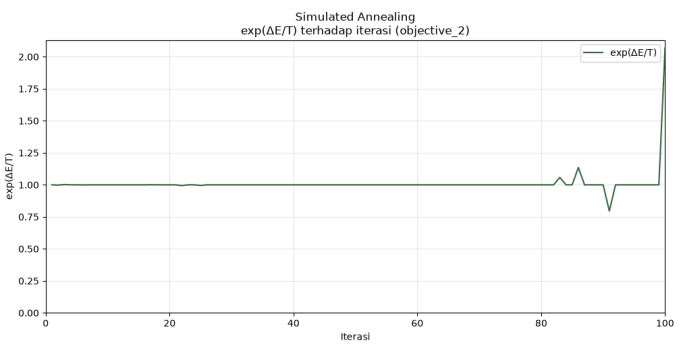
Plot *objective function* terhadap banyak iterasi	
Plot nilai eksponen terhadap iterasi	

Percobaan 2	
Initial state	

Best state	
Plot *objective function* terhadap banyak iterasi	
Plot nilai eksponen terhadap iterasi	

Percobaan 2

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Plot nilai eksponen terhadap iterasi	

Simulated Annealing menghasilkan nilai objektif terbaik sebesar 22.69, 25.71, dan 19.59 pada ketiga percobaan yang dilakukan. Jika dibandingkan dengan nilai objektif awal, percobaan pertama dan kedua tidak mengalami peningkatan. Pada percobaan ketiga, terdapat peningkatan nilai dari *initial* objektif sebesar 0.65. Rata-rata nilai objektif yang diperoleh adalah 22.66. Hasil ini menunjukkan bahwa penerimaan perpindahan dengan nilai lebih rendah belum menghasilkan perbaikan pada 2 dari 3 total percobaan dalam konfigurasi ini.

Grafik memperlihatkan perbedaan antara nilai objektif state saat ini dan nilai terbaik sepanjang pencarian. Pada percobaan pertama, nilai objektif sempat menurun hingga berada di bawah 14, kemudian meningkat kembali tanpa melampaui nilai awal sebesar 22.69. Percobaan kedua juga menunjukkan pola yang mirip dengan nilai tidak lebih dari 25.71. Pada percobaan ketiga sempat terjadi penurunan secara signifikan, namun memperoleh peningkatan hingga melampaui nilai objektif awal dengan kendaraan terdefinisi yakni W, D, N, C, V, M, J, dan K. Pola ini sesuai dengan mekanisme algoritma *Simulated Annealing* yang memungkinkan penerimaan *neighbor* dengan nilai objektif lebih rendah. Dalam konteks pemuatan geladak kapal, perpindahan dapat terjadi ke susunan kendaraan dengan skor lebih kecil untuk melanjutkan penelusuran ke kemungkinan lain, melampaui *local maximum*. Penilaian skor sendiri diperoleh dari kombinasi total tarif pengiriman, ETA maksimum, dan rasio penggunaan kapasitas berat, melalui `objective_2.`

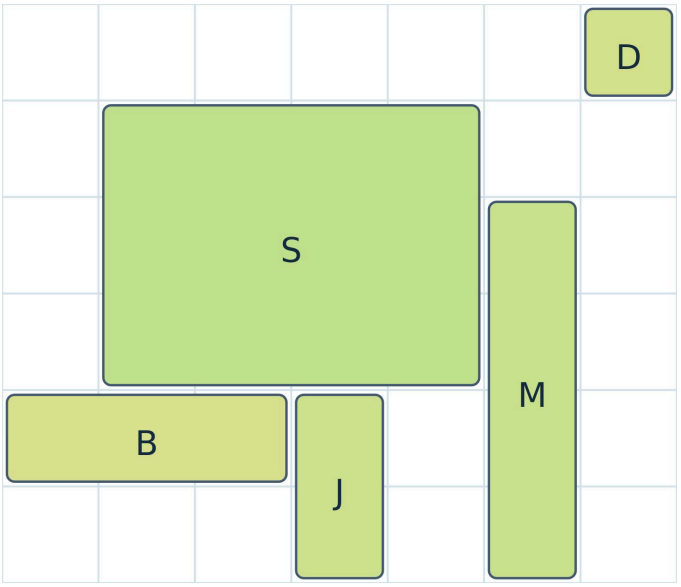
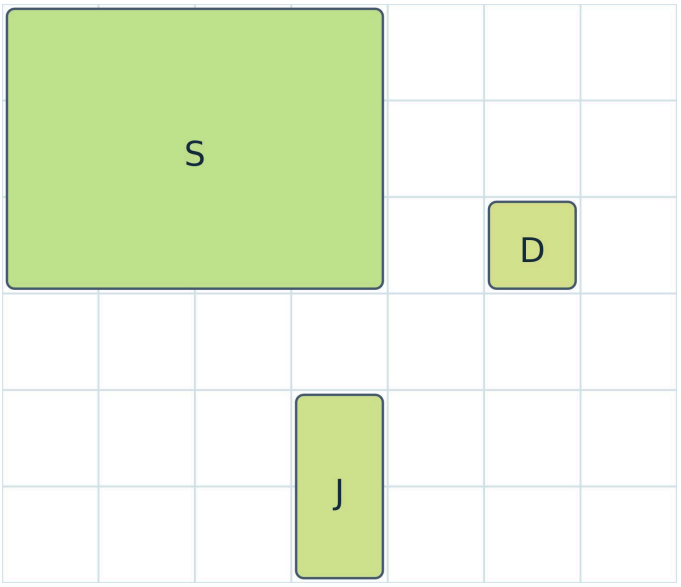
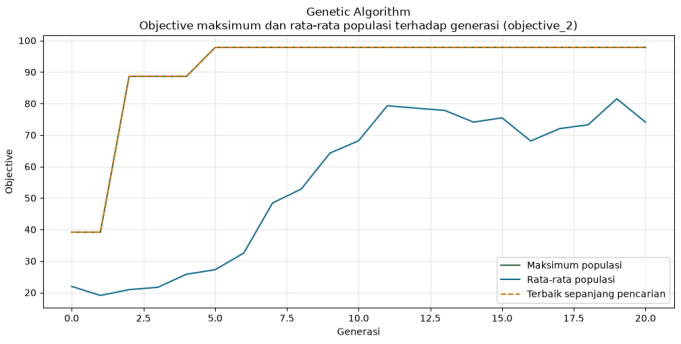
Masing-masing percobaan berlangsung selama 100 iterasi dengan temperatur awal 1000 dan *cooling rate* 0.95. Frekuensi `stuck_count` yang tercatat masing-masing sebesar 4, 6, 2. Perhitungan frekuensi ini bertambah apabila *neighbor* tidak menemukan kandidat dengan nilai objektif lebih tinggi dibanding *state* saat ini. Angka tersebut secara tidak langsung merepresentasikan jumlah *local maximum* yang berbeda. Sementara jika ditinjau dari durasi, rata-ratanya berada di sekitar 24.07 detik. Hasil ini memperlihatkan bahwa kemampuan menerima penurunan nilai objektif dapat membuka kemungkinan untuk menuju *global optimum*, meski belum menjamin diperolehnya solusi yang lebih baik dengan batas yang telah didefinisikan.

2.6.6. Genetic Algorithm

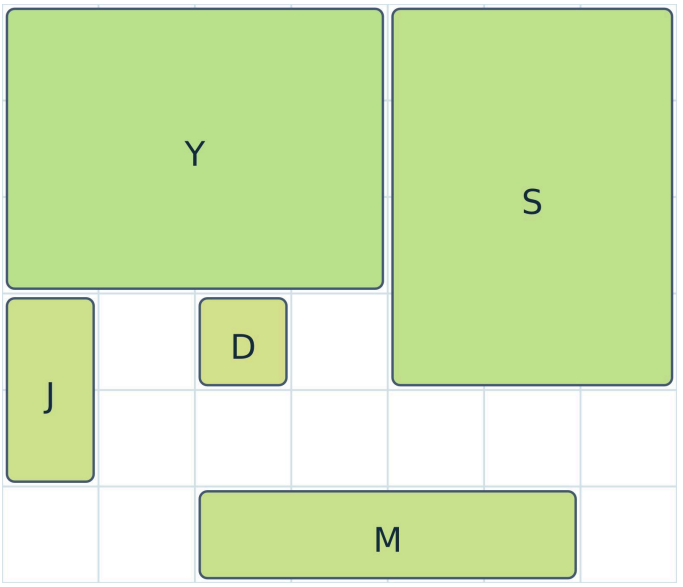
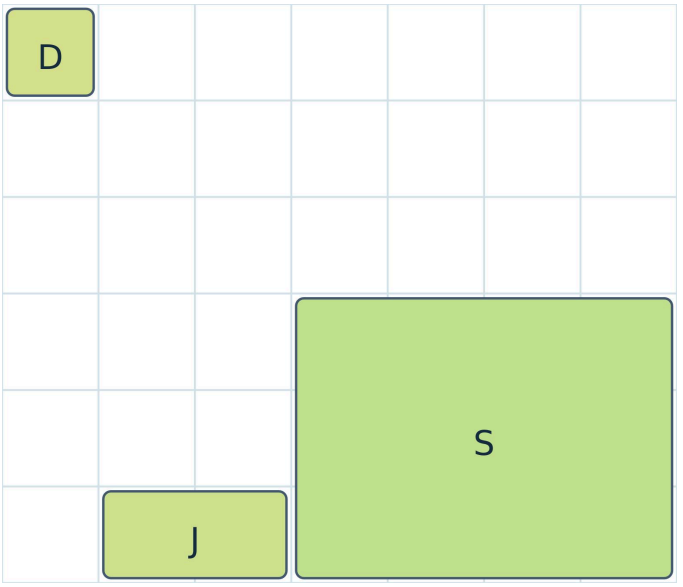
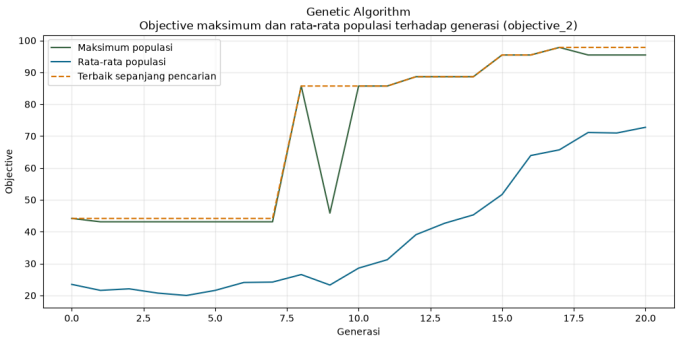
Tabel 2.6.6 Hasil Percobaan *Genetic Algorithm*

Percobaan	*Jumlah Generasi*	*Jumlah Populasi*	Nilai *initial objective function*	Nilai *final objective function*	Durasi (*s*)
1	20	50	39.17	97.83	19.0477
2	20	50	44.17	97.83	17.6836
3	20	50	40.63	97.83	17.0887
4	20	100	40.83	97.83	34.1992
5	20	100	44.17	97.83	39.9462
6	20	100	47.41	97.83	35.0281
7	20	150	40.83	97.83	51.9375
8	20	150	44.17	97.83	55.0074
9	20	150	47.41	97.83	55.9050
10	10	50	39.17	97.83	9.6172
11	10	50	44.17	85.17	8.9991
12	10	50	40.63	97.83	8.7227
13	20	50	39.17	97.83	18.4688
14	20	50	44.17	97.83	17.9121
15	20	50	40.63	97.83	16.6695
16	40	50	39.17	97.83	36.4147
17	40	50	44.17	97.83	36.3945
18	40	50	40.63	97.83	34.6056

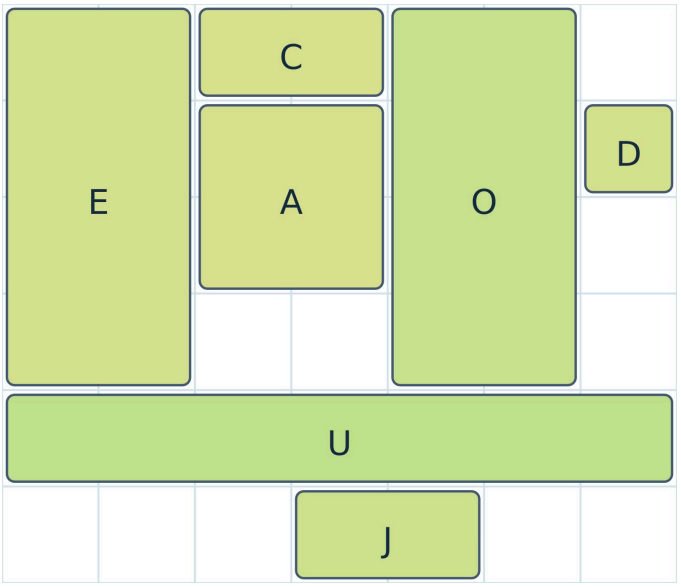
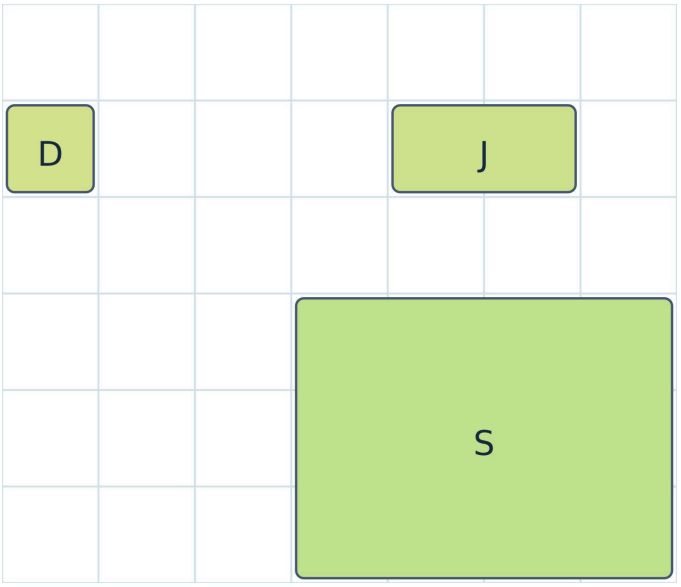
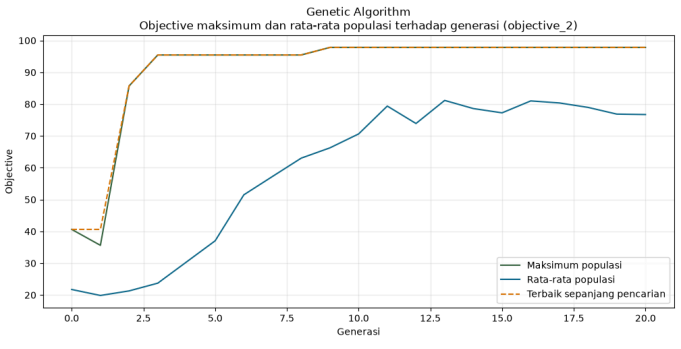
Percobaan 1

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

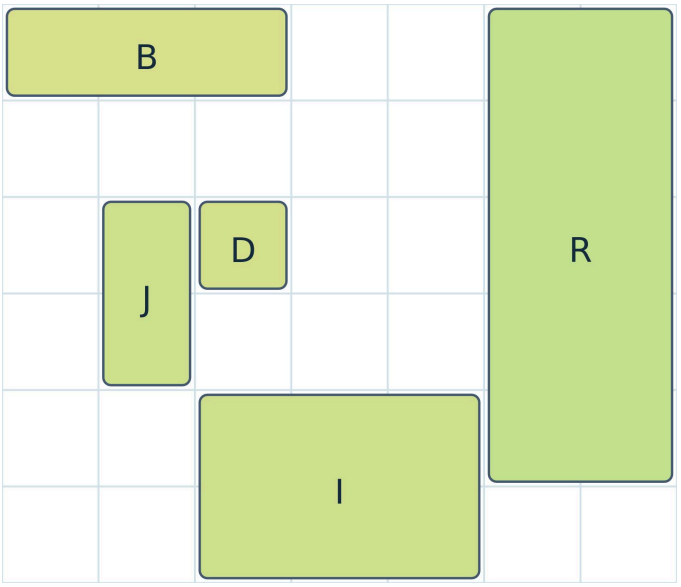
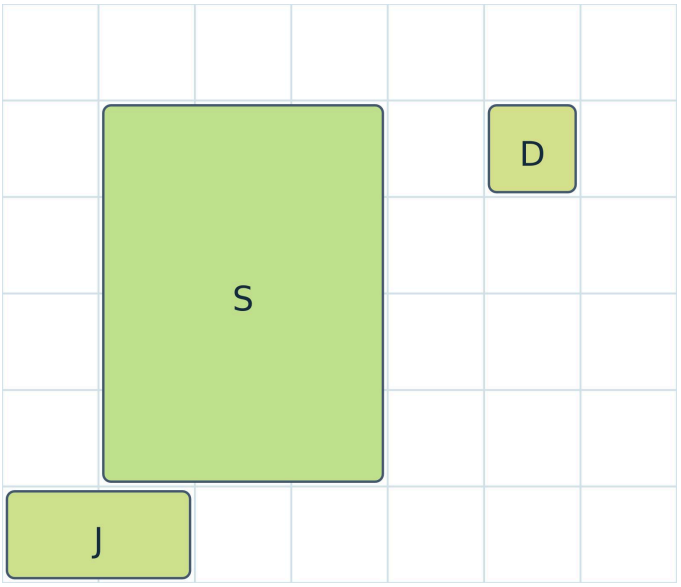
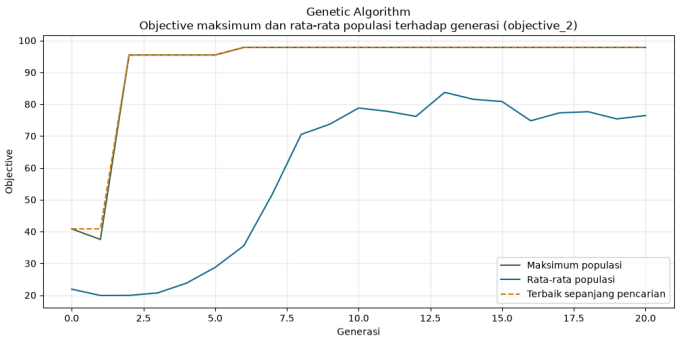
Percobaan 2

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

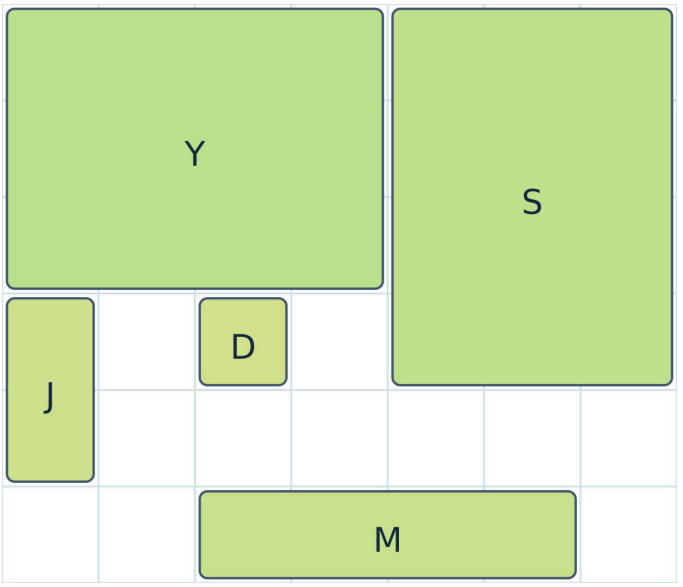
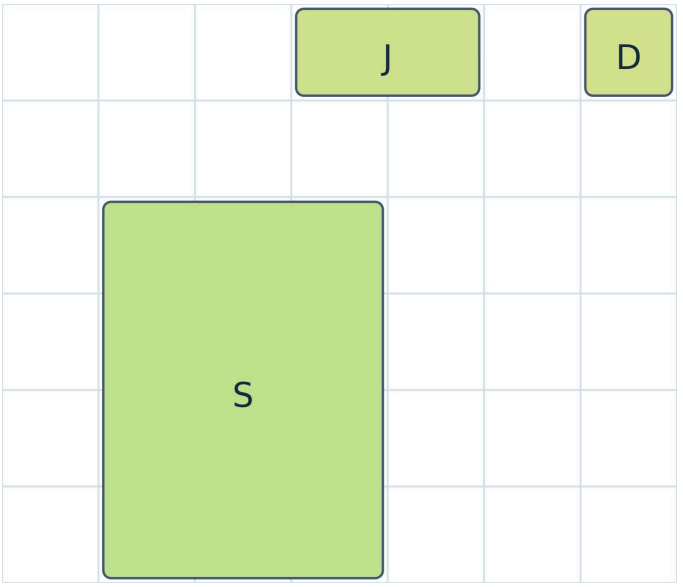
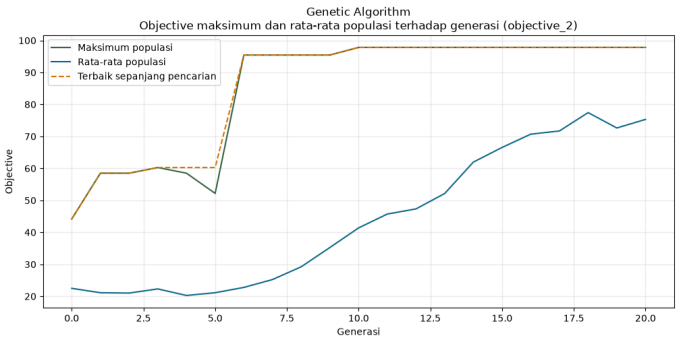
Percobaan 3

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

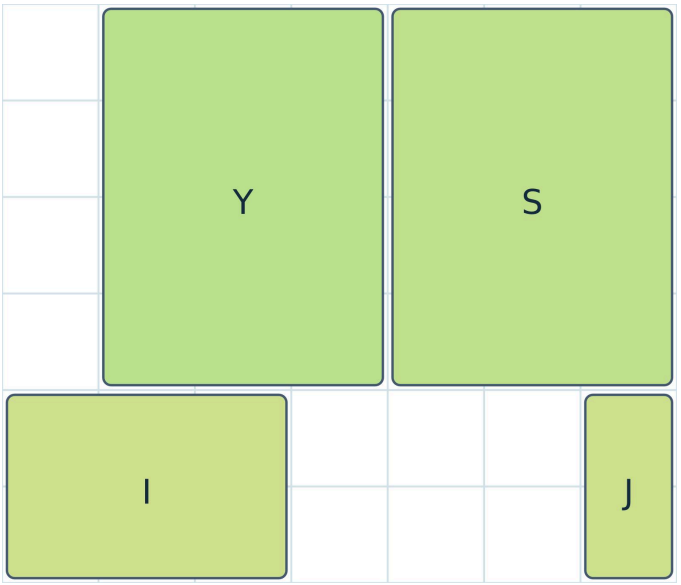
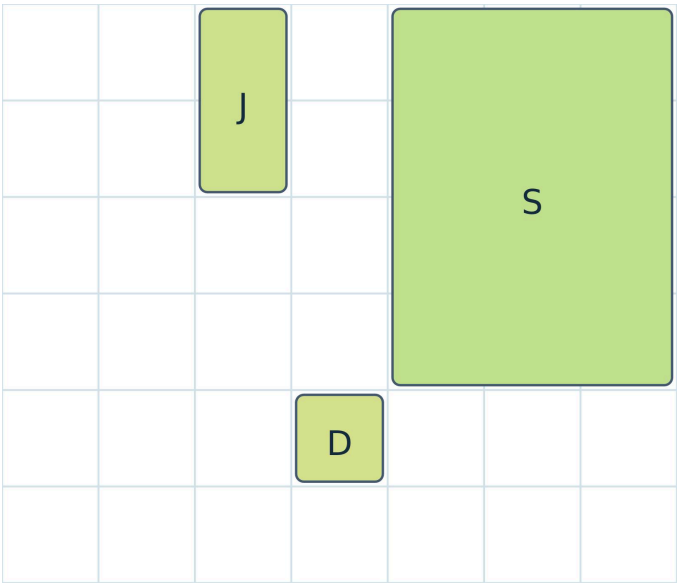
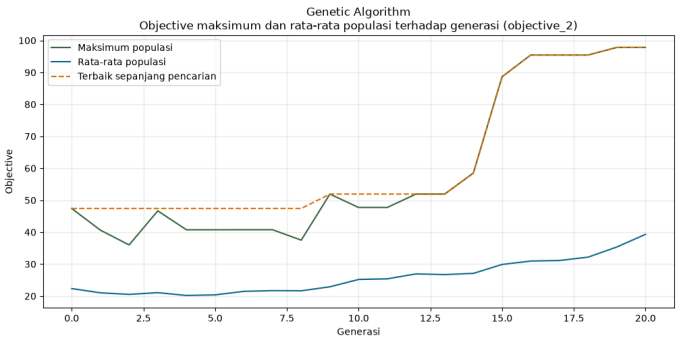
Percobaan 4

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

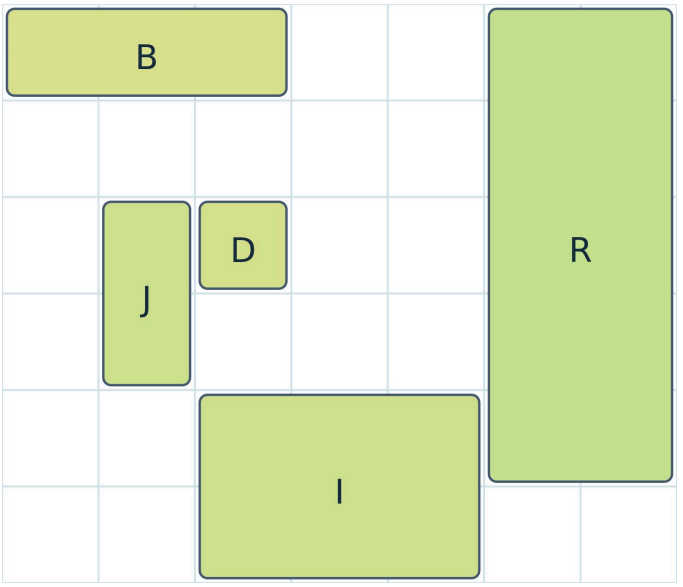
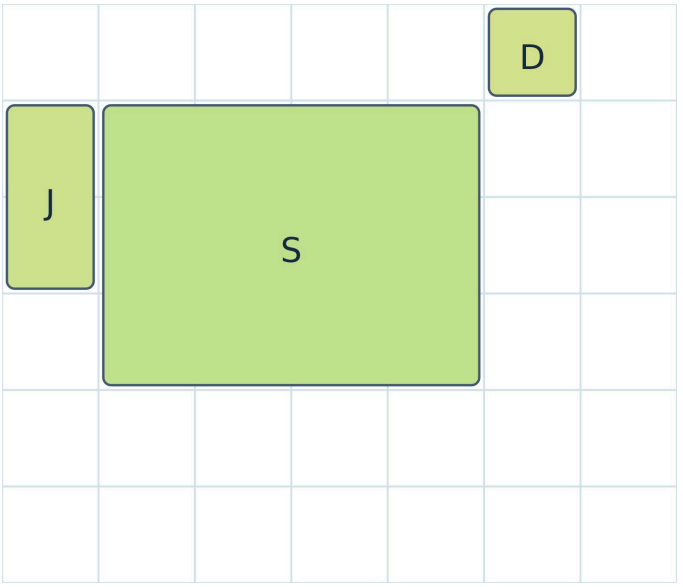
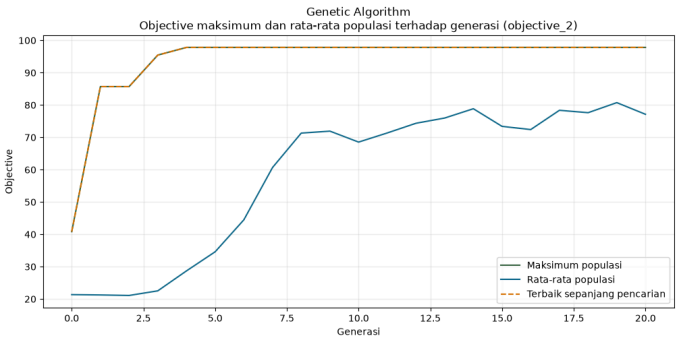
Percobaan 5

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

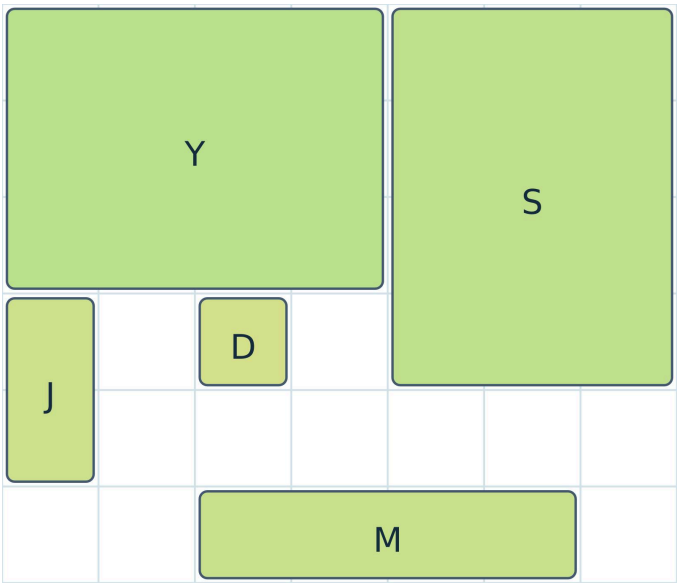
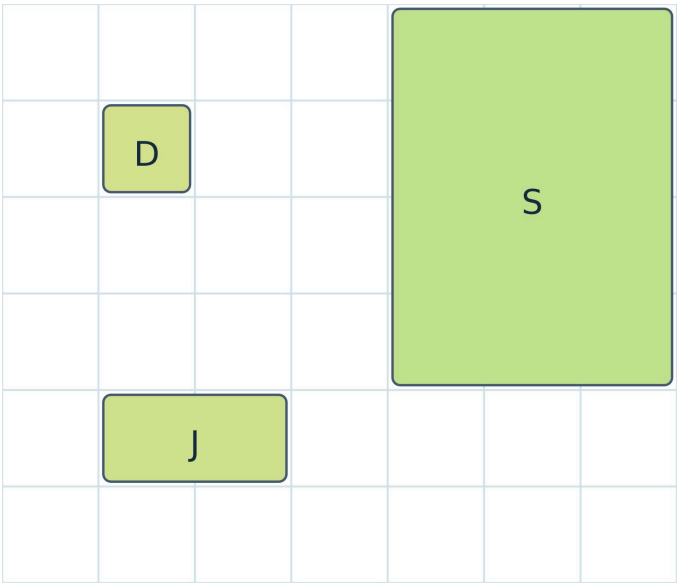
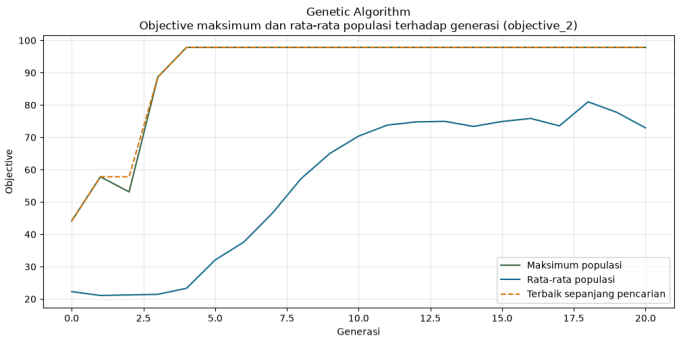
Percobaan 6

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

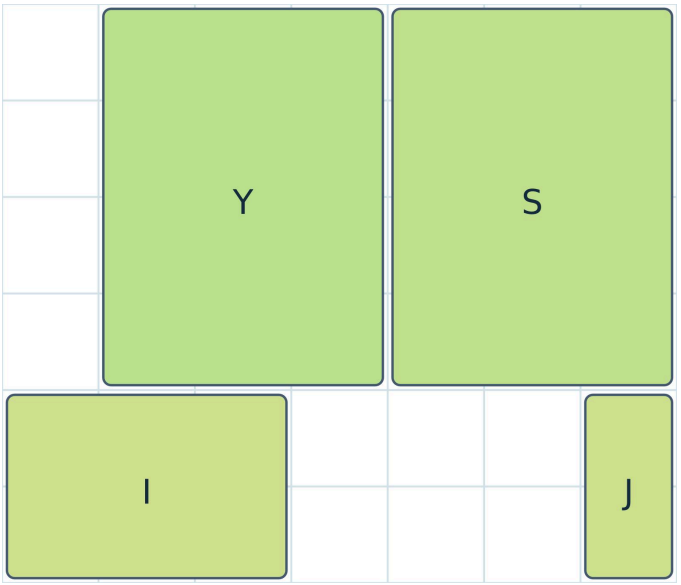
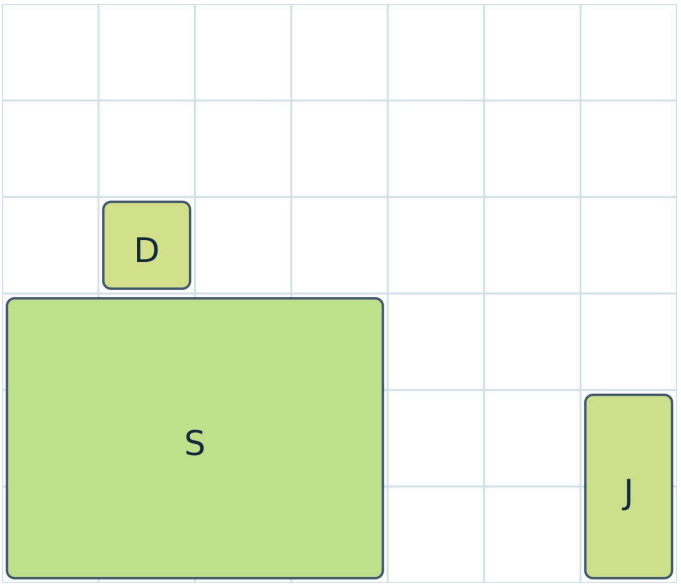
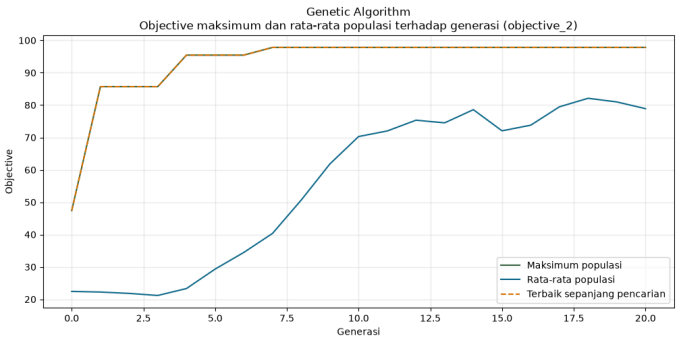
Percobaan 7

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

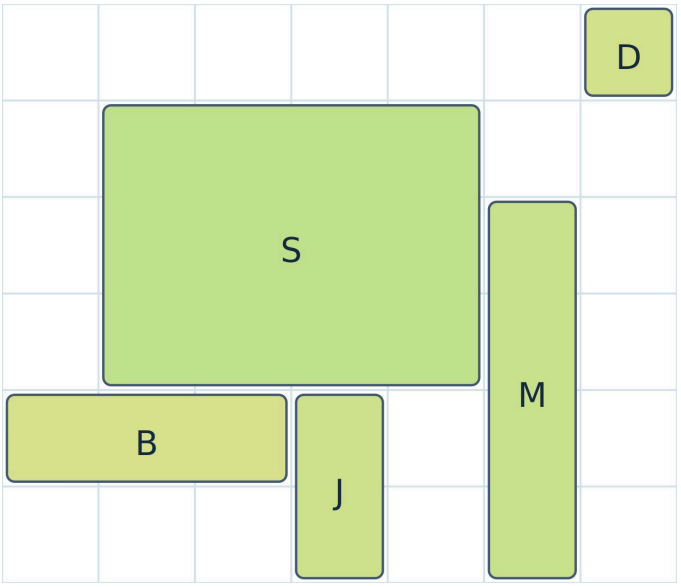
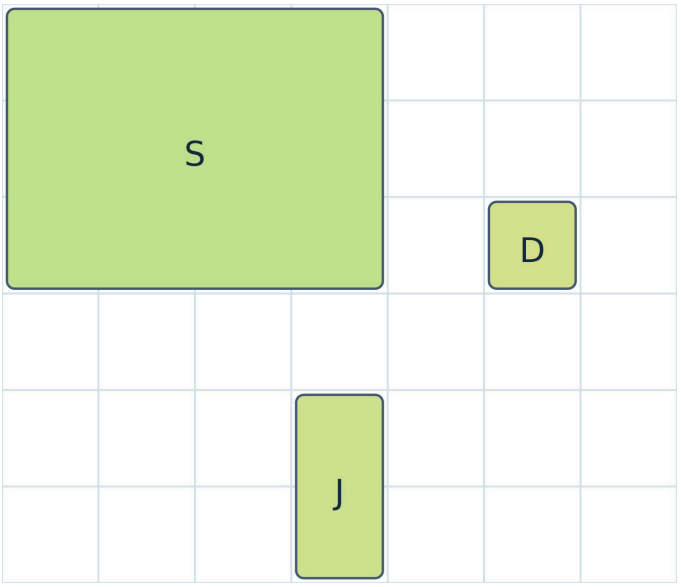
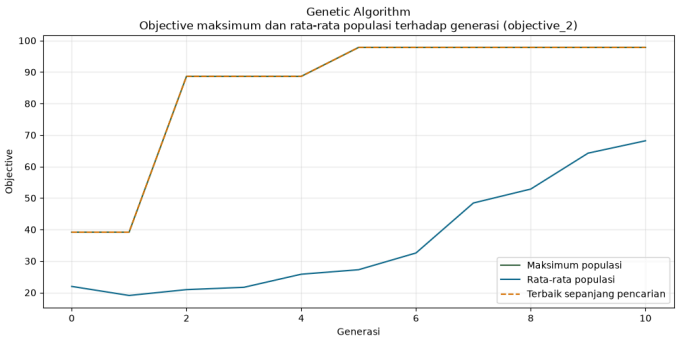
Percobaan 8

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

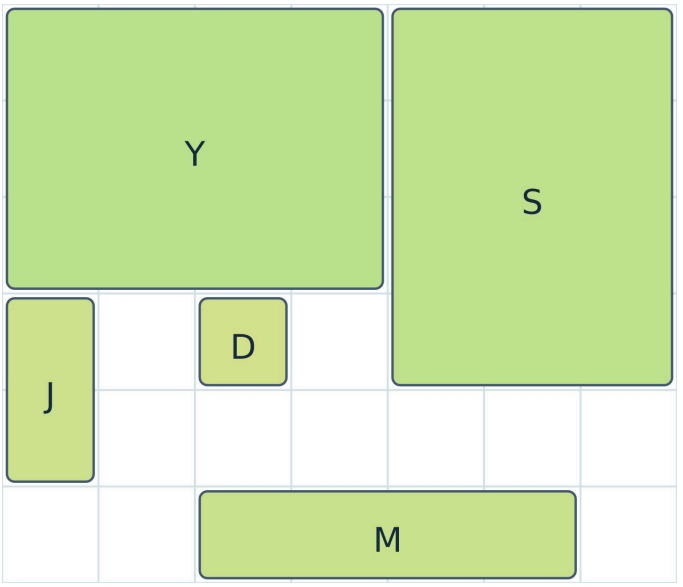
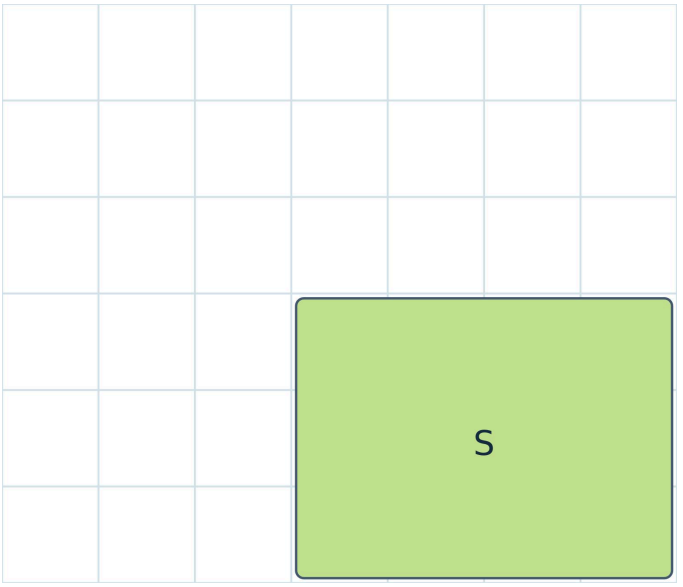
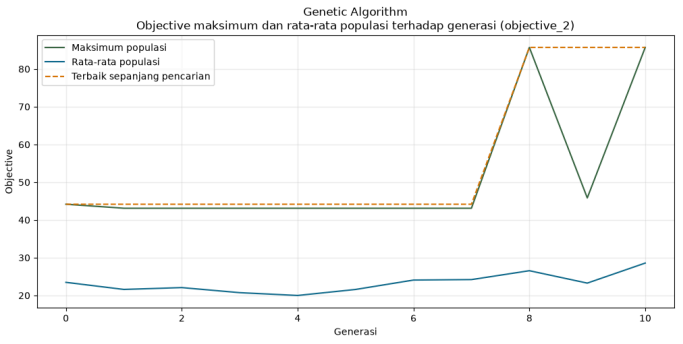
Percobaan 9

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

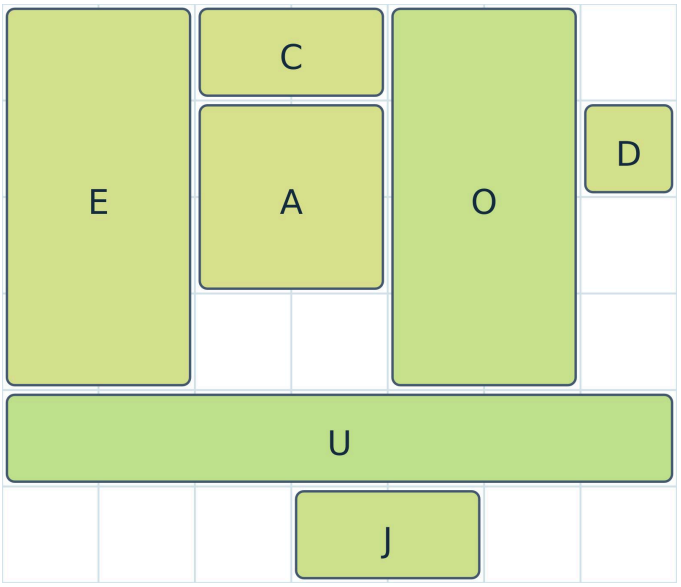
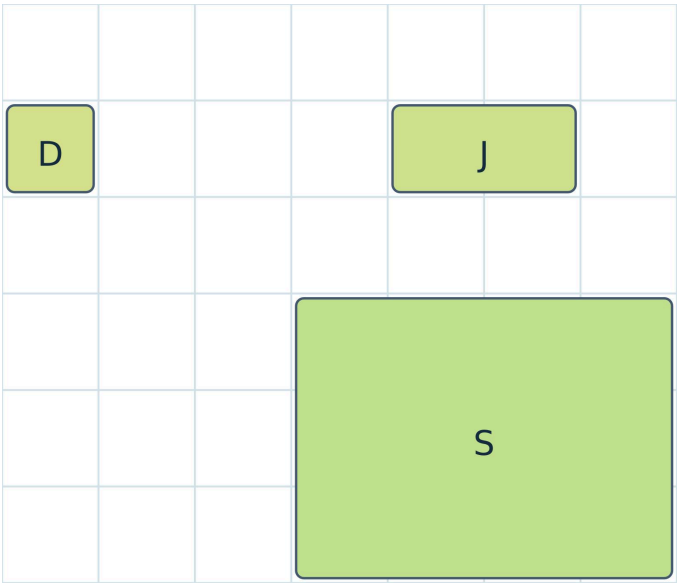
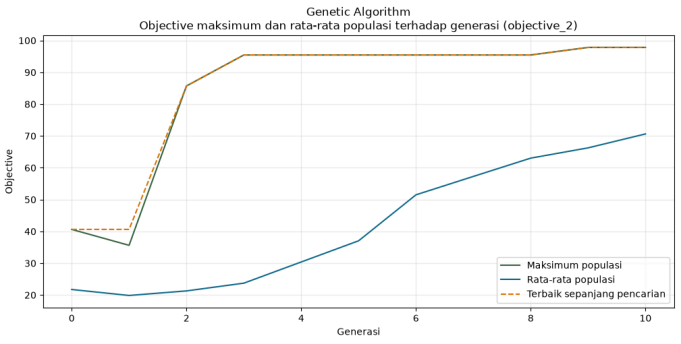
Percobaan 10

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

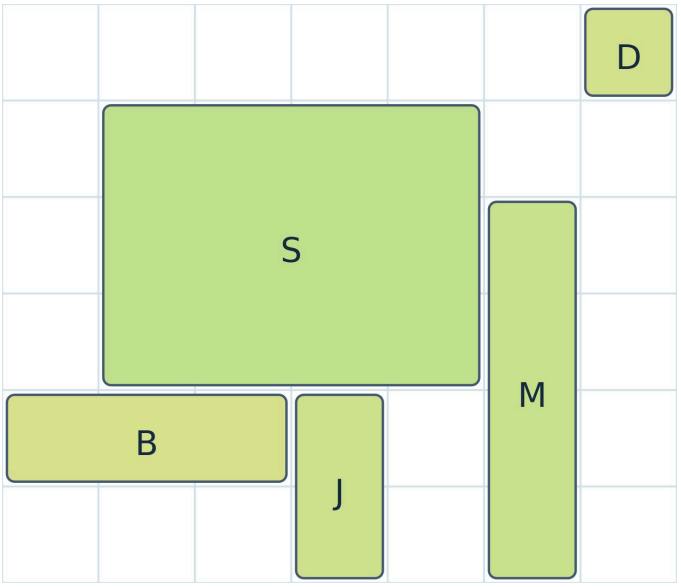
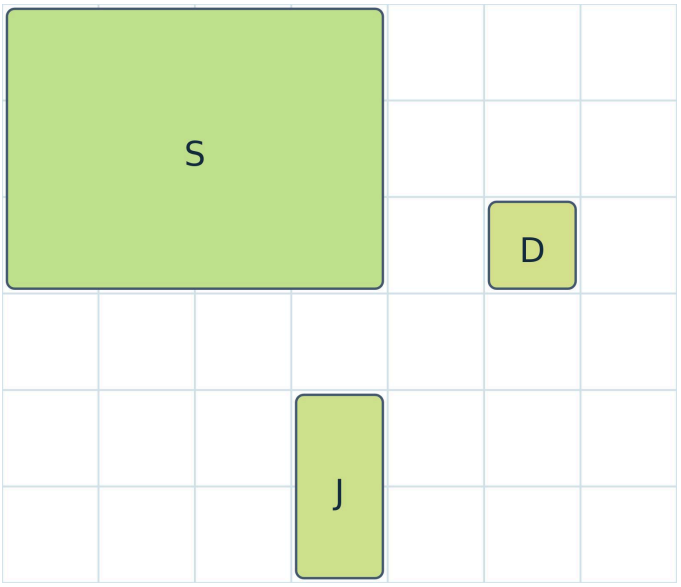
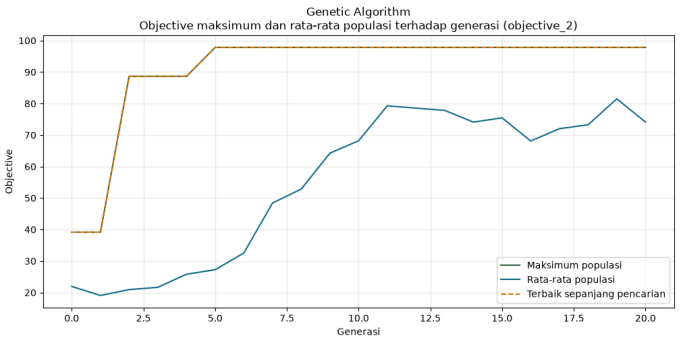
Percobaan 11

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

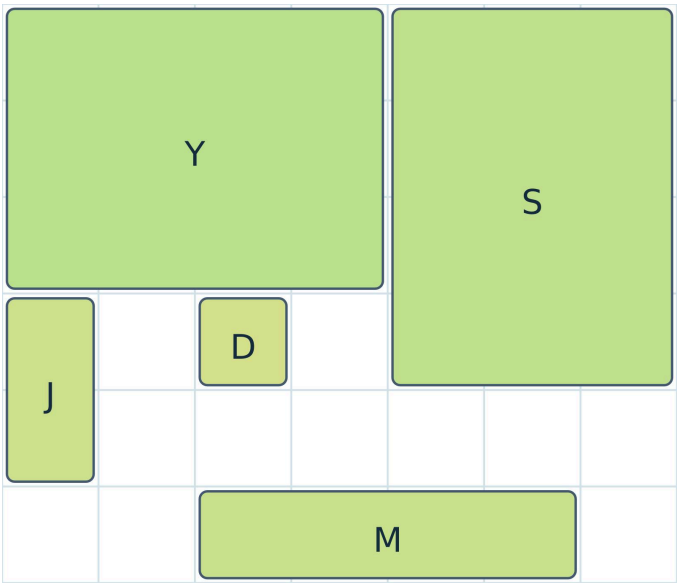
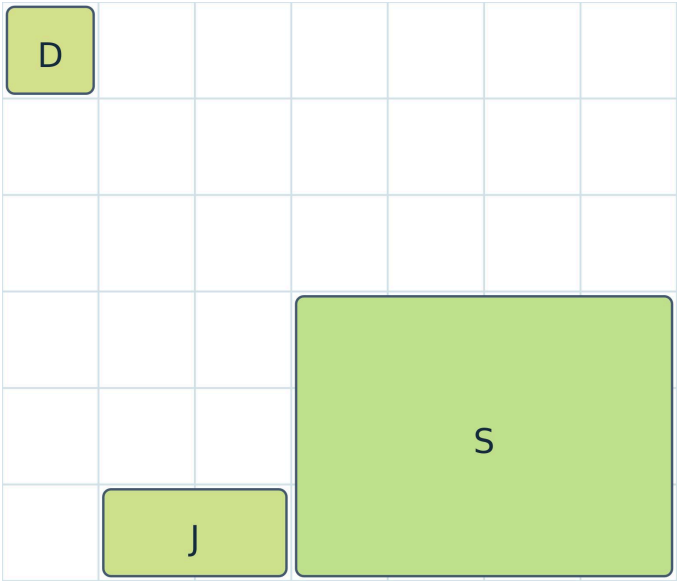
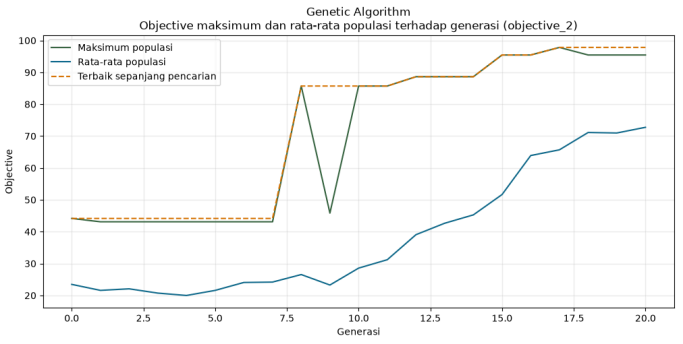
Percobaan 12

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

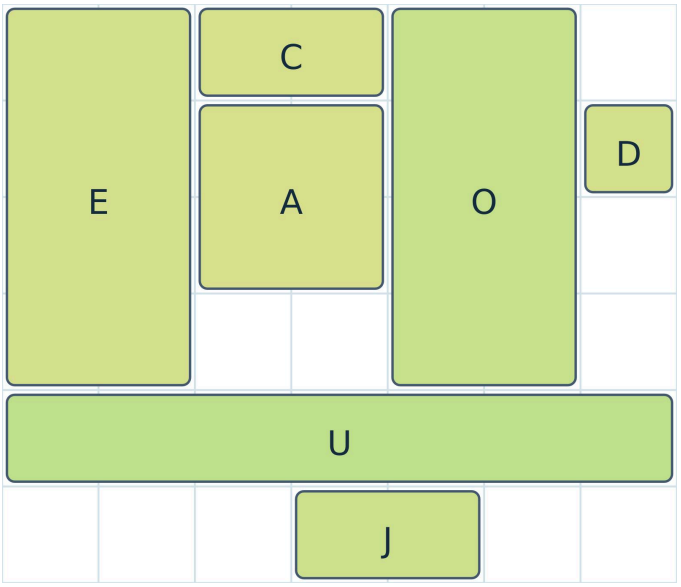
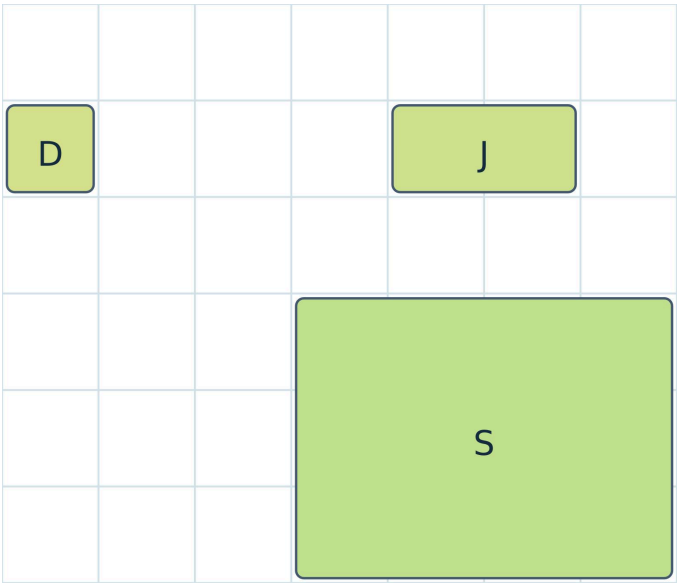
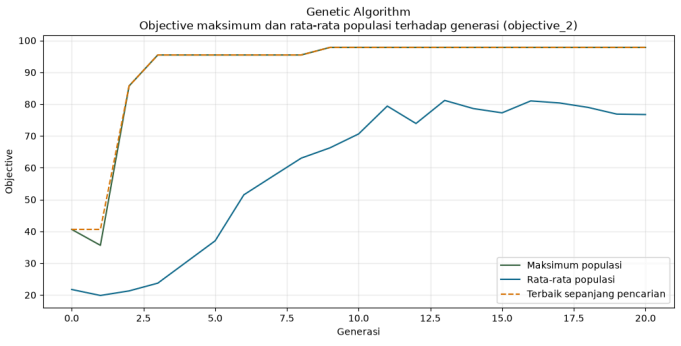
Percobaan 13

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

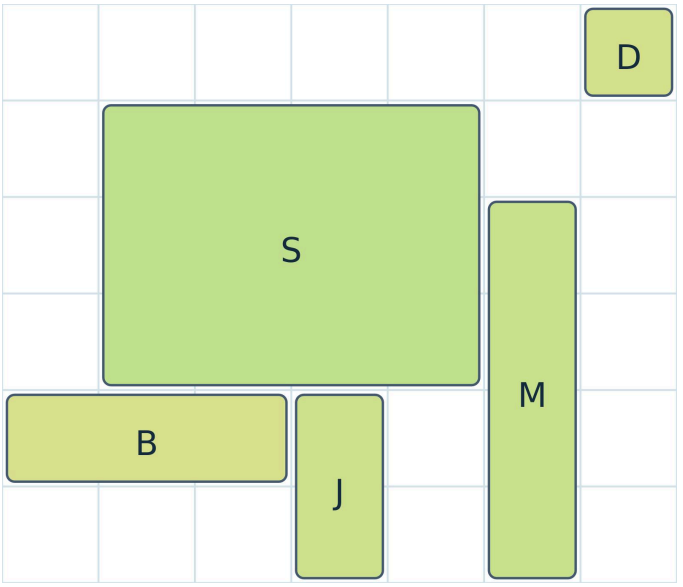
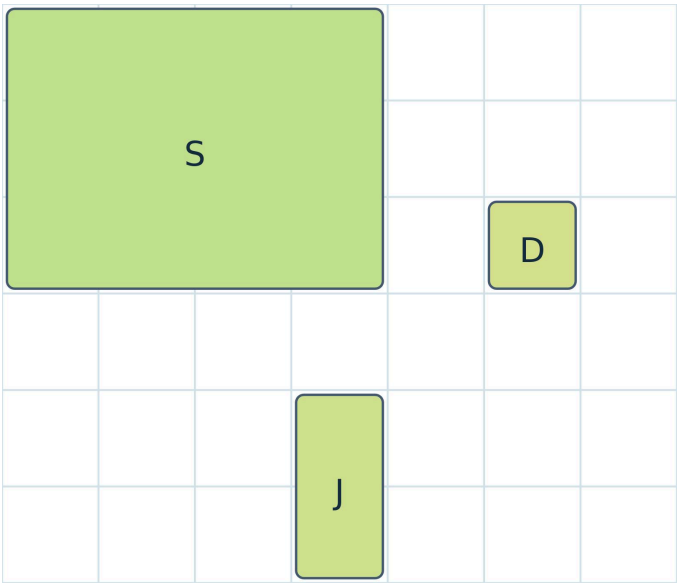
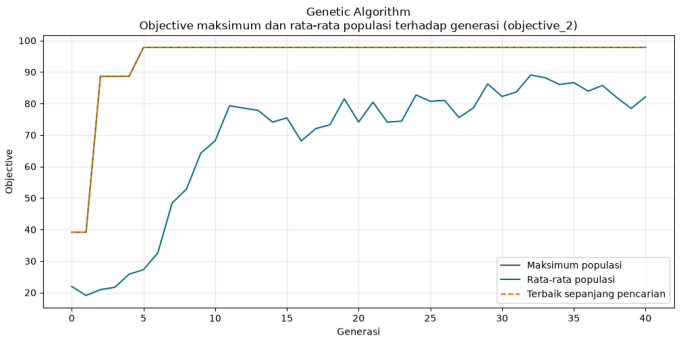
Percobaan 14

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

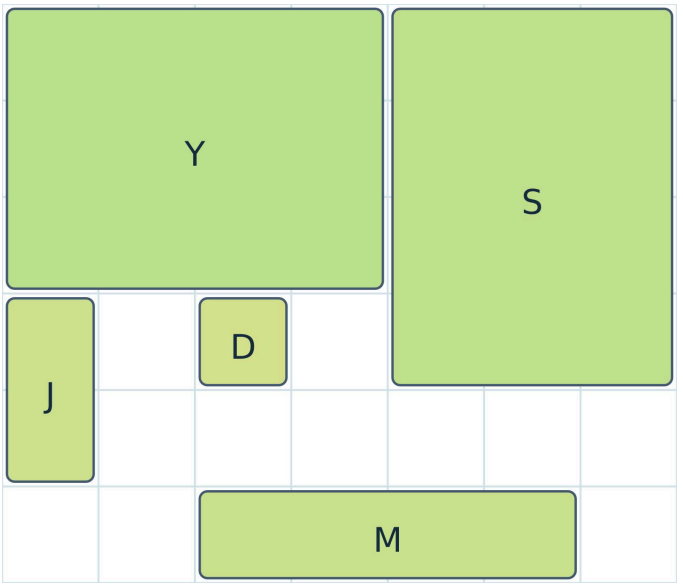
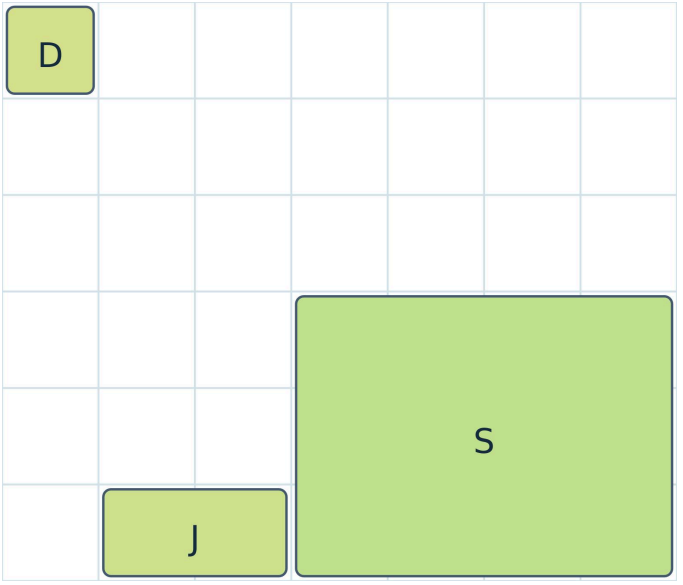
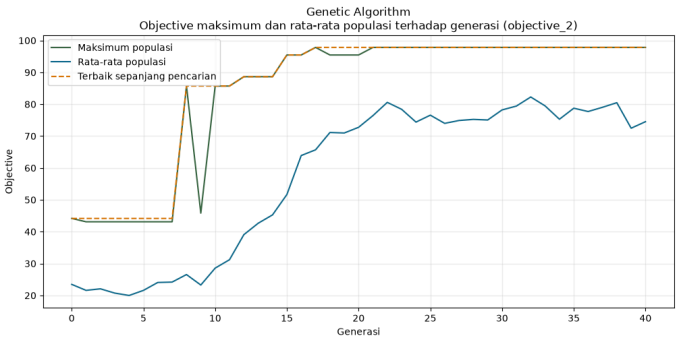
Percobaan 15

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

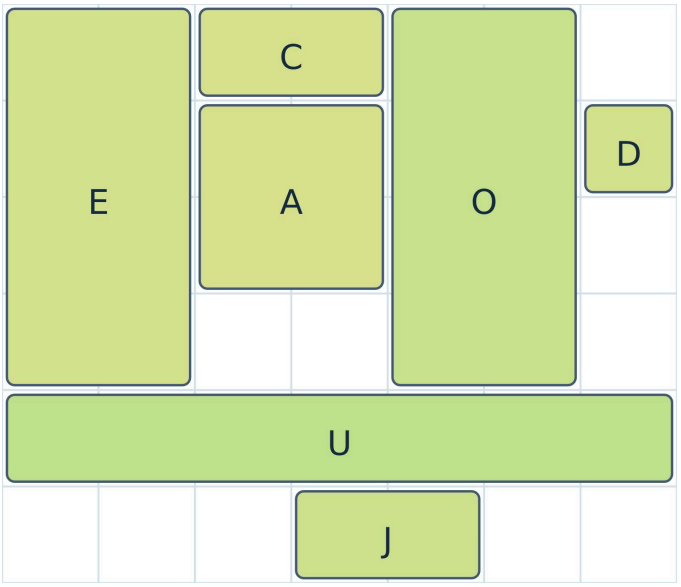
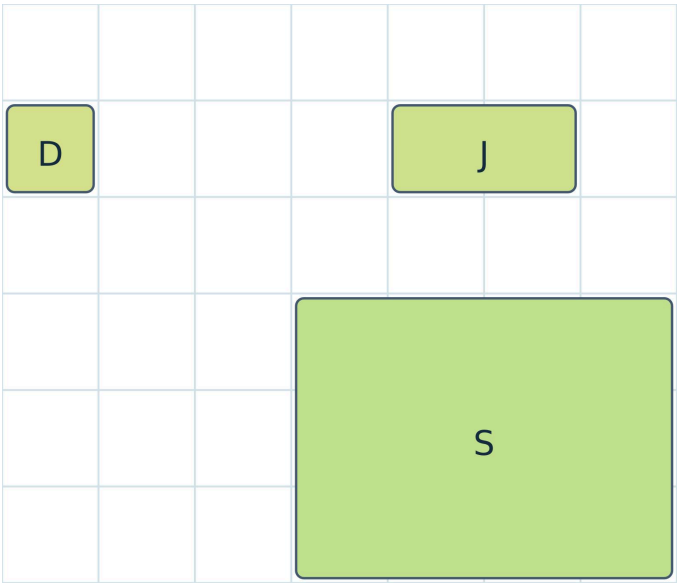
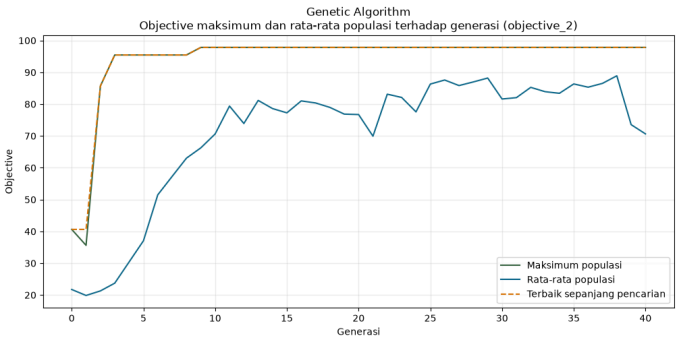
Percobaan 16

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 17

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Percobaan 18

Initial state	
Best state	
Plot *objective function* terhadap banyak iterasi	

Berdasarkan 18 percobaan yang dilakukan, pembahasan akan dikelompokkan dari parameter jumlah populasi dan jumlah generasi. Proses pengujiannya menggunakan beberapa kombinasi antara dua parameter yang dapat diamati. Percobaan 1-9 mempertahankan jumlah generasi sebanyak 20 dengan jumlah populasi

sebesar 50, 100, dan 150 individu. Pada pengujian jumlah populasi, tampak seluruh percobaan menghasilkan nilai objektif sebesar 97.83. Kesamaan hasil tersebut menunjukkan bahwa dengan 50 individu, hasil yang didapat akan sama dengan percobaan dengan 100 dan 150 individu. Penambahan individu mampu memperbanyak susunan kendaraan yang dapat dievaluasi, namun belum memberikan peningkatan nilai objektif akhir.

Pada pengujian dengan mempertahankan jumlah generasi, hasil nilai objektif menunjukkan nilai 97.83 dan 85.17. Konfigurasi 20 dan 40 generasi menghasilkan nilai 97.83 pada seluruh percobaan, sementara 10 generasi hanya mencapai nilai terbaik pada 2 dari total 3 percobaan. Penambahan generasi memberikan kesempatan eksplorasi penempatan kendaraan untuk mencapai nilai tertinggi dengan berbagai proses seleksi, *cross over*, dan mutasi. Durasi yang tampak juga berjalan linear bergantung pada jumlah generasi yang digunakan untuk mengevaluasi nilai akhir percobaan.

Konfigurasi 50 individu dan 20 generasi memberikan nilai akhir yang sama dengan konfigurasi lebih besar pada seluruh percobaan yang tercatat. Hanya saja pemenuhan nilai tertinggi hanya mencakup 2 dari 3 percobaan sehingga hasilnya belum selalu sama. Dapat disimpulkan pula bahwa pada percobaan ini, nilai 97.83 merupakan *global optimum*.

BAB III
ANALISIS

Analisis dilakukan berdasarkan hasil eksperimen pada Bab II dengan menggunakan dataset yang terdiri atas 25 kendaraan dan satu kapal. Geladak kapal berukuran tujuh kolom dan enam baris dengan kapasitas berat maksimum 15 unit. Data kendaraan dan kapal dibuat sama agar hasil setiap algoritma dapat dibandingkan. Seluruh hasil percobaan yang dibahas pada bab ini menggunakan `'objective_2'.` Fungsi tersebut memperhitungkan total tarif pengiriman, ETA maksimum kendaraan yang diangkut, dan penggunaan kapasitas berat kapal. Semakin tinggi nilai objektif yang diperoleh, semakin baik solusi berdasarkan fungsi tersebut.

Setiap algoritma selain *Genetic Algorithm* dijalankan sebanyak tiga kali dengan inisialisasi yang berbeda. *Genetic Algorithm* diuji menggunakan beberapa konfigurasi jumlah populasi dan banyak generasi. Pada perbandingan utama antar algoritma, hasil *Genetic Algorithm* yang digunakan adalah tiga percobaan dengan populasi 50 dan 20 generasi.

3.1 Kedekatan dengan Global Optimum

Untuk membandingkan hasil pencarian, dihitung rata-rata nilai objektif akhir dari tiga percobaan pada setiap algoritma. Semakin tinggi rata-rata nilai objektif yang diperoleh, semakin baik hasil pencariannya. Nilai tertinggi yang ditemukan dalam pengujian adalah 97,83 dan digunakan sebagai acuan untuk membandingkan hasil algoritma lainnya. Pada *Genetic Algorithm*, hasil yang digunakan dalam perbandingan ini adalah tiga percobaan dengan jumlah populasi 50 dan banyak generasi 20.

Tabel 3.1. Perbandingan Algoritma dan Rata-rata Nilai Objektif Akhir

Algoritma	Rata-rata nilai objektif akhir
Genetic Algorithm	97.83
Random Restart Hill-Climbing	72.50
Hill-Climbing with Sideways Move	30.02
Steepest Ascent Hill-Climbing	29.81
Stochastic Hill-Climbing	27.02
Simulated Annealing	22.66

Berdasarkan hasil percobaan, *Genetic Algorithm* memperoleh rata-rata nilai objektif paling tinggi, yaitu 97,83. Ketiga percobaan pada konfigurasi tersebut berhasil mencapai nilai yang sama. Hasil ini didukung oleh pencarian yang dilakukan terhadap banyak solusi dalam satu populasi. Melalui seleksi, crossover, dan mutasi, algoritma dapat mencoba kombinasi

kendaraan baru pada setiap generasi. Solusi terbaik yang diperoleh mengangkut kendaraan D, J, dan S dengan total tarif pengiriman 150, total berat delapan unit, dan ETA maksimum nol. Kombinasi tersebut memberikan hasil yang baik pada `objective_2` karena tarif pengiriman, ETA, dan penggunaan kapasitas berat turut diperhitungkan.

Random Restart Hill-Climbing memperoleh rata-rata nilai objektif sebesar 72.50 dan menempati urutan kedua. Nilai tersebut diperoleh pada ketiga percobaan. Algoritma menjalankan pencarian dari beberapa state awal sehingga memiliki lebih banyak kesempatan untuk menemukan susunan yang baik. Dengan maksimum sepuluh restart, terdapat sebelas rangkaian pencarian yang terdiri atas satu pencarian awal dan sepuluh pencarian tambahan. Hasil terbaik dari seluruh rangkaian kemudian dipilih sebagai solusi akhir. Mekanisme ini menghasilkan perbaikan yang cukup besar dibandingkan *Steepest Ascent* yang hanya memulai pencarian dari satu state awal.

Hill-Climbing with Sideways Move memperoleh rata-rata nilai objektif sebesar 30,02. Hasil tersebut sedikit lebih tinggi dibandingkan *Steepest Ascent* yang memperoleh 29,81. Ketika tidak menemukan neighbor dengan nilai objektif lebih tinggi, algoritma masih dapat berpindah ke neighbor yang memiliki nilai sama. Perubahan susunan tersebut memberikan kesempatan untuk menemukan peningkatan pada langkah berikutnya. Pada percobaan pertama dan ketiga, *Sideways Move* menghasilkan nilai objektif yang sedikit lebih tinggi daripada *Steepest Ascent.* Namun, ketiga percobaan tetap mencapai batas 100 iterasi dengan jumlah sideways move sebanyak 95, 96, dan 95. Hal ini menunjukkan bahwa sebagian besar langkah pencarian digunakan untuk berpindah di antara susunan dengan nilai objektif yang sama.

Steepest Ascent Hill-Climbing memperoleh rata-rata nilai objektif sebesar 29,81. Pada setiap iterasi, algoritma memeriksa seluruh neighbor dan memilih neighbor dengan nilai objektif tertinggi yang lebih baik daripada state saat ini. Ketiga percobaan berhenti setelah lima iterasi karena tidak ditemukan lagi neighbor yang dapat meningkatkan nilai objektif secara langsung. Aturan tersebut membuat pencarian cepat berhenti ketika mencapai local maximum atau bagian datar yang tidak memiliki neighbor lebih baik. Karena tidak menerima perpindahan dengan nilai yang sama maupun lebih rendah, algoritma memiliki kesempatan yang terbatas untuk mencoba susunan lain setelah kondisi tersebut tercapai.

Stochastic Hill-Climbing memperoleh rata-rata nilai objektif sebesar 27.02. Hasil ini lebih rendah dibandingkan *Steepest Ascent* karena pemilihan satu neighbor secara acak tidak selalu menghasilkan kandidat dengan peningkatan terbesar. Algoritma hanya menerima perpindahan jika neighbor yang terpilih memiliki nilai objektif lebih tinggi. Jika kandidat memiliki nilai yang sama atau lebih rendah, state saat ini dipertahankan dan pencarian dilanjutkan. Ketiga percobaan berjalan hingga batas 100 iterasi, tetapi tambahan iterasi tersebut belum menghasilkan solusi sebaik *Steepest Ascent.* Hasil ini menunjukkan bahwa

jumlah iterasi yang lebih banyak belum tentu memberikan nilai objektif yang lebih tinggi ketika pemilihan neighbor dilakukan secara acak.

Simulated Annealing memperoleh rata-rata nilai objektif terendah, yaitu 22,66. Pada percobaan pertama dan kedua, solusi terbaik yang disimpan masih berupa state awal. Pada percobaan ketiga, nilai objektif terbaik hanya meningkat dari sekitar 18,94 menjadi 19,59. Algoritma menggunakan temperatur awal 1000 dan cooling rate 0,95 sehingga masih dapat menerima perpindahan ke state dengan nilai objektif lebih rendah. Namun, perpindahan yang dilakukan belum diikuti oleh penemuan solusi yang jauh lebih baik. Sebagian besar kandidat neighbor juga memiliki nilai objektif yang sama dengan state saat itu. Dengan batas 100 iterasi, pencarian berakhir ketika temperatur masih sekitar 5,92. Oleh karena itu, hasil yang diperoleh menunjukkan bahwa konfigurasi tersebut belum memberikan peningkatan yang besar pada data pengujian.

3.2 Perbandingan Hasil Pencarian

Berdasarkan rata-rata nilai objektif pada bagian 3.1, diperoleh urutan hasil pencarian dari yang tertinggi hingga terendah sebagai berikut :

1. *Genetic Algorithm*
2. *Random Restart Hill-Climbing*
3. *Hill-Climbing with Sideways Move*
4. *Steepest Ascent Hill-Climbing*
5. *Stochastic Hill-Climbing*
6. *Simulated Annealing*

Genetic Algorithm memberikan hasil pencarian terbaik, diikuti oleh *Random Restart Hill-Climbing*. Keduanya memperoleh rata-rata nilai objektif yang lebih tinggi dibandingkan algoritma lainnya. Sementara itu, hasil *Hill-Climbing* with *Sideways Move* hanya sedikit lebih tinggi daripada *Steepest Ascent Hill-Climbing*. *Stochastic Hill-Climbing* dan *Simulated Annealing* memperoleh rata-rata nilai objektif yang lebih rendah pada percobaan ini. Perbedaan hasil tersebut dipengaruhi oleh cara setiap algoritma melakukan pencarian. *Genetic Algorithm* membentuk kombinasi solusi melalui seleksi, crossover, dan mutasi, sedangkan *Random Restart* mengulang pencarian dari beberapa state awal. Pada algoritma lainnya, hasil pencarian dipengaruhi oleh pemilihan neighbor dan aturan penerimaan perpindahan. Selain itu, parameter seperti batas iterasi, maksimum *sideways move*, maksimum *restart*, temperatur, dan cooling rate turut mempengaruhi kesempatan algoritma untuk menemukan solusi yang lebih baik.

3.3 Perbandingan Durasi Proses Pencarian

Durasi pencarian menunjukkan perbedaan biaya komputasi untuk memperoleh hasil pada bagian sebelumnya. Tabel berikut merupakan rincian durasi proses pencarian untuk setiap algoritma terurut dari rata-rata durasi tercepat hingga terlama.

Tabel 3.3. Perbandingan Algoritma dan Rentang serta Rata-rata Durasi

Algoritma	Rentang durasi (*s*)	Rata-rata durasi (*s*)
Stochastic Hill Climbing	0,5351 - 0,5391	0.5367
Steepest Ascent Hill Climbing	0.6798 - 0.7268	0.6971
Hill Climbing with Sideways Move	10.4481 - 11.5085	10.9341
Random Restart Hill Climbing	11.3232 - 12.1217	11.6965
Genetic Algorithm (20 generasi, 50 populasi)	17.0887 - 19.0477	17.9400
Simulated Annealing	22.82839 - 24.94490	24.0728

Stochastic Hill CLimbing merupakan algoritma tercepat dengan rata-rata 0,5367 detik, meskipun menjalankan 100 iterasi. *Steepest Ascent Hill Climbing* hanya menjalankan lima iterasi, tetapi memerlukan rata-rata 0,6971 detik karena mengevaluasi seluruh *neighbor* pada setiap putaran. Perbandingan ini menunjukkan bahwa jumlah iterasi tidak dapat langsung digunakan untuk menentukan algoritma yang lebih cepat. Banyaknya kandidat yang dibentuk dan dievaluasi dalam satu iterasi juga menentukan durasi pencarian.

Hill Climbing with Sideways Move memerlukan rata-rata 10,9341 detik karena terus mengevaluasi seluruh neighbor hingga 100 iterasi, termasuk saat menerima perubahan bernilai sama. Durasinya sekitar 15,69 kali durasi *Steepest Ascent* sedangkan kenaikan rata-rata skor akhirnya hanya sekitar 0,21 poin. Pada konfigurasi ini, penambahan waktu untuk melewati *plateau* belum diimbangi peningkatan kualitas solusi yang besar.

Random Restart Hill Climbing memerlukan rata-rata 11,6965 detik untuk menjalankan satu pencarian awal dan sepuluh pencarian ulang. Total iterasi yang tercatat adalah 50, 48, dan 48, tetapi durasinya juga mencakup pembentukan state awal baru. Dibandingkan dengan *Sideways Move*, rata-rata durasinya hanya sekitar 6,97% lebih panjang, sementara rata-rata skor akhirnya meningkat dari 30,02 menjadi 72,50. Dengan demikian, pada eksperimen ini, pengulangan dari state awal yang berbeda memberi peningkatan kualitas hasil yang lebih besar daripada melanjutkan perpindahan bernilai sama selama banyak iterasi.

Genetic Algorithm pada konfigurasi utama membutuhkan rata-rata 17,9400 detik untuk menghasilkan skor 97,83. Biaya tersebut berasal dari pembentukan dan evaluasi populasi, pemilihan *parent*, *crossover*, serta mutasi. Pada kode yang ditampilkan, mutasi menelusuri *neighbor* untuk memilih satu kandidat secara acak, sehingga biaya prosesnya tidak terbatas pada menghitung fitness seluruh individu. Penggunaan populasi juga tidak berarti proses dilakukan secara paralel dimana implementasi yang ditampilkan membentuk keturunan melalui perulangan. Walaupun lebih lama daripada *Random Restart*, konfigurasi *Genetic Algorithm* menghasilkan skor akhir yang lebih tinggi.

Simulated Annealing merupakan algoritma terlama dalam perbandingan utama, dengan rata-rata 24,0728 detik, tetapi menghasilkan rata-rata skor akhir terendah. Hal ini memperlihatkan bahwa waktu pencarian yang lebih panjang tidak menjamin kualitas solusi yang lebih baik. Jika seluruh variasi *Genetic Algorithm ikut dipertimbangkan, durasinya berada pada rentang 8,7227–55,9050 detik. Karena itu, p*ernyataan bahwa Simulated Annealing paling lama hanya berlaku terhadap konfigurasi utama yang tercantum pada tabel, bukan terhadap semua konfigurasi Genetic Algorithm.

3.4 Konsistensi Hasil Akhir Eksperimen

Konsistensi hasil akhir dianalisis menggunakan rata-rata, standar deviasi sampel, dan koefisien variasi dari nilai objektif akhir. Rata-rata menunjukkan tingkat kualitas solusi yang diperoleh, sedangkan standar deviasi menunjukkan penyebaran hasil antarpercobaan terhadap rata-ratanya. Berbeda dengan rentang yang hanya menggunakan nilai tertinggi dan terendah, standar deviasi memperhitungkan seluruh nilai dalam kelompok percobaan. Semakin kecil standar deviasi, semakin seragam hasil yang dicatat dalam satuan skor objektif.

Rata-rata dan standar deviasi sampel dihitung menggunakan persamaan berikut:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}$$

Dalam persamaan tersebut, x_i merupakan nilai objektif akhir percobaan ke-i, $\bar{x}$ merupakan rata-rata nilai objektif akhir, dan n merupakan jumlah percobaan dalam kelompok. Penyebut $n - 1$ digunakan karena hasil percobaan diperlakukan sebagai sampel dari kemungkinan pengulangan pencarian. Nilai statistik dihitung dari skor yang dilaporkan pada tabel eksperimen sehingga ketelitiannya mengikuti pembulatan data sumber yang digunakan.

Koefisien variasi digunakan untuk melengkapi pengukuran tersebut dengan menunjukkan besar standar deviasi relatif terhadap rata-rata:

$$CV = \frac{s}{\bar{x}} \times 100\%$$

CV yang lebih kecil menunjukkan variasi relatif yang lebih rendah pada kelompok yang diamati. Ukuran ini membantu membandingkan algoritma dengan rata-rata skor yang berbeda namun tetap tidak dapat menggantikan penilaian terhadap kualitas solusi yang dihasilkan algoritma pencarian. Interpretasinya juga berlaku pada skala *objective function* yang digunakan dalam eksperimen ini.

Untuk *Genetic Algorithm*, perhitungan dilakukan secara terpisah pada setiap konfigurasi jumlah generasi dan populasi. Pengelompokan ini diperlukan agar variasi antar percobaan tidak tercampur dengan pengaruh perubahan parameter yang telah dilakukan. Konfigurasi 20 generasi dan populasi 50 muncul pada dua rangkaian pengujian, yaitu percobaan 1-3 dan 13-15. Setiap baris pada tabel berikut memuat tiga percobaan sehingga menggunakan $n = 3$ kecuali untuk konfigurasi *Genetic Algorithm* yang menggunakan $n = 6$. Berikut rincian untuk rata-rata, standar deviasi, dan koefisien variasi untuk setiap algoritma pencarian.

Tabel 3.4 Perbandingan Algoritma dan Standar Deviasi serta Koefisien Sampel

Algoritma	Rata-rata	Standar deviasi sampel	Koefisien variasi
Steepest Ascent Hill Climbing	29.81	6.36	21.32%
Stochastic Hill Climbing	27.02	4.35	16.08%
Hill Climbing with Sideways Move	30.02	6.18	20.59%
Random Restart Hill Climbing	72.50	0.00	0.00%
Simulated Annealing	22.66	3.06	13.50%
Genetic Algorithm (10 generasi, 50 populasi)	93.61	7.31	7.81%
Genetic Algorithm (20 generasi, 50 populasi)	97.83	0.00	0.00%
Genetic Algorithm (20 generasi, 100 populasi)	97.83	0.00	0.00%
Genetic Algorithm (20 generasi, 150 populasi)	97.83	0.00	0.00%
Genetic Algorithm (40 generasi, 50 populasi)	97.83	0.00	0.00%

Random Restart menghasilkan rata-rata 72,50 dengan standar deviasi dan CV sebesar nol. Hal ini mengartikan ketiga percobaan menghasilkan skor akhir yang sama pada presisi

pelaporan. Pengulangan pencarian dari beberapa state awal memberikan kesempatan untuk menemukan solusi pada bagian ruang pencarian yang berbeda, dan pada ketiga percobaan yang dicatat hasil terbaiknya sama.

Steepest Ascent memiliki standar deviasi 6,36 dan CV 21,32%, sedangkan *Sideways Move* memiliki standar deviasi 6,18 dan CV 20,59%. Di antara lima algoritma selain *Genetic Algorithm*, keduanya menunjukkan penyebaran absolut dan relatif terbesar. *Sideways Move* sedikit meningkatkan rata-rata dari 29,81 menjadi 30,02 sekaligus sedikit menurunkan penyebarannya. Namun, perbedaan tersebut masih kecil pada tiga percobaan ini, sehingga belum cukup untuk menyatakan bahwa sideways move secara umum meningkatkan konsistensi. Variasi hasil yang masih besar menunjukkan bahwa perbedaan kondisi awal tetap berkaitan dengan perbedaan solusi akhir.

Stochastic menghasilkan rata-rata 27,02 dengan standar deviasi 4,35 dan CV 16,08%. Hasil yang diberikan lebih seragam daripada *Steepest Ascen*t dan *Sideways Move* berdasarkan kedua ukuran penyebaran, tetapi rata-rata kualitas solusinya juga lebih rendah. Dengan demikian, pemilihan *neighbor* secara acak tidak otomatis menghasilkan variasi akhir terbesar. Konsistensi perlu dinilai dari data yang diperoleh, bukan hanya dari ada atau tidaknya unsur pengacakan dalam algoritma.

Simulated Annealing memiliki rata-rata 22,66, standar deviasi 3,06, dan CV 13,50%. Penyebarannya lebih kecil daripada *Steepest Ascent*, *Sideways Move*, dan *Stochastic* namun terjadi pada tingkat skor yang paling rendah. Dua percobaan tidak memperbaiki nilai terbaik dari state awal, sementara percobaan ketiga hanya meningkat dari 18,94 menjadi 19,59. Oleh karena itu, variasi yang relatif kecil pada *Simulated Annealing* tidak menunjukkan keberhasilan memperoleh solusi berkualitas tinggi secara berulang, melainkan hasil akhir yang sama-sama masih dekat dengan skor awal.

Pada *Genetic Algorithm* dengan 10 generasi dan populasi 50, nilai akhirnya adalah 97,83, 85,17, dan 97,83. Selain itu, algoritma ini memiliki rata-rata sebesar 93,61, standar deviasi 7,31, dan CV-nya 7,81%. Standar deviasi absolut konfigurasi ini lebih tinggi daripada *Steepest Ascent*, tetapi CV-nya lebih rendah karena rata-rata skornya jauh lebih tinggi. Perbandingan tersebut menunjukkan manfaat membaca kedua metrik bersama-sama: penyebaran dalam satuan skor lebih besar, sedangkan penyebaran relatif terhadap rata-rata lebih kecil. Satu hasil yang lebih rendah pada percobaan ke-11 menunjukkan bahwa 10 generasi belum menghasilkan skor pembanding secara seragam pada seluruh percobaan yang dicatat.

Seluruh kelompok *Genetic Algorithm* dengan 20 atau 40 generasi menghasilkan rata-rata 97,83, standar deviasi nol, dan CV nol. Pada kelompok tersebut, skor yang tinggi disertai hasil akhir yang seragam. Penambahan generasi dari 10 menjadi 20 pada populasi 50 berhubungan dengan peningkatan rata-rata dari 93,61 menjadi 97,83 dan penurunan standar

deviasi dari 7,31 menjadi nol. Sebaliknya, peningkatan populasi dari 50 menjadi 100 atau 150 pada 20 generasi, maupun peningkatan generasi dari 20 menjadi 40 pada populasi 50, tidak lagi memperbaiki keseragaman skor akhir karena kelompok pembandingnya sudah memiliki standar deviasi nol.

Secara keseluruhan, 17 dari 18 catatan *Genetic Algorithm* menghasilkan skor 97,83. Namun, seluruh catatan tersebut tidak digabungkan untuk menghitung satu standar deviasi sebagai ukuran konsistensi pada konfigurasi tetap, karena parameter pengujiannya berbeda dan terdapat konfigurasi yang dicatat kembali. Proporsi tersebut hanya merangkum catatan eksperimen, bukan estimasi peluang keberhasilan dari 18 pengulangan independen dengan kondisi identik.

Seluruh ukuran pada bagian ini merupakan statistik deskriptif dari tiga percobaan per kelompok. Perbedaan *state* awal dan pengacakan pencarian sama-sama dapat menyumbang variasi hasil, sehingga penyebaran yang dihitung tidak mengisolasi pengaruh pengacakan algoritma saja. Standar deviasi nol juga hanya menunjukkan kesamaan skor pada data dan presisi yang dilaporkan. Pengulangan dengan lebih banyak *seed* independen diperlukan untuk menilai apakah pola konsistensi tersebut bertahan di luar percobaan ini. Berdasarkan data yang tersedia, *Random Restart* serta konfigurasi *Genetic Algorithm* dengan 20 atau 40 generasi menunjukkan hasil paling seragam, dengan *Genetic Algorithm* memberikan tingkat skor akhir yang lebih tinggi.

3.5 Pengaruh Jumlah Populasi dan Banyak Generasi pada *Genetic Algorithm*

Pada percobaan yang dilakukan, pengaruh jumlah populasi dan banyak generasi diamati melalui nilai objektif akhir, grafik pencarian, dan durasi proses pencarian. Pengujian jumlah populasi menggunakan 50, 100, dan 150 individu dengan banyak generasi tetap sebanyak 20. Sementara itu, pengujian banyak generasi menggunakan 10, 20, dan 40 generasi dengan jumlah populasi tetap sebanyak 50 individu. Setiap konfigurasi dijalankan sebanyak tiga kali dengan crossover rate sebesar 0.8 dan mutation rate sebesar 0.2.

Pada pengujian banyak generasi, konfigurasi 10 generasi menghasilkan rata-rata nilai objektif akhir sebesar 93.79. Dua percobaan memperoleh nilai 97.83, sedangkan satu percobaan memperoleh 85.71. Ketika banyak generasi ditambah menjadi 20, ketiga percobaan berhasil memperoleh nilai 97.83. Hal ini menunjukkan bahwa penambahan generasi memberikan kesempatan lebih panjang bagi algoritma untuk membentuk kombinasi kendaraan melalui seleksi, crossover, dan mutasi. Pada percobaan yang belum memperoleh hasil tertinggi dalam sepuluh generasi, pencarian dapat dilanjutkan hingga menemukan solusi yang lebih baik. Namun, penambahan menjadi 40 generasi tidak meningkatkan nilai objektif akhir. Ketiga percobaan tetap memperoleh nilai 97,83 sehingga rata-ratanya sama dengan konfigurasi 20 generasi. Perbedaan terlihat pada rata-rata durasi pencarian yang meningkat

dari sekitar 17,68 detik pada 20 generasi menjadi 35,80 detik pada 40 generasi. Pada pengujian ini, solusi terbaik sudah ditemukan sebelum pencarian mencapai 20 generasi. Oleh karena itu, tambahan generasi berikutnya menambah waktu pencarian tanpa memberikan peningkatan terhadap solusi terbaik yang disimpan.

Pada pengujian jumlah populasi, populasi 50, 100, dan 150 individu menghasilkan rata-rata nilai objektif akhir yang sama, yaitu 97,83. Hasil tersebut menunjukkan bahwa populasi 50 sudah memberikan kesempatan yang cukup untuk menemukan solusi terbaik pada data dan batas 20 generasi yang digunakan. Penambahan populasi memperbanyak individu yang diproses, tetapi belum menghasilkan nilai objektif akhir yang lebih tinggi. Pengaruh penambahan populasi lebih terlihat pada durasi pencarian. Rata-rata durasi untuk populasi 50 adalah sekitar 17,94 detik, kemudian meningkat menjadi 36,39 detik pada populasi 100 dan 54,28 detik pada populasi 150. Peningkatan waktu tersebut terjadi karena semakin banyak individu yang harus dibentuk dan dievaluasi pada setiap generasi. Dengan hasil akhir yang sama, populasi 50 membutuhkan waktu paling singkat di antara ketiga ukuran populasi yang diuji.

Perubahan nilai pada grafik juga perlu dibedakan antara nilai maksimum populasi, rata-rata populasi, dan nilai terbaik selama pencarian. Nilai maksimum dan rata-rata populasi dapat naik atau turun ketika generasi berganti karena populasi berikutnya dibentuk kembali melalui seleksi, crossover, dan mutasi. Implementasi yang digunakan tidak menerapkan elitism sehingga individu terbaik tidak otomatis dimasukkan ke generasi berikutnya. Meskipun demikian, program tetap menyimpan solusi terbaik yang pernah ditemukan. Oleh karena itu, penurunan nilai maksimum populasi pada suatu generasi tidak menghilangkan solusi terbaik yang sudah disimpan sebelumnya.

Berdasarkan hasil percobaan, konfigurasi populasi 50 dan 20 generasi memberikan hasil yang baik dengan waktu pencarian lebih singkat dibandingkan konfigurasi populasi yang lebih besar atau generasi yang lebih banyak. Penggunaan 10 generasi membutuhkan waktu lebih singkat, tetapi belum menghasilkan nilai tertinggi pada seluruh percobaan. Sementara itu, penambahan populasi hingga 150 individu atau penambahan hingga 40 generasi belum memberikan peningkatan nilai objektif akhir pada data yang digunakan.

3.6 Pengaruh Temperatur Penerimaan Neighbor pada *Simulated Annealing*

Temperatur pada *Simulated Annealing* secara eksplisit memengaruhi hasil penerimaan *neighbor* yang memiliki nilai objektif lebih rendah dibanding state saat ini. Peluang diuraikan menggunakan rumus probabilitas $P = e^{\frac{delta\ E}{T}}$, dengan delta E merupakan nilai objektif *neighbor* dikurangi nilai objektif state saat ini. Untuk penurunan nilai objektif yang sama, temperatur lebih tinggi menghasilkan peluang penerimaan lebih besar. *Neighbor* dengan nilai objektif sama atau lebih tinggi akan tetap diterima tanpa bergantung pada temperatur.

Pada pengujian ini, temperatur awal ditetapkan sebesar 1000 dengan *cooling rate* sebesar 0.95. Temperatur minimum ada di 0.001 dan batas pencarian iterasi sebesar 100 kali. Untuk setiap iterasi, temperatur akan dikalikan dengan *cooling rate* mengikuti hubungan:

$$Tk = 1000(0.95)^{k}$$

Dengan *k* merujuk pada jumlah iterasi. Setelah 100 iterasi, temperatur akan berada di nilai ± 5.92, yang artinya nilai masih berada diatas temperatur minimum. Dengan demikian, hasil percobaan ini akan berhenti diluar batasan temperatur minimum terjadi. Pengaruh temperatur perlu dilihat terhadap besarnya selisih nilai objektif, Pada temperatur terkecil yang mungkin pada iterasi maksimum 100, peluang penurunan yang sama menjadi sekitar 42,98%. Hal ini membuat pendinginan mengurangi peluang penerimaan solusi yang lebih buruk di setiap iterasi yang berlangsung terus-menerus.

Pada percobaan *Simulated Annealing* yang merujuk pada bab sebelumnya, percobaan pertama dan kedua menunjukkan solusi terbaik dengan nilai objektif sama seperti state awal. Sementara percobaan ketiga memperoleh peningkatan menjelang akhir pencarian. Hasil ini menjadi bukti bahwa perpindahan susunan ke skor lebih rendah bisa memiliki kesempatan penemuan solusi yang melampaui nilai awal, meski tidak berlaku secara pasti. Ketiga percobaan dengan pengaturan sama ini membuktikan bahwa perilaku penerimaan *neighbor* pada batas terdefinisi belum cukup untuk menentukan temperatur awal dan *cooling rate* yang ideal. Perbandingan perlu dilakukan dengan pengulangan memadai untuk menilai apakah pendinginan berbeda dapat menghasilkan peningkatan nilai objektif yang lebih baik dan konsisten.

3.7 Batasan Analisis

Batasan analisis yang didefinisikan mencakup batasan data, parameter, dan jumlah iterasi pada percobaan yang dilakukan. Hasil yang diperoleh menunjukkan kinerja algoritma pada konfigurasi pengujian tersebut dan belum dapat digeneralisasikan untuk seluruh permasalahan pemuatan geladak kapal atau *geometric knapsack problem*. Oleh karenanya, perbedaan nilai objektif perlu dipahami sebagai hasil gabungan antara karakteristik algoritma, state awal, proses randomisasi, dan batas pencarian yang didefinisikan.

Pada algoritma *Hill Climbing Steepest Ascent,* pencarian berhenti ketika tidak ditemukan *neighbor* dengan nilai objektif lebih tinggi. Hasil akhirnya dipengaruhi oleh susunan awal dan bentuk perubahan yang diperbolehkan dalam pembentukan *neighbor*. Meskipun seluruh *neighbor* diperiksa pada setiap iterasi, pemeriksaan tersebut hanya mencakup perubahan langsung dari state saat ini. Sementara pada *Stochastic Hill Climbing* yang dibatasi hingga 100 iterasi, proses pengacakan terhadap pemilihan *neighbor* hanya dilakukan satu kali setiap iterasi. Batas tersebut belum tentu memberikan kesempatan yang cukup bagi pencarian *neighbor* yang berpotensi menghasilkan peningkatan. Pada *Hill*

Climbing with Sideways Move, batas iterasi dan jumlah maksimum *sideways move* juga berhenti di masing-masing maksimal 100 jumlah *sideways move* dan iterasi. Seluruh percobaan mencapai batas iterasi dengan *sideways move* sebanyak 95 dan 96, sehingga pencarian selesai sebelum batas tercapai. Pada *Random Restart*, jumlah *restart* dibatasi sebanyak sepuluh, sehingga terdapat total sebelas pencarian termasuk pencarian awal. Kesamaan nilai akhir menunjukkan hasil konsisten namun belum cukup membuktikan bahwa jumlah *restart* telah mencakup seluruh ruang pencarian yang memadai dalam menghasilkan *global optimum*.

Pada *Simulated Annealing*, seluruh percobaan menggunakan temperatur awal 1000, *cooling rate* 0.95, temperatur minimum 0.001, dan batas maksimum 100 iterasi. Akibat temperatur awal dan *cooling rate* dipertahankan, pengujian tidak membandingkan pengaruh dua parameter ini terhadap ketercapaian objektif. Pencarian juga berakhir ketika temperatur masih berada di angka ±5.92 setelah mengalami iterasi maksimum, artinya angka ini pun belum mencapai temperatur minimum yang didefinisikan dengan nilai 0.001.

Pada *Genetic Algorithm*, keterbatasan berada pada penguraian parameter yang mempertahankan antara jumlah populasi atau jumlah generasi. Pengujian yang dilakukan sebanyak 18 kali ini belum mencakup seluruh kombinasi kedua parameter, sehingga interaksi antara populasi dan generasi belum dapat dinilai efektivitasnya secara eksak. Kesamaan nilai akhir pada beberapa konfigurasi menunjukkan bahwa penambahan populasi atau generasi belum memberikan peningkatan dalam percobaan yang tercatat.

BAB IV
KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan hasil percobaan menggunakan `objective_2`, ***Genetic Algorithm*** dengan populasi 50 dan 20 generasi memperoleh hasil pencarian terbaik dengan rata-rata nilai objektif akhir sebesar 97,83. ***Random Restart Hill-Climbing*** menempati urutan berikutnya dengan rata-rata sebesar 72,50 sekaligus menjadi varian *Hill-Climbing d*engan hasil terbaik. Selanjutnya, urutan hasil pencarian diikuti oleh ***Hill-Climbing with Sideways Move***, ***Steepest Ascent Hill-Climbing, Stochastic Hill-Climbing,*** dan ***Simulated Annealing.*** *Genetic Algorithm* pada konfigurasi tersebut dan *Random Restart* juga menghasilkan nilai objektif akhir yang sama pada ketiga percobaan.

Dari segi durasi, *Stochastic Hill-Climbing* dan *Steepest Ascent Hill-Climbing* memiliki waktu pencarian paling singkat, dengan rata-rata masing-masing sekitar 0,5367 dan 0,6971 detik. *Hill-Climbing with Sideways Move* membutuhkan waktu lebih panjang daripada *Steepest Ascent,* tetapi peningkatan nilai objektifnya masih kecil. Sementara itu, *Random Restart* membutuhkan waktu lebih lama untuk mengulang pencarian dari beberapa state awal dan memperoleh peningkatan hasil yang lebih besar. *Simulated Annealing* memiliki durasi terpanjang dalam perbandingan konfigurasi utama, tetapi menghasilkan rata-rata nilai objektif terendah. Hasil tersebut menunjukkan bahwa waktu pencarian yang lebih panjang tidak selalu menghasilkan solusi yang lebih baik.

Pada *Genetic Algorithm,* penambahan dari 10 menjadi 20 generasi meningkatkan rata-rata nilai objektif akhir dan membuat ketiga percobaan memperoleh hasil yang sama. Namun, penambahan menjadi 40 generasi tidak meningkatkan hasil akhir. Penambahan populasi dari 50 menjadi 100 atau 150 individu juga menghasilkan nilai objektif akhir yang sama dengan durasi lebih panjang. Oleh karena itu, konfigurasi populasi 50 dan 20 generasi memberikan hasil yang baik dengan waktu pencarian lebih singkat dibandingkan konfigurasi yang lebih besar pada data pengujian.

Secara keseluruhan, hasil pencarian dipengaruhi oleh susunan awal, cara pemilihan neighbor, aturan perpindahan, dan parameter algoritma. Pemilihan algoritma perlu mempertimbangkan kualitas solusi serta waktu pencarian yang dibutuhkan. Pada pengujian ini, *Genetic Algorithm* memberikan nilai objektif tertinggi, sedangkan *Stochastic* dan *Steepest Ascent* memberikan waktu pencarian paling singkat.

4.2 Saran

Berdasarkan hasil pengujian, untuk memenuhi waktu pencarian yang lebih singkat, *Stochastic* dan *Steepest Ascent Hill Climbing* dapat dipilih karena memiliki durasi paling singkat dalam pengujian. Sementara *Random Restart Hill Climbing* dapat dipilih untuk memperoleh nilai objektif lebih tinggi dibandingkan algoritma *Hill Climbing* yang lain, meski dengan waktu pencarian lebih

lama. Pengembangan selanjutnya disarankan berfokus pada efisiensi proses pencarian, terutama pada beberapa algoritma yang kondisi penghentiannya telah didefinisikan sejak awal. Pada *Genetic Algorithm*, pencarian dapat dihentikan lebih awal jika nilai objektif terbaik telah didapatkan dan nilainya tetap sama selama beberapa generasi. Sementara pada *Simulated Annealing* diperlukan penilaian pengaruh parameter terhadap kualitas solusi dan durasi yang dibutuhkan. Pengujian juga perlu diperluas dengan data yang lebih beragam dengan jumlah pengulangan granular.

BAB V
PEMBAGIAN TUGAS

Nama	NIM	Tugas
Abel Gani	18224016	1. Penyusunan laporan akhir 2. Implementasi algoritma *Simulated Annealing* 3. Implementasi algoritma *Genetic Algorithm*
Kayta Rechia Mazaya	18224022	1. Implementasi algoritma *Steepest Hill Climbing* 2. Implementasi algoritma *Stochastic Hill Climbing* 3. Implementasi algoritma *Hill Climbing with Sideways Move* 4. Implementasi algoritma *Random Restart Hill-Climbing* 5. Penyusunan laporan akhir
R. Athalla Ananda Putra	18224070	1. Perancangan arsitektur program 2. Implementasi representasi persoalan (core) 3. Pemodelan fungsi objektif utama dan bonus 4. Validasi konfigurasi serta integrasi sistem

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Abel Gani / 18224016
: Kayta Rechia Mazaya / 18224022
: R. Athalla Ananda Putra / 18224060
Sekolah : STEI - K / Sistem Teknologi Informasi
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Intelegensi Artifisial
Judul Tugas : Tugas Besar 1 – Optimalisasi *Local Search* dalam Permasalahan Pemuatan Kapal dalam Geladak Kargo

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Menguraikan beberapa algoritma dan mencari sumber pada bagian deskripsi persoalan untuk *generic knapsack problem*
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Membantu memahami dinamika masalah dan menguraikannya
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya	Menghasilkan gambar 1.2 untuk menguraikan permasalahan yang diangkat pada tugas besar ini secara lebih kontekstual
Lainnya, Jelaskan:		
Proses diskusi dan pembenahan untuk struktur kode yang ada pada *Simulated Annealing* dan *Genetic Algorithm*		Dilakukan dengan memberikan struktur/kerangka fungsi dan parameter awal beserta implementasi kodenya. Meminta saran perbaikan terkait hal yang kurang dan mengikuti perbaikan, proses dilakukan secara iteratif.

Proses diskusi perbaikan tata bahasa dan struktur pada file README.md	Penggunaan AI dilakukan dengan memberikan draf awal README.md lalu meminta saran perbaikan untuk menghasilkan struktur file markdown yang baik.
Membuat *test case* untuk pengecekan kode implementasi pada algoritma yang sudah dibuat	Penggunaan AI untuk generate kode *test case* untuk melakukan pengecekan pada berbagai parameter serta batasan yang didefinisikan.

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Abel Gani

18224016



Kayta Rechia Mazaya

18224022



R. Athalla Ananda Putra

18224060

REFERENSI

Beyond classical search. Slide perkuliahan IF3070 Dasar Inteligensi Artifisial. (2026). Institut Teknologi Bandung.

Buchem, M., Deuker, P., & Wiese, A. (2024). Approximating the geometric knapsack problem in near-linear time and dynamically. In W. Mulzer & J. M. Phillips (Eds.), *40th International Symposium on Computational Geometry (SoCG 2024)* (Vol. 293, pp. 26:1–26:14). Schloss Dagstuhl–Leibniz-Zentrum für Informatik. https://doi.org/10.4230/LIPIcs.SoCG.2024.26

Galvez, W., Grandoni, F., Heydrich, S., Ingala, S., Khan, A., & Wiese, A. (2017). *Approximating geometric knapsack via L-packings* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1711.07710

Rapine, C., Pedroso, J. P., & Akbalik, A. (2022). The two-dimensional knapsack problem with splittable items in stacks. *Omega, 112*, Article 102692. https://www.sciencedirect.com/science/article/abs/pii/S0305048322000998

LAMPIRAN